

网络设备的Web服务安全研究

罗夏朴

电子计算学系
计算机及数学科学学院
香港理工大学

背景 - 网络设备Web接口安全与技术瓶颈

- 首要攻击面： 物联网（IoT）及网络设备的Web接口（如路由器管理面板）是对外暴露的主要通信接口，极易成为命令注入、缓冲区溢出等污点型漏洞的攻击目标
- 传统分析工具（静态分析/Fuzzing）在IoT Web固件上面临四大挑战：
 1. 入口识别难： 缺乏源码，传统工具难以定位后端二进制程序真实的污点源
 2. 数据流极其复杂： 现代固件高度解耦，数据流常因多进程通信（如NVRAM、环境变量）或多语言混合架构（C与Lua等脚本交互）而截断
 3. 隐藏接口泛滥： 厂商遗留的测试接口缺乏前端入口且无认证保护，传统爬虫无法触达，成为严重后门
 4. 漏洞验证困难： 纯静态分析告警海量且误报极高；而动态Fuzzing由于硬件依赖难以运行固件，且在黑盒设备上缺乏代码覆盖率反馈

维度一：输入源点识别 (Source)

- 核心痛点：在后端二进制中盲目寻找输入点 (Source) 会引入大量无效路径与误报
- 技术演进逻辑（从盲找到语义协同）：
 1. 基石（关键字匹配）：首次提出利用前端（HTML/JS）与后端共享的“关键字”作为Oracle，定位后端输入点【SaTC, USENIX Security 2021】
 2. 演进1（模糊匹配）：解决硬编码与格式差异导致的漏报，引入基于编辑距离与自然语言处理（NLP）的模糊匹配与语义相似度计算【HermeScan, NDSS 2024】
 3. 演进2（结构化建模+LLM）：单纯匹配关键字依然受限，将输入结构化为URI和Key（键）的映射，并利用大语言模型（LLM）理解代码上下文，剔除误报【LARA, USENIX Security 2024】

Sharing More and Checking Less: Leveraging Common Input Keywords to Detect Bugs in Embedded Systems

Libo Chen ^{*1}, Yanhao Wang^{*2}, Quanpu Cai¹, Yunfan Zhan¹, Hong Hu³, Jiaqi Linghu², Qinsheng Hou^{2,6},
Chao Zhang^{4,5}, Haixin Duan^{4,5}, Zhi Xue ^{†1}

¹*School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University*

²*QI-ANXIN Technology Research Institute* ³*Pennsylvania State University*

⁴*BNRist & Institute for Network Science and Cyberspace, Tsinghua University*

⁵*Tsinghua University-QI-ANXIN Group JCNS* ⁶*Shandong University*

{bob777, zxue}@sjtu.edu.cn, wangyanhao@qianxin.com, honghu@psu.edu, {chaoz, duanhx}@tsinghua.edu.cn

USENIX Security 2021

内容摘要

➤ 关键动机

- 网络设备固件缺少源码和符号，传统静态/动态分析难以定位后端真实用户输入，导致漏洞检测误报和漏报严重

➤ 本文工作：提出基于前后端共享输入关键词的 Web 输入源识别方法

1. 利用前端页面与后端二进制共享的输入关键词作为锚点
2. 定位二进制中真正处理用户输入的数据提取函数
3. 执行目标导向静态污点分析，检测命令注入和缓冲区溢出等漏洞

➤ 关键数据

- 在 39 个固件上发现 33 个未知漏洞，其中 30 个获得 CVE/CNVD/PSV 等认可

研究背景

➤IoT 路由器、摄像头等设备普遍暴露 Web 管理服务

- 攻击者可以通过 HTTP 参数触发后端二进制中的危险操作

➤传统分析方式容易出现大量误报和漏报

- 动态执行/仿真覆盖率低且依赖硬件环境
- 传统静态分析又难以识别前后端交互中的真实用户输入

➤关键瓶颈

- 如果不知道后端哪里真正读取了 Web 输入，后续污点分析会在无关路径上浪费大量时间

核心洞察

- 物联网设备通常包含Web管理界面（前端）和二进制服务程序（后端）
 - **共享关键词 (Shared Keywords):** 前端用于构建请求的参数名（如 "username", "passwd"），通常以字符串常量的形式直接出现在后端二进制中，用于解析数据
- 不去模拟整个系统，而是利用这些“关键词”作为锚点，直接定位后端处理用户输入的代码区域

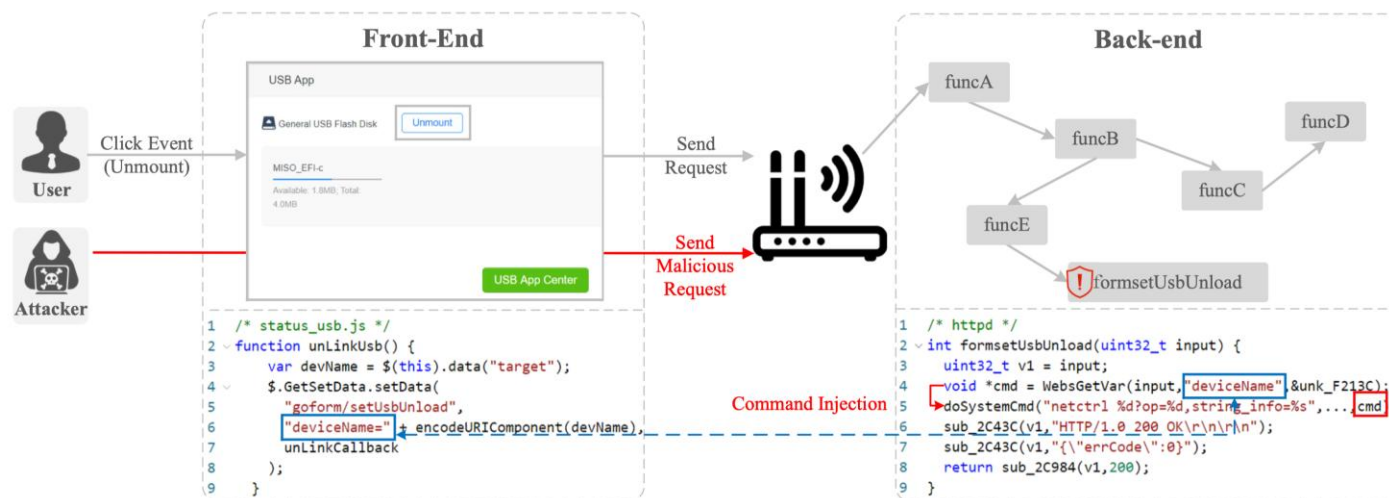


Figure 1: Motivating example. The left-hand side shows the front-end of the Tenda AC18 router: the USB management interface and the source code of the web page; the right-hand side shows the back-end: the call graph of the message processing process. `deviceName` is used by the code in the front-end and the back-end. An attacker can inject arbitrary command via sending a request with a malicious device name.

SaTC 系统架构总览

1. 前端关键词提取:

- 从文件系统中解析Web文件

2. 后端参考点定位:

- 将关键词映射到二进制文件中的引用点

3. 目标导向污点分析:

- 从引用点出发, 追踪数据流向危险函数

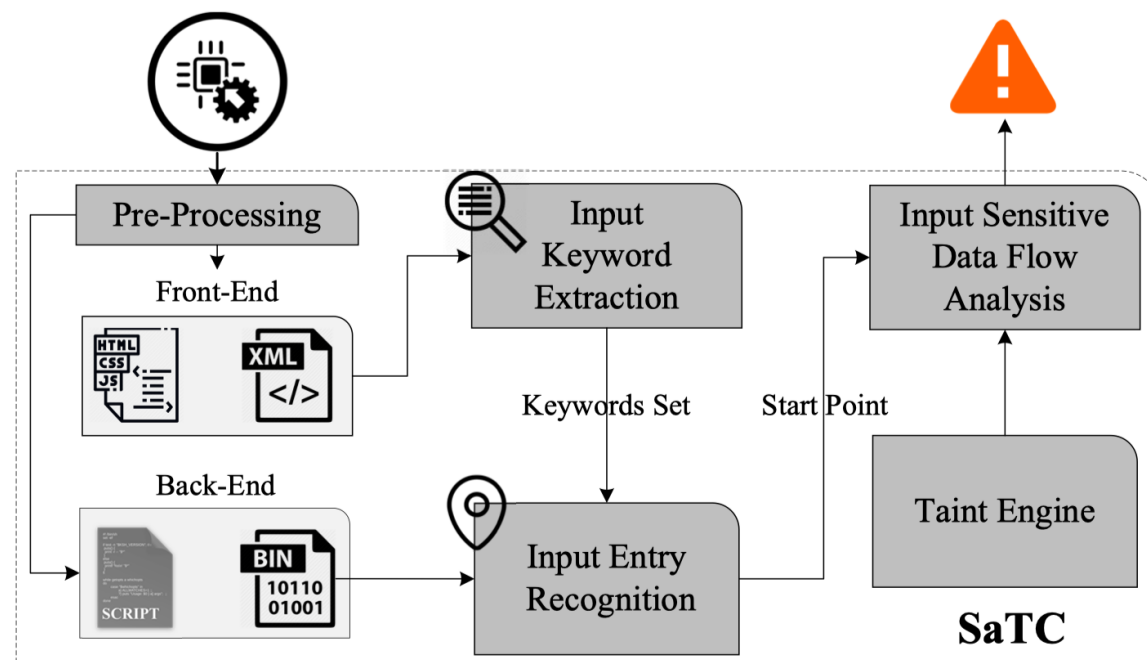


Figure 2: Structure of SaTC. SaTC searches in the firmware front-end to find input keywords and locates their references in the back-end. Starting from reference points, SaTC uses input-sensitive taint analysis to discover vulnerabilities in the back-end.

核心方法一：前端关键词提取

- 输入: 固件解压后的文件系统
- 处理:
 1. 解析 HTML, JavaScript, Lua, PHP 文件
 2. 识别 `<input>`, `<select>`, `textarea` 等标签
 3. 识别 `XMLHttpRequest`, `fetch` 等API调用
- 建立停用词表 (Stoplist), 剔除 CSS 类名、DOM ID、常见 UI 词汇 ("submit", "cancel", "width")
- 输出: 候选关键词列表
 - `<input name="X">`, `<select name="X">` → 提取 **X**
 - `param = "user=" + v1 + "&pwd=" + v2` → 提取 **"user", "pwd"**
 - `"key": value` → 提取 **"key"**

核心方法二： 后端参考点定位

- 目标: 关注数据提取函数 (Data Retrieval Functions, DRFs):
 - 例如: `http_get_param`, `cJSON_GetObjectItem`, `getenv`
- 算法: 如果一个关键词作为参数传递给了疑似DRF的函数, 则该位置被标记为入口点
 1. Candidate Selection: 收集所有接受前端搜集到的关键词作为参数的函数
 2. Clustering: 如果该函数在不同调用点接受了多个不同的前端关键词 (e.g., "username", "password", "ip"), 则该函数的候选分数 Score 增加
 3. Data Flow Check: 检查该函数的返回值是否被后续指令使用 (Live Variable)
 4. Threshold: 选取 Top-N Score 的函数作为 DRF

核心方法三：污点分析技术

- 污点源建模 (Source Modeling)
 1. Explicit Source: DRF 函数的返回值
 2. Implicit Source: 某些函数通过输出参数返回数据，需识别指针类型的参数作为 Source
 - 例如：`json_get_value(obj, "key", &out_buf)`
- Taint Tagging: 为每个 Source 分配唯一的 Tag，以便后续区分不同输入源
- 汇聚点与危险操作建模 (Sink Modeling)
 1. Command Injection: `system`, `popen`, `execve`
 - 漏洞模式：污点数据作为命令字符串的一部分，且未被引号包裹或转义
 2. Buffer Overflow: `strcpy`, `strcat`, `sprintf`
 - 漏洞模式：污点源作为 Source Buffer，且未进行长度检查

实验环境与数据集

- Dataset: 39个固件，来自 D-Link, Netgear, TP-Link 等顶级厂商
- Metric:
 - True Positives (TP): 验证成功的漏洞
 - False Positives (FP): 误报
 - Analysis Time: 每个二进制的耗时

Table 3: Dataset of device samples. We selected 39 device samples from six vendors, including 37 routers and two cameras on two architectures. SaTC found 33 previously unknown bugs, and 30 have been confirmed by developers. SizeP and SizeUP represent the average size before and after unpacking, respectively.

Vendor	Type	Series	#	SizeP	SizeUP	Arch	Bugs
Netgear	Router	R/XR/WNR	19	38M	192M	ARM32 (LE)	5
Tenda	Router	AC/G/W	9	12M	105M	ARM32 (LE)	10
TOTOLink	Router	A/T	2	5M	60M	ARM32 (LE)	3
D-Link	Router	DIR/DSR	5	8M	123M	MIPS32 (LE)	12
Motorola	Router	C1/M2	2	12M	64M	MIPS32 (LE)	3
Axis	Camera	P/Q	2	60M	700M	ARM32 (LE)	0
Total	2	14	39	135M	1,244M	2	33

性能评估

- 漏洞检测：总计 33 confirmed 0-days
 - Command Injection (20): 主要是 sprintf 拼接命令字符串
 - Buffer Overflow (13): 主要是 strcpy 处理长参数
 - 这些漏洞均位于 "非标准" 的数据处理路径上，传统工具无法定位入口
- 运行效率
 - Keyword Extraction: < 10秒/固件，正则匹配极快
 - Taint Analysis: 平均 20-30 分钟/二进制

Table 4: Vulnerabilities discovered by SaTC. For the bug type, BoF means buffer overflow; CI represents command injection; IAC indicates incorrect access control. **Ksrc** represents the type of the front-end file where the vulnerability-related keyword is found. **Service** represents the service where the vulnerability occurs.

Vendor	Device Series	Type	Bug IDs	Ksrc	Service
Netgear	R7000/R7000P	BoF	PSV-2020-0267 CVE-2020-28373	HTML XML	HTTP UPnP
	R6400v2	CI	CNVD-2020-15102 CNVD-2020-28091	HTML+ HTML+	HTTP HTTP
	XR300	CI	PSV-2020-0277	HTML	HTTP
Tenda	W20E	CI	CNVD-2019-22866	JS	HTTP
			CNVD-2019-22867	JS	HTTP
			CNVD-2019-22869	HTML	HTTP
	G1/G3	IAC	1 unassigned	JS	HTTP
		CI	CNVD-2020-46058	JS	HTTP
	CNVD-2020-46059		JS	HTTP	
	AC15/AC18	CI	CNVD-2020-29725	JS	HTTP
CNVD-2020-40766			JS	HTTP	
CNVD-2020-40767			JS	HTTP	
		CNVD-2020-40768	JS	HTTP	
TOTOLink	T10	CI	CNVD-2020-28089	JS	HTTP
	A950RG	CI	CNVD-2020-28090	JS	HTTP
			1 unassigned	JS	HTTP
D-Link	DIR 823G	IAC	CVE-2019-7388	JS	HTTP
			CVE-2019-7389	JS	HTTP
			CVE-2019-7390	JS	HTTP
			CVE-2019-8392	JS	HTTP
	DIR 878	CI	CVE-2019-8312	XML	HNAP
			CVE-2019-8314	XML	HNAP
			CVE-2019-8316	XML	HNAP
			CVE-2019-8317	XML	HNAP
			CVE-2019-8318	XML	HNAP
			CVE-2019-8316	XML	HNAP
			1 unassigned	JS	HTTP
DIR 878 882	IAC CI	CNVD-2020-23845	XML	HNAP	
Motorola	C1 M2	CI	CVE-2019-9117	JS	HTTP
			CVE-2019-9118	JS	HTTP
			CVE-2019-9119	JS	HTTP
Total		3	33	3	3

文章评价

➤ 设计优势

- 关键洞察简洁、工程可落地性强，适合作为固件 Web 安全扫描的基础模块
- 把 Source 识别从纯二进制推断转向前后端协同，显著提升了分析效率

➤ 局限性分析

1. Web依赖: 如果攻击面不在 Web (如私有二进制协议 UDP 5353), 则该技术失效
2. 前端混淆: 现代前端框架 (React/Vue) 打包后的 JS 变量名被压缩 (a, b, c)导致语义丢失, 则该技术失效
3. 隐藏接口: 如果开发者留了后门, 前端完全没写, 则该技术无法提取关键词, 产生漏报

➤ 商业化潜力

- 鉴于2021年的技术, 更推荐商业化落地同一类目的后续技术

Faster and Better: Detecting Vulnerabilities in Linux-based IoT Firmware with Optimized Reaching Definition Analysis

Zicong Gao^{1,2}, Chao Zhang^{3,4*}, Hangtian Liu^{1,2}, Wenhou Sun³, Zhizhuo Tang^{1,2},
Liehui Jiang^{1,2}, Jianjun Chen³, Yong Xie⁵

¹School of Cyber Science and Engineering, Information Engineering University,

²State Key Laboratory of Mathematical Engineering and Advanced Computing,

³Tsinghua University, ⁴Zhongguancun Laboratory, ⁵Qinghai University

NDSS 2024

内容摘要

➤ 关键动机

- SaTC 在识别Source时受限于关键词精确匹配、在追溯Source-Sink路径时存在跨库 CFG 缺失和符号执行路径爆炸，难以高效覆盖真实固件漏洞

➤ 本文工作：结合模糊 Source 匹配、跨库 CFG 恢复与轻量级可达定义分析(Reaching Definition Analysis) 的漏洞检测框架

1. 在 Source 识别上引入编辑距离和 BERT 语义相似度，缓解前后端关键词格式差异造成的漏报
2. 在数据流分析上用轻量级、上下文敏感、按需的可达定义分析替代符号执行
3. 在控制流层面恢复跨主程序与共享库的 CFG，从而找回间接调用和库函数包装导致的丢失路径

➤ 关键数据

- 在 30 个最新固件上发现 163 个漏洞，TPR 达 81%，SaTC 仅发现 32 个
- 平均分析速度约为 SaTC 的 7.5 倍

研究背景

➤SaTC 等方法依赖前后端关键词精确匹配

- 遇到 hostname_1.1 与 hostname_%s、user_name 与 username 等格式差异就会漏报

➤SaTC 常使用符号执行来检测漏洞

- 面对复杂固件时容易路径爆炸

➤现实固件还大量使用函数指针表、共享库包装函数和被剥离符号的二进制

- 导致传统 CFG 不完整，Source 和 Sink 都可能无法连通

动机案例

- 案例: ASUS 路由器 httpd 中的命令注入漏洞
- 现有工具为何失败 (SaTC)?
 - 该设备使用了函数指针数组 (如 ej_handler, *Subgraph-1&2*) 进行间接跳转来分发请求
 - 现有工具无法解析这种间接调用, 真实用户输入 (websGetVar, *Subgraph-3*) 的处理函数未能加入控制流图中
- 困境: 如果强行让现有工具 (如符号执行) 去跨库追踪并解析所有间接跳转, 分析一个固件将耗费数小时甚至数十小时, 这在实际应用中是不可接受的

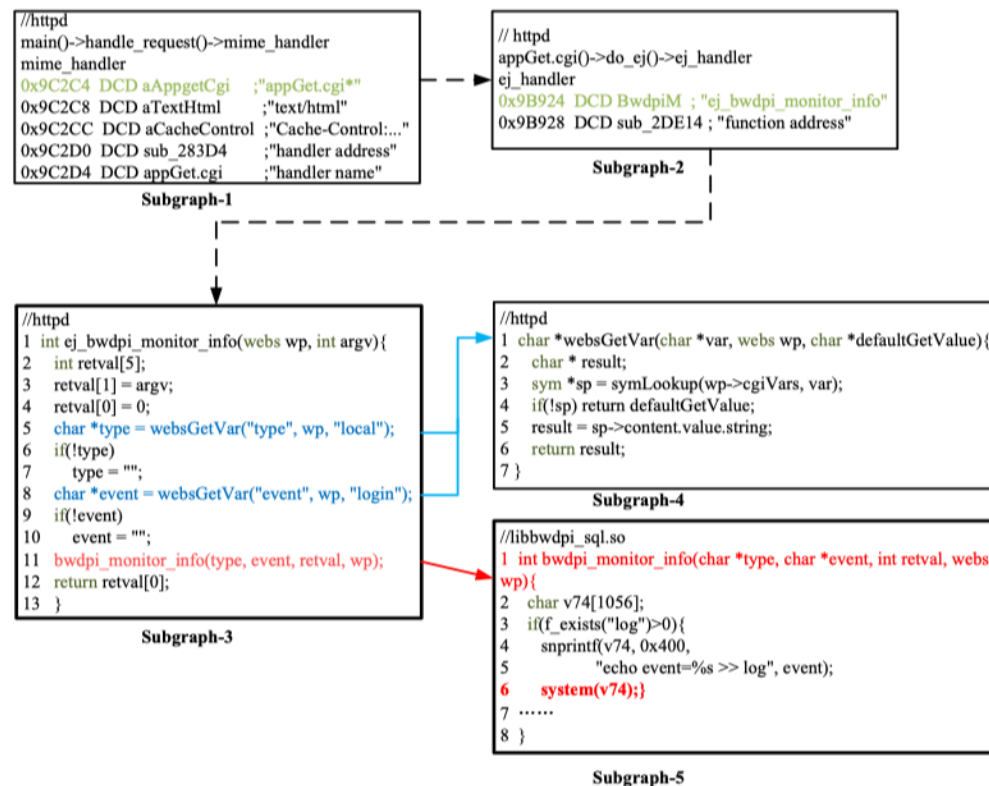


Fig. 2: A simplified example of the zero-day vulnerability in ASUS router. The function ej_bwdpi_monitor_info in the httpd program introduces unrestricted user input in the websGetVar function (blue part), resulting in OS command injection (red part) in the library file libbwdpi_sql.

核心洞察

➤ **Insight 1:** 将污点追踪降维为“数据可达性”问题 (Lightweight RDA)

- 不需要沉重的符号执行来求解每一条路径的约束，污点追踪的本质只是判断“源点的数据能否到达 Sink 点”
- 通过引入编译原理中轻量级的可达定义分析 (Reaching Definition Analysis, RDA)，高效遍历所有语句

➤ **Insight 2:** 重建跨越边界的代码图谱 (Enhanced CFG)

- 有了轻量级的 RDA 作为底座，我们可以轻量级扩展到外部库分析
- 通过分析库依赖关系，将二进制主程序的 CFG (Bin-CFG) 与共享库的 CFG (Lib-CFG) 打通，找回丢失的 Sink 点

➤ **Insight 3:** 从“硬编码匹配”走向“语义容错” (Fuzzy Matching)

- 前后端的关键字匹配容易因细微差别就匹配失败 (例如: username vs. user_name)
- 引入基于自然语言处理 (如 BERT) 的模糊匹配与语义相似度推断，找回非标准的隐藏 Source 点

把“路径能否被求解”转化为“定义是否能到达危险使用点”，保留漏洞追踪能力，同时显著
降低大规模固件分析成本

HermeScan 系统架构

1. 增强的 CFG 恢复 (Enhanced CFG Recovery):

- 识别边界二进制文件 (Border Binary)
- 恢复函数边界、符号名、调用约定, 连接 Bin-CFG 和 Lib-CFG

2. 源输入识别 (Source Input Identification):

- 模糊匹配策略: 结合编辑距离和语义相似度 (BERT) 寻找候选函数
- 候选函数检查: 通过 Def-Use 分析筛选真正的输入参数

3. 高效数据流分析 (Efficient Dataflow Analysis):

- LCO-RDA: 轻量级、上下文敏感、按需的到达定值分析
- 路径合并: 缓解路径爆炸问题

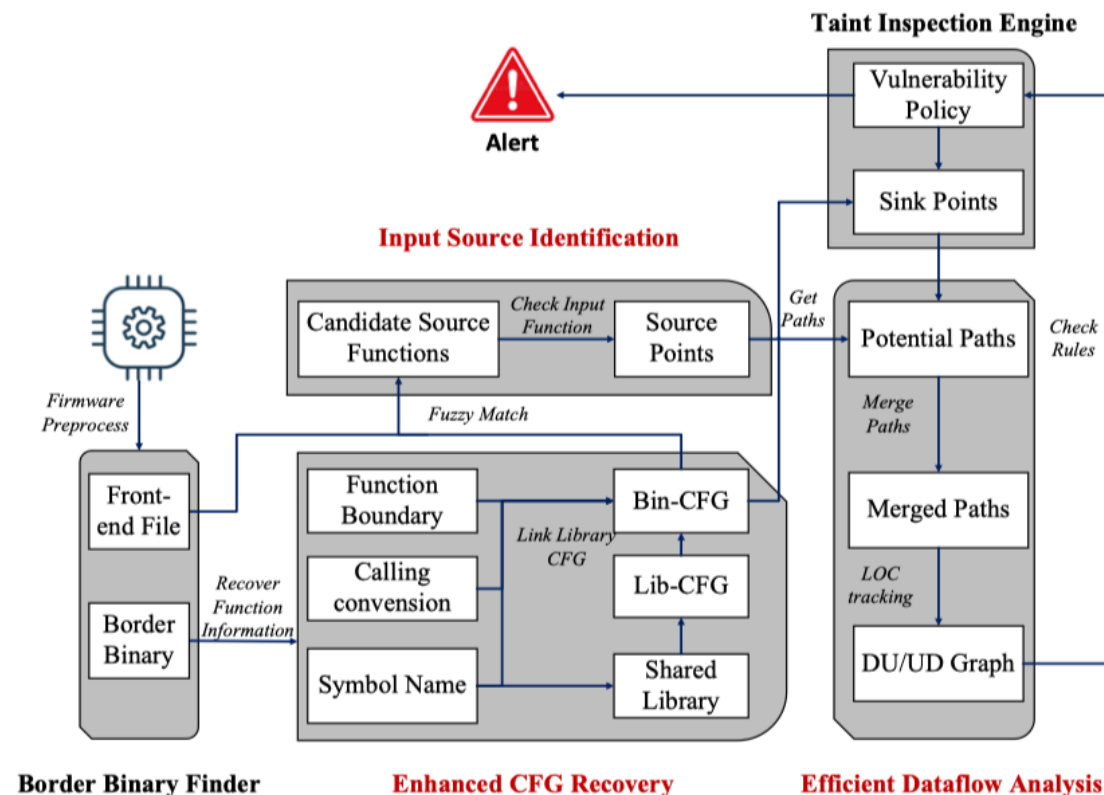


Fig. 3: Overview of HermeScan.

关键技术 1 - 增强的 CFG 恢复

1. 函数边界识别:

- 扫描汇编代码特征 (如 Prologue), 补充反汇编工具遗漏的函数

2. 符号名恢复:

- 即使 Section Header 被剥离 (Stripped), 也通过解析 ELF 的 Program Header 和 PT_DYNAMIC 段来恢复符号表

3. 调用约定 (Calling Convention) 恢复:

- 恢复参数传递规则 (如 MIPS 下的 a0-a3), 确保数据流分析的连贯性

4. 库连接:

- 建立共享库的 CFG (Lib-CFG) 并将其与主程序 CFG 连接, 解决跨库分析问题

关键技术 2 - 源输入识别 (模糊匹配)

- 问题:
 - 前端 (Web文件) 和后端 (二进制) 共享的关键词可能存在形态或语义上的差异，精确匹配会漏报
- 解决方案: 双重模糊匹配策略
 1. 字面相似度 (Format Similarity): 使用归一化的 Levenshtein 编辑距离
 - 例子: 前端 *hostname_1.1* vs 后端 *hostname_%s*
 2. 语义相似度 (Semantic Similarity): 使用预训练模型 BERT 计算 Embedding 的余弦相似度
 - 例子: 前端 *banned for security* 与后端 *sec_ip_ban* 建立语义关联

关键技术 2 - 源输入识别 (候选检查)

- 候选函数检查:
 - 并非所有引用共享字符串的函数都是输入函数
- 算法: 对候选函数进行 Def-Use 分析
 - 检查参数: 参数指向的内存是否被存入 (Stored) 数据? 如果是, 标记为污点源
 - 检查返回值: 返回值是否被后续操作使用 (Used)? 如果是, 标记为污点源
- 约束推断 (Constraint Inference):
 - 利用值集分析(VSA, Value Set Analysis) 推断输入长度约束 (例如 strncpy 的长度参数)
 - 避免无限制的符号变量导致的误报

关键技术 3 - LCO 跨过程分析

1. 轻量级 (Lightweight):

- 使用 RDA (基于 VEX IR) 替代符号执行，无需遍历所有分支，避免状态爆炸

2. 上下文敏感 (Context-sensitive):

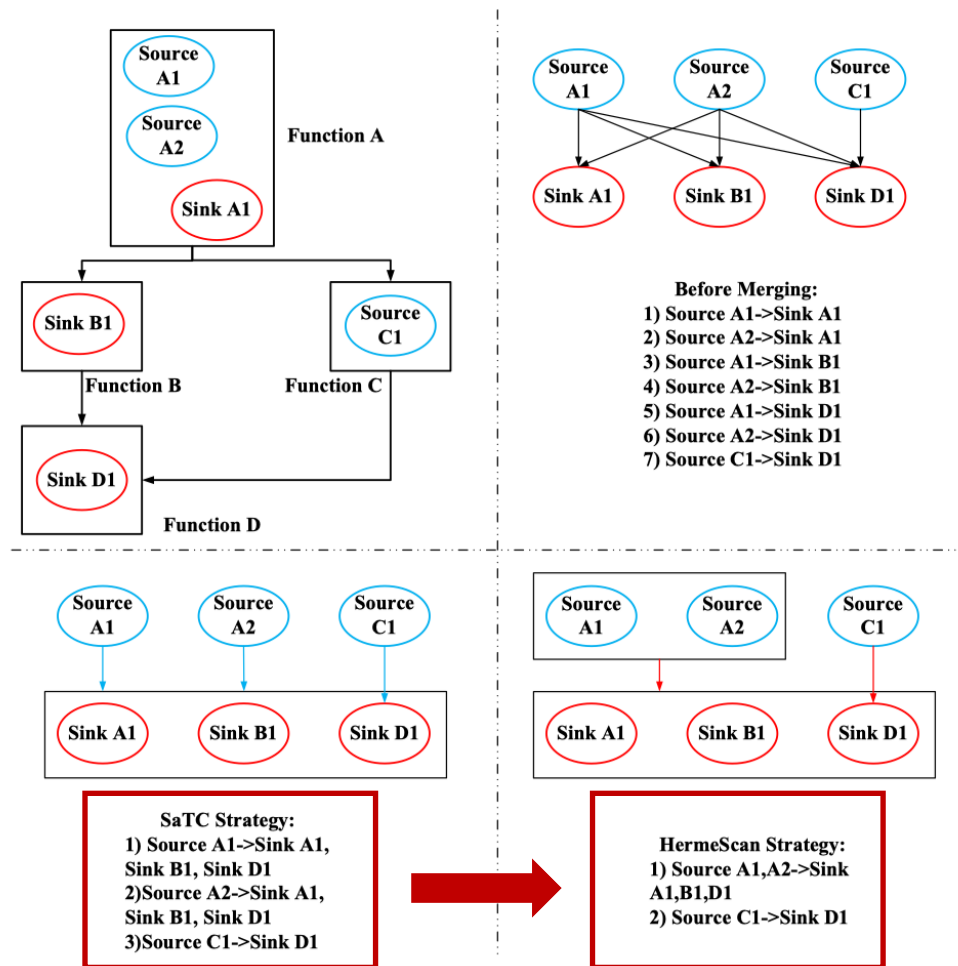
- 在函数调用时传递变量定义，处理别名 (Aliasing) 问题

3. 按需分析 (On-demand):

- 仅当函数参数被标记为污点时，才进入该函数分析
- 对常见 Libc 函数 (如 strcpy, sprintf) 使用摘要 (Summary)，对其他库函数则跟进分析

关键技术 3 - 路径合并策略

- 问题:
 - 污点分析通常面临路径爆炸，尤其是存在多个源和汇时
- 策略: 利用 RDA 的路径不敏感特性 (Path-insensitive)
 1. 多源污点 (Multi-source): 在一次 RDA 分析中，为同一函数内的多个源点标记不同的 Label，同时追踪
 2. 多汇点观察 (Multi-sink): 一次遍历覆盖调用图中所有可达的汇点 (Sinks)
- 效果:
 - 如图所示，可将需要分析的路径数从 7 条减少到 2 条，显著降低冗余



实验评估 - 设置

- 数据集:
 - 0-day Dataset: 30 个最新固件 (来自 Linksys, ASUS, Netgear, D-Link 等 8 个 厂商)
 - N-day Dataset: 98 个旧版本固件 (用于大规模测试)
- 对比工具:
 - SaTC 和 KARONTE (均为基于符号执行的 SOTA 工具)
- 环境:
 - Ubuntu 21.04, 4-core CPU, 32GB RAM

实验结果 - 0-day 漏洞发现

- 漏洞发现数量:
 - HermeScan 发现了 163 个漏洞 (87个未知, 76个已知但未修补)
 - 相比之下, SaTC 仅发现 32 个, KARONTE 发现 0 个
- 漏洞类型: 涵盖缓冲区溢出 (BoF)、命令注入 (CLI) 以及 跨站请求伪造 (CSRF) 等其他类型
- 准确率 (TPR): HermeScan: 81%、SaTC: 42%、KARONTE: 0% 。

TABLE III: **0-day dataset evaluation** results of HermeScan, SaTC, and KARONTE. The program name refers to the analyzed boundary binary. We counted the number of alerts generated by the tool (Alerts), the number of confirmed vulnerabilities (Vuls), and the tool execution time (Time). *Italics* in the time column represent unexpected exits or non-execution of the sample. **Bold** in the Vendor-Model column means HermeScan finds more vulnerabilities on this sample than other tools. The number of Vuls in () indicates the number of unique vulnerabilities after deduplication.

Vendor&Model	Program Name	HermeScan			Time	SaTC			Time	Karnote		
		Alerts	Vuls (bof+cli)	Vuls (other)		Alerts	Vuls (bof+cli)	Vuls (other)		Alerts	Vuls (bof+cli)	Time
LINKSYS MR7350	bluetoothd	0	0	0	3min	1	0	0	30min	0	0	2h22min
LINKSYS E9450	httpd	0	0	0	11min	0	0	0	24min	0	0	2h08min
LINKSYS EA4500	twonkymediaserver	1	1	0	17min	0	0	0	26h	0	0	1h21min
ASUS GT-AX6000	httpd	0	0	0	9min	5	0	0	27h28min	0	0	4h31min
ASUS GT-AC2900	cfg-server	0	0	0	12min	2	0	0	23h58min	0	0	7min
ASUS RT-AX56U	httpd	4	4	0	13min	5	0	0	26h21min	0	0	48min
Tenda AX-12	httpd	9	6	3	1h20min	0	0	0	12min	0	0	3h36min
Tenda AX-3	httpd	17	11	0	2h23min	27	6	36h	0	0	0	1h24min
Tenda AX-1803	thttpd	20	12	2	2h03min	38	8	16h28min	0	0	0	5min
Tenda AX-1806	thttpd	20	14	0	2h11min	44	13	18h53min	0	0	0	42min
Tenda W15E	httpd	17	15	2	2h39min	50	5	21h11min	0	0	0	1h22min
TOTOLINK T8	cstecgi	14	4	0	14min	0	0	0	3min	0	0	2min
TOTOLINK LR350	cstecgi	24	9	0	13min	0	0	0	47min	0	0	24min
TOTOLINK A7000	cstecgi	18	12	0	12min	0	0	0	4h09min	0	0	39min
TOTOLINK A8000	cstecgi	29	13	0	13min	2	0	0	39min	0	0	6h04min
D-LINK COVR-1201	prog.cgi	9	4	2	5h27min	0	0	0	10min	0	0	4h42min
D-LINK COVR-1210	prog.cgi	8	4	2	5h16min	0	0	0	10min	0	0	4h28min
Netgear RAX-10	net-cgi	6	0	0	19min	0	0	0	49min	0	0	2h20min
Netgear RAX-30	ntgr_ra_iot	0	0	0	6min	8	0	0	5h54min	0	0	3h10min
Netgear RAX-120	net-cgi	1	0	0	37min	0	0	0	1h09min	0	0	72h
Netgear MR-62	httpd	1	1	0	51min	0	0	0	23h56min	4	0	2h56min
Trendnet tve 829	samba_multicall	1	1	0	30min	0	0	0	2min	0	0	1h57min
Trendnet tew 823	ssi	39	17	0	9min	0	0	0	11min	0	0	1h54min
Trendnet tew 827	ssi	31	18	2	18min	0	0	0	27min	0	0	59min
Trendnet tew 818	rc	12	5	0	14min	0	0	0	18h32min	0	0	2h14min
Trendnet tew 752	cgibin	5	1	0	9min	0	0	0	20min	0	0	2h10min
TP-LINK AX3000	fapi_wlan_cli	0	0	0	0min	0	0	0	15min	0	0	2h42min
TP-LINK XDR1850	dms	2	0	0	18min	0	0	0	3min	0	0	2min
TP-LINK XDR3060	dms	3	2	0	1h44min	0	0	0	2min	0	0	28min
TP-LINK XTR7880	dms	6	2	0	2h10min	0	0	0	3min	0	0	19min
Total	/	297	156(152)	13(11)	30h4min	182	32(32)	252h19min	4	0	0	127h58min
Average	/	9.9	5.2	/	1h7min	6.66	1.9	8h25min	0.13	0	0	4h16min

bof (buffer overflow) and cli (command line injection)

实验结果 - 效率与 N-day 对比

- 效率 (Efficiency):
 - HermeScan 平均每个样本分析耗时 1小时7分钟，速度是 SaTC 的 7.5倍，KARONTE 的 3.8倍
- N-day 数据集表现:
 - HermeScan 发现 204 个已知漏洞 (比 SaTC 多 66 个)
 - TPR 达到 79%，远高于 SaTC (63%) 和 KARONTE (72%)

TABLE VI: *N-day dataset evaluation results* of HermeScan, SaTC, and KARONTE. For each vendor, we report the device series, the number of firmware samples, the total number of alerts (ALERT), the total number of true positives (TP), the average true positive rate (TPR), and the total analysis time (TIME: hour).

Vendor	Device Series	Samples	HermeScan				SaTC				KARONTE			
			ALERT	TP	TPR	TIME	ALERT	TP	TPR	TIME	ALERT	TP	TPR	TIME
NETGEAR	R/XR/WNR/AC	31	668	505	0.76	56.6	582	299	0.51	540.8	35	26	0.74	137.3
D-Link	DIR/DWR/DCS/COVR	17	36	29	0.81	12.8	20	14	0.70	20.9	16	13	0.81	39.1
TP-Link	TD/WA/WR/TX/KC	18	6	2	0.33	5.9	8	3	0.38	79.1	5	2	0.40	16.8
Tenda	AC/WH/FH/AX	20	289	251	0.87	34.2	262	229	0.87	227.9	23	15	0.65	7.9
TOTOLINK	T/A/LR	6	64	52	0.81	1.6	6	4	0.67	33	0	0	/	1.3
MOTOROLA	C1/M2	2	8	8	1	0.4	0	0	/	0.33	0	0	/	1.1
AXIS	P/Q	2	0	0	/	0.5	0	0	0	4.1	0	0	/	0
Others	/	2	0	0	/	0	0	0	/	0	4	4	1.00	1.6
Total	/	98	1071	847	/	112.0	878	549	/	906.1	83	60	/	205.1
Average	/	/	10.93	8.64	0.79	1.14	8.96	5.60	0.63	9.25	0.85	0.61	0.72	2.09

文章评价

➤设计优势

- 系统性补齐了 Source 模糊匹配、跨库 CFG 与轻量数据流三类关键缺口
- 适合大规模固件扫描

➤局限性分析

1. RDA 的路径不敏感特性仍可能带来误报
2. BERT 相似度也可能引入语义噪声

➤商业化潜力

- 适合作为固件安全平台的高速静态引擎
- 但在Source识别方面，建议参考同一类目的后续技术

Leveraging Semantic Relations in Code and Data to Enhance Taint Analysis of Embedded Systems

Jiaxu Zhao^{1,2,3,4}, Yuekang Li⁵, Yanyan Zou^{1,2,3,4}✉, Zhaohui Liang^{1,2,3,4}, Yang Xiao^{1,2,3,4}, Yeting Li^{1,2,3,4},
Bingwei Peng^{1,2,3,4}, Nanyu Zhong^{1,2,3,4}, Xinyi Wang^{1,2,3,4}, Wei Wang^{1,3,4}, Wei Huo^{1,2,3,4}✉

¹*Institute of Information Engineering, Chinese Academy of Sciences, China*

²*School of Cyber Security, University of Chinese Academy of Sciences, China*

³*Key Laboratory of Network Assessment Technology, Chinese Academy of Sciences, China*

⁴*Beijing Key Laboratory of Network Security and Protection Technology, China*

⁵*University of New South Wales, Australia*

USENIX Security 2024

内容摘要

➤ 关键动机

- 现有技术仅依赖前后端关键词匹配会遗漏隐藏 Key，也会把语义无关的字符串误判为 Source

➤ 本文工作：融合 URI-Key 语义建模与 LLM 代码理解的 Web 输入源识别方法

1. 把输入结构化为 URI 和 Key 的映射关系
2. 通过模式匹配与 LLM 语义分析共同识别 URI 处理函数、Key 提取函数和隐藏输入
3. 递归提取危险函数包装层，最后执行 Key-sensitive 静态污点分析

➤ 关键数据

- 在 203 款设备固件上，相比 SaTC 提取 URI/Key 分别提升 5.4 倍和 4.7 倍
- 在漏洞检测上，比 SaTC 多发现 556 个漏洞，其中 245 个是 0-day

研究背景

1. 精确匹配的缺陷

- 现有技术高度依赖前端文件与后端二进制文件中“关键字”的精确匹配来定位污点源

2. 隐藏输入（Hidden Inputs）的漏报

- 由于前端文件并未包含所有用户输入逻辑，许多后端处理的“隐藏参数”缺乏前端映射，现有工具因无法匹配此类关键字而产生大量漏报

3. 缺乏语义理解导致的误报

- 传统的模式匹配无法理解代码上下文语义，只要函数参数中包含类似关键字的字符串（如 PWD_password），即可能被错误标记为输入处理函数，从而引发误报

核心洞察

➤ **Insight 1:** 用户输入的结构化分类（“URI-Key”二元结构）

- 用户输入通常以表单（Forms）形式组织，具有明显的层级结构
 - 将输入严格划分为“URI (统一资源标识符)”和“Key (键)”，有助于系统性地还原前后端映射关系
- 利用已知 URI 的后端处理代码，可以推导出与之关联的已知及隐藏 Key，进而有效识别隐藏的污点源

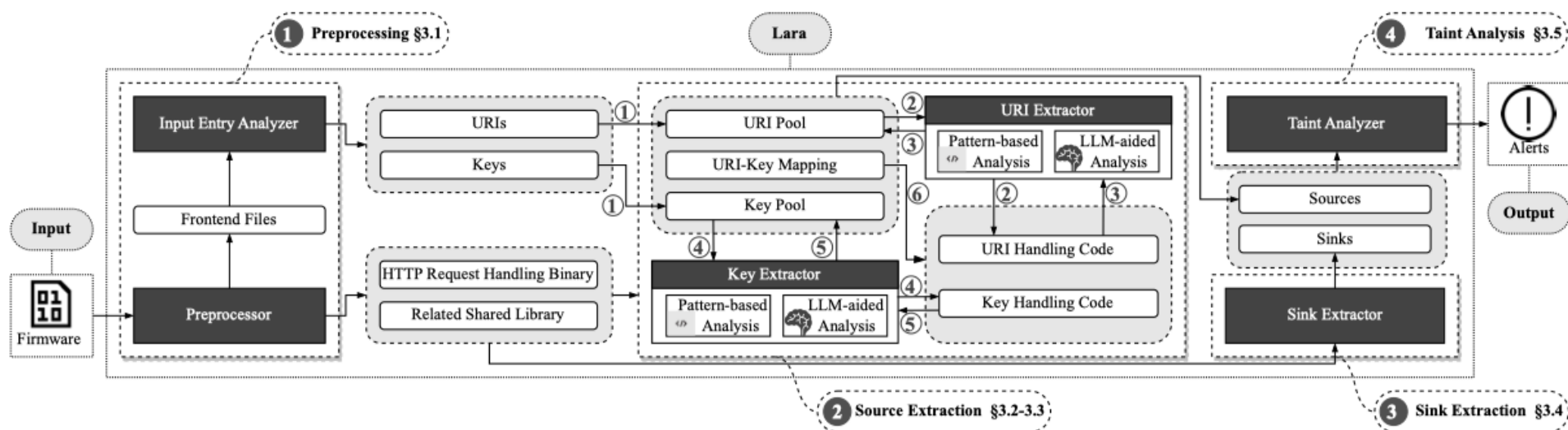
➤ **Insight 2:** LLM 辅助的语义理解

- 大语言模型（LLM）具备强大的代码语义总结与推断能力
- 基于模式匹配的静态分析与基于语义的 LLM 分析在产生误报的原因上具有正交性
 - 将两者结合（取交集），能够有效过滤由单纯模式匹配引起的语义误报

让规则负责高召回、LLM 负责语义判断、再用静态模式校验 LLM 输出，从而兼顾覆盖率与可控性

LARA 系统架构总览

1. **预处理 (Preprocessing):** 从固件中提取前端文件（HTML/JS/XML）和后端服务程序及共享库，并利用正则表达式和 AST 分析提取初始的 URI 和 Key 集合
2. **污点源提取 (Source Extraction):** 结合模式匹配与 LLM 辅助分析，定位后端程序中的 URI 处理函数与 Key 提取函数，进而系统性地扩充 URI 与 Key 集合
3. **汇聚点提取 (Sink Extraction):** 在主程序及共享库中，递归地提取危险函数（如 system）及其多层包装函数（Wrapper Functions）作为最终 Sink 点
4. **污点分析 (Taint Analysis):** 基于提取的 Source 与 Sink 集合，执行键值敏感（Key-sensitive）的静态污点分析与漏洞条件检测



核心模块1：基于模式的污点源识别（上）

➤ **设计目标：**利用代码模式定位 URI 与 Key 处理逻辑

➤ LARA 首先利用逆向工程中的启发式模式规则进行代码分析：

1. **URI 提取机制：**LARA 总结了三种主流的 URI 绑定代码模式：注册函数（Registration Function）、常量数据（Constant Data）以及进程创建（Process Creation）

→ 通过识别这些模式，定位负责路由分发的处理代码

2. **Key 提取机制：**在确认 URI 处理函数后，LARA 进一步分析其内部的函数调用

→ 若某函数接收前端提取的 Key 作为参数（如 websGetVar），且该函数的调用上下文与 HTTP 请求对象存在交集，则将其标记为候选的 Key 处理函数

核心模块1：大语言模型辅助语义分析（中）

➤ **设计目标：**引入 LLM 提升代码功能识别的语义准确性

➤ 为弥补静态模式匹配的语义缺失，LARA 引入基于 LMQL 的 LLM 自动化交互模型：

1. **自动化交互模型构建：**设计了包含 Query、From 和 Where 子句的 LMQL 模板，通过控制流和约束条件实现与 LLM（如 GPT-3）的无缝程序化交互，无需人工介入
 - LMQL 是一种面向大语言模型（LLM）的编程语言，它基于 Python 的超集.

2. Prompt 设计策略：

- 针对 **URI 注册函数**：提示 LLM 分析包含特定 URI 引用的代码片段，判断其是否具备“路由注册（Registration）”的语义功能
- 针对 **Key 处理函数**：提示 LLM 判断函数的参数是否源自 HTTP 请求，并推断该函数的核心意图是否为“检索信息及赋值（Retrieving information）”

核心模块1：复合分析与交叉验证（下）

➤ **设计目标：**消除单一分析方法的误报噪音

➤ LARA 通过融合模式匹配与 LLM 的分析结果，实现高精度的源点确认：

- **针对 URI 处理代码：**LLM 能够识别出具有非标准代码结构、被静态模式遗漏的注册函数（如 websAspDefine），从而显著降低漏报率

→ LARA 会将 LLM 的增量结果纳入，并进行基本的模式合法性二次校验

- **针对 Key 处理代码：**LLM 有效排除了由误导性参数引发的模式匹配误报

→ LARA 对两种方法的输出取交集，利用两者的互补性，精准剔除了语义不符的伪处理函数，最大化保证了 Key 提取的准确性

核心模块2： 多层包装汇聚点提取

➤ **设计目标：** 应对跨二进制文件与跨函数的 Sink 隐藏

➤ 传统静态分析工具通常仅将标准库中的危险函数（如 `system`, `popen`）定义为 Sink，导致大量漏报

- **包装函数（Wrapper Functions）识别：** 危险操作往往被封装在共享动态库或深层调用中

- LARA 通过预分析共享库和主程序，递归查找调用了底层危险函数的调用者（Callers）

- **准确性过滤：** 为防止提取到无关的包装函数，LARA 应用两项策略保障精度：

1. 借助轻量级污点分析，确保包装函数的参数确实能够传递至内部的危险函数
2. 过滤掉包含 URI 或 Key 提取逻辑的无关函数

核心模块3：键值敏感的污点追踪

➤ 设计目的：精确的漏洞验证与状态传播

➤ 获取高质量的 Source 与 Sink 后，LARA 启动定制化的静态污点引擎：

- 数据流追踪：基于抽象语法树（AST）进行路径敏感的前向分析

- 采用包含“污点属性、分配内存/值、最大可分配内存”的三元组结构，精确记录指针、数组和全局变量的状态变化

- 漏洞条件检测：当受污染数据到达 Sink 时，LARA 会执行严格的安全约束检查

- 针对 strcpy 引发的缓冲区溢出，评估目标缓冲区的容量约束

- 针对 execv 导致的代码注入，检查相关格式化字符串的构造条件

实验评估

- LARA 在包含 21 个厂商、203 款设备的真实固件数据集上进行了全面评估:

Table 2: Vulnerability Detection Result of LARA and SOTAs on the Dataset. (x(y) means #All(#Hidden). “Prec.” and “Reca.”Indicate Precision and Recall, respectively. Time is overall analysis time which unitIs min.)

Vendor	Samples	URI Type	#Vuln	LARA						SATC						KARONTE					
				TP	FP	FN	Prec.	Reca.	Time	TP	FP	FN	Prec.	Reca.	Time	TP	FP	FN	Prec.	Reca.	Time
Tenda	19	I&II	233 (51)	231 (51)	28	2	89.2%	99.1%	4,306	64 (1)	89	169	41.8%	27.5%	8,903	12	23	221	34.3%	5.2%	9,120
TOTOLink	11	II&III	134 (13)	134 (13)	32	0	80.7%	100.0%	736	8	23	126	25.8%	6.0%	5,280	0	5	134	0.0%	0.0%	4,667
H3C	13	I&I&III	88	88	9	0	90.7%	100.0%	894	0	22	88	0.0%	0.0%	6,240	0	11	88	0.0%	0.0%	6,240
DLink	14	I&II&III	67 (11)	62 (11)	13	5	82.7%	92.5%	1,603	0	42	67	0.0%	0.0%	6,720	8	22	59	26.7%	11.9%	5,903
TRENDnet	13	I&II&III	23 (2)	23 (2)	11	0	67.6%	100.0%	1,103	3	10	20	23.1%	13.0%	5,080	0	3	23	0.0%	0.0%	5,914
Wavlink	3	III	21	21	9	0	70.0%	100.0%	96	1	6	20	14.3%	4.8%	903	0	2	21	0.0%	0.0%	1,003
NetGear	34	I&II&III	16 (1)	16 (1)	15	0	51.6%	100.0%	8,799	1	11	15	8.3%	6.3%	16,011	11	15	5	42.3%	68.8%	14,080
Cisco	4	II&III	10	10	8	0	55.6%	100.0%	153	1	16	9	5.9%	10.0%	603	3	6	7	33.3%	30.0%	667
Linksys	13	I&II&III	9 (3)	9 (3)	0	0	100.0%	100.0%	768	0	15	9	0.0%	0.0%	2,301	0	3	9	0.0%	0.0%	3,005
DrayTek	9	I&II&III	9	9	5	0	64.3%	100.0%	1,203	0	0	9	0.0%	0.0%	3,908	0	0	9	0.0%	0.0%	4,320
TPLink	8	I	9 (1)	8 (0)	3	1	72.7%	88.9%	803	1	0	8	100.0%	11.1%	1,608	0	0	9	0.0%	0.0%	1,303
Zavio	1	III	9	9	1	0	90.0%	100.0%	82	0	2	9	0.0%	0.0%	480	0	0	9	0.0%	0.0%	480
Motorola	2	I	6 (2)	6 (2)	2	0	75.0%	100.0%	147	0	3	6	0.0%	0.0%	960	0	1	6	0.0%	0.0%	960
Belkin	3	II&III	5	5	0	0	100.0%	100.0%	283	3	4	2	42.9%	60.0%	803	2	1	3	66.7%	40.0%	966
SonicWall	2	II&III	4	4	1	0	80.0%	100.0%	182	0	0	4	0.0%	0.0%	960	0	0	4	0.0%	0.0%	960
ASUS	18	I&II	1	1	0	0	100.0%	100.0%	2,108	0	3	1	0.0%	0.0%	5,166	0	3	1	0.0%	0.0%	3,806
QNAP	5	III	1	1	0	0	100.0%	100.0%	603	0	3	1	0.0%	0.0%	1,608	0	0	1	0.0%	0.0%	1,303
Fortinet	1	II	1	1	2	0	33.3%	100.0%	20	0	0	0	0.0%	0.0%	480	0	0	0	0.0%	0.0%	480
Zyxel	25	I&III	0	0	0	0	0.0%	0.0%	3,260	0	0	0	0.0%	0.0%	9,605	0	0	0	0.0%	0.0%	7,221
Mercury	3	I&III	0	0	3	0	0.0%	0.0%	228	0	0	0	0.0%	0.0%	1,440	0	0	0	0.0%	0.0%	920
AXIS	2	III	0	0	0	0	0.0%	0.0%	51	0	0	0	0.0%	0.0%	452	0	0	0	0.0%	0.0%	960
Total	203	—	646 (84)	638 (83)	142	8	81.8%	98.8%	27,428	82 (1)	249	563	24.8%	12.7%	79,511	36	95	609	27.5%	5.6%	74,278

关键结果

1. 污点源提取对比:

- 与 SaTC 相比, LARA 提取的 URI 和 Key 数量分别提升 5.4 倍和 4.7 倍, 同时误报率降低了 57.0%, 分析时间减少 65.5%

2. 漏洞检测对比:

- 与 SaTC 相比, LARA 多检测出 556 个漏洞
- 与 KARONTE 相比, LARA 多检测出 602 个漏洞

3. 0-day 漏洞挖掘成果:

- 在针对真实世界设备的未知漏洞挖掘中, LARA 成功发现了 245 个 0-day 漏洞 (分布于 57 款设备中)
- 截至目前, 所有漏洞均已被厂商确认或修复, 并已获分配 162 个 CVE 编号

文章评价

➤设计优势

- 把源点识别从字符串层推进到代码意图层，对 Source 的识别能力更出色

➤局限性分析

1. LLM 引入成本、稳定性和可复现性问题
2. LLM 效果依赖反编译代码片段质量

➤商业化潜力

- 适合用于新一代基于大模型的Source 识别技术
- 但需要提示词固化、缓存、本地模型或审核流程来降低不确定性

维度一总结：精准识别输入源点

SaTC 【USENIX SEC 2021】

首次利用前后端共享的“关键字”在二进制文件中定位输入入口

但受限于硬编码匹配，存在漏报

HermeScan 【NDSS 2024】

引入编辑距离（字形相似度）与基于NLP的BERT模型（语义相似度）



从匹配模式本身入手：通过模糊匹配策略大幅找回了因拼接和格式差异而丢失的输入源点

LARA 【USENIX SEC 2024】

引入大语言模型（LLM）对候选的输入处理函数进行上下文意图分析



从匹配上下文入手：缓解模式匹配带来的语义误报

维度二：复杂后端数据流 (Propagation)

➤ 核心痛点：即便定位输入点后，数据在后端往往不是被单一程序处理

→ 现代网络设备的架构极其复杂，数据流追踪面临规模爆炸与路径截断

➤ 技术路线（跨进程/语言 + 大规模分析性能）

1. **跨进程建模**：提出多二进制文件交互（Multi-binary）模型，追踪Web输入如何通过NVRAM、环境变量传播
【Karnote, IEEE S&P 2020】
2. **跨语言架构**：突破传统的“C/C++单一假设”，解决现代固件中广泛存在的C-Lua混合架构跨语言污点分析难题
【FirmCross, NDSS 2026】
3. **规模化与路径过滤**：为应对路径爆炸，采用“Sink-to-Source反向分析”，深入处理NVRAM传播中的间接调用
【MangoDFA, USENIX Security 2024】
4. **底层优化**：引入基于结构化符号表达式（SSE）的按需别名分析，解决间接跳转导致的数据流丢失
【OctopusTaint, ACM CCS 2024】

KARONTE: Detecting Insecure Multi-binary Interactions in Embedded Firmware

Nilo Redini*, Aravind Machiry*, Ruoyu Wang[†], Chad Spensky*, Andrea Continella*,
Yan Shoshitaishvili[†], Christopher Kruegel*, and Giovanni Vigna*

*UC Santa Barbara [†]Arizona State University

{nredini, machiry, cspensky, conand, chris, vigna}@cs.ucsb.edu
{fishw, yans}@asu.edu

IEEE S&P 2020

内容摘要

➤ 关键动机

- IoT 固件漏洞常跨多个二进制传播
- 单二进制分析无法捕获通过环境变量、文件、Socket 等 IPC 传递的攻击输入

➤ 本文工作:面向多二进制固件交互的跨进程数据流建模与漏洞检测框架

- 关注单个二进制内部分析无法覆盖的多二进制漏洞
 1. 把跨进程通信抽象为数据键上的 Setter/Getter 关系
 2. 构建 Binary Dependency Graph 并进行跨二进制污点传播

➤ 关键数据

- 在 53 个固件样本上发现 46 个真实 0-day，并将平均告警数从 722 个降低到 2 个

研究背景: 多二进制架构下的数据流追踪挑战

➤ 多二进制漏洞的隐蔽性

- 攻击者通过网络暴露的入口（如 Web 服务器）注入恶意数据
- 但该数据往往不直接在入口程序中触发漏洞
- 而是通过环境变量、共享内存或文件传递至深层组件（如 CGI 脚本或底层系统服务）执行

➤ 只分析入口程序会漏掉跨进程漏洞，全量分析又会产生海量告警

核心洞察

➤ 核心洞察

- 尽管固件架构复杂，但二进制文件之间的通信必然依赖于有限的进程间通信（IPC）范式

➤ 利用数据键机制（Data Keys）抽象跨进程通信

- 任何跨进程通信均依赖特定的“数据键”，如特定的环境变量名、文件路径或硬编码的内存地址
- 通过静态提取这些数据键及其操作语义，可精确地重构固件内不同组件之间的数据依赖关系

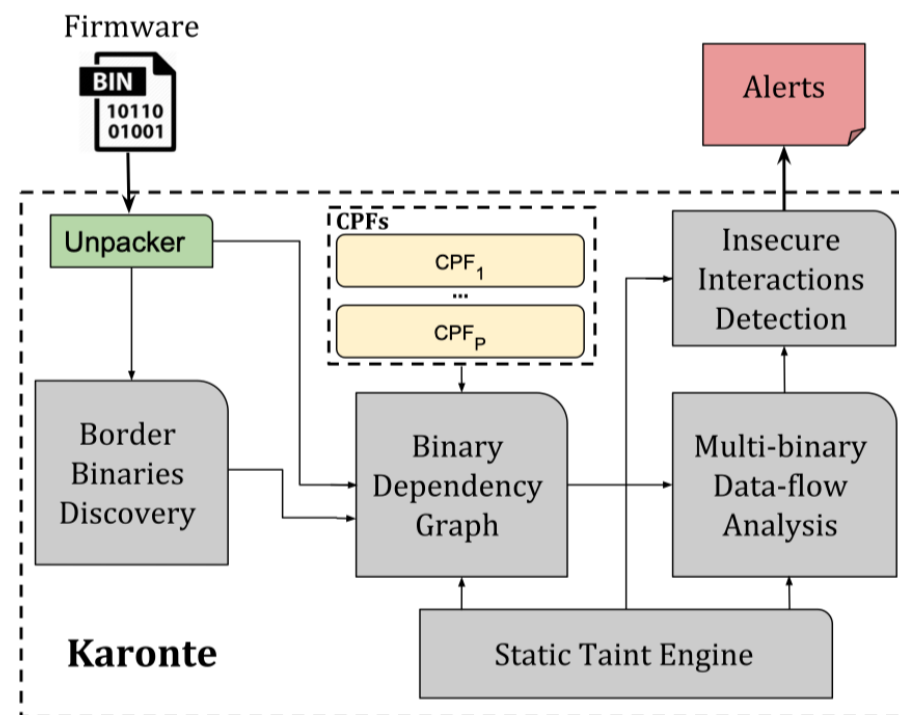
用“数据键”把分散程序连接成全局数据流网络，使跨进程漏洞可以被静态分析捕获，并大幅压低无关告警数量

```
1  int process_request(char *query, char *log_path) {
2      char *q, arg[128];
3      char log_dir[128];
4      if (!(q=strchr(query, "op=")))
5          return;
6      strcpy(arg, q); // query string argument
7      strcpy(log_dir, dirname(log_path));
8      // ...
9      return 0;
10 }
11 int main(int argc, char *argv[], char *envp[]) {
12     char *query = getenv("QUERY_STRING");
13     char *log_path = getenv("LOG_PATH");
14     process_request(query, log_path);
15 }
```



KARONTE 系统架构总览

1. 固件预处理 (**Firmware Pre-processing**): 解包固件镜像, 提取文件系统与二进制可执行文件
2. 边界二进制文件发现 (**Border Binaries Discovery**): 自动化识别对外暴露网络服务、接收攻击者初始输入的二进制文件
3. 二进制依赖图恢复 (**BDG Recovery**): 基于通信范式查找器 (CPF, Communication Paradigm Finders), 例如基于文件、Socket, 推断并重构二进制文件之间的数据流网络
4. 多二进制数据流分析 (**Multi-binary Data-flow Analysis**): 结合跨文件数据流进行具备约束感知的静态污点分析
5. 不安全交互检测 (**Insecure Interactions Detection**): 识别违背安全约束的污染数据流并输出漏洞警报



关键模块1: 边界二进制文件发现

➤ 设计目标: 定位攻击者输入在固件中的初始立足点（边界二进制文件）

- 边界二进制（**Border Binaries**）定义：在固件样本中，负责导出网络服务并直接接收、解析外部（网络）用户提供数据的二进制程序
- 自动化识别机制：
 - KARONTE 扫描固件中的所有二进制文件，寻找调用了网络数据读取函数（如 `recv`, `read` 应用于 `socket` 描述符）的代码块
 - 将这些网络数据读取点标记为初始的污点源（**Taint Sources**）
- 分析缩减：通过界定边界二进制文件，系统能够排除与外部输入无关的底层守护进程，极大缩小后续数据流分析的搜索空间

关键模块2: 通信范式查找器（上）

➤ 设计目标: 抽象与建模多种进程间通信机制 (Communication Paradigm Finders, CPFs)

- 为了识别二进制文件间的数据共享, KARONTE 设计了一系列模块化的通信范式查找器 (CPFs)
- 每种 CPF 针对特定的 IPC 范式执行以下核心任务
 1. 数据键恢复 (Data Key Recovery): 提取标识共享数据的关键值 (如环境变量名 "CONTENT_TYPE")
 2. 流向判定 (Flow Direction Determination): 识别代码中访问该数据键的程序点, 并判定其角色是数据写入者 (Setter) 还是数据读取者 (Getter)
 3. 二进制集合放大 (Binary Set Magnification): 在整个固件样本中交叉搜索引用了相同数据键的其他二进制文件, 从而发现潜在的通信接收方或发送方
 4. 支持的范式: 包括环境变量 CPF、文件 CPF、Socket CPF 等

关键模块2: 语义 CPF 设计（中）

➤ 设计目标：面向无符号及底层架构的通用通信建模

➤ 在许多嵌入式操作系统或闭源固件 Blob（如 Bootloader）中，标准的系统调用抽象可能缺失

➤ KARONTE 提出了不依赖操作系统的语义 CPF（Semantic CPF）：

- **核心原理：**进程间的任何通信必然涉及对共享信息（即数据键）的索引

→ 在二进制层面，这些数据键通常表现为硬编码的常量值（如特定的内存地址或特征字符串）

- **匹配策略：**Semantic CPF 通过静态分析二进制代码，检测硬编码值是否被用作内存索引以访问感兴趣的数据（如攻击者控制的数据）

关键模块2：二进制依赖图构建（下）

➤ **设计目标：** 重构多组件环境下的全局数据流拓扑

➤ 基于 CPFs 收集的通信端点与数据键，KARONTE 构建了一个有向图

➤ 二进制依赖图（Binary Dependency Graph, BDG）

1. 图的定义： $G=(B,E)$; 其中节点 B 代表处理受控数据的二进制文件集合
2. 有向边属性： 边 E 由一个三元组表征，包含源节点属性、目标节点属性以及关联的数据键 k
3. 连接逻辑： 如果二进制文件 $b1$ 在某处作为 Setter（如调用 `setenv` 设置了键 k ），而 $b2$ 作为 Getter（如调用 `getenv` 获取了键 k ），则在 BDG 中生成一条从 $b1$ 指向 $b2$ 的有向边

➤ 作用： BDG 精确描绘了污染数据如何跨越进程边界传播，为后续全局污点跟踪提供了导航路径

关键模块3：多二进制静态污点分析

➤ 设计目标：具备约束感知的跨边界污点追踪

➤ KARONTE 基于 BDG (Binary Dependency Graph)对固件执行深度的静态污点传播与符号路径探索

1. **跨边界传播机制**：当污点数据在二进制文件 b1 中到达一个 IPC Setter 点时，引擎会提取当前的污点数据及其路径约束 → 随后，分析引擎跳转至 BDG 中相连的二进制文件 b2 的 Getter 点，并继续追踪
2. **路径优先级策略（Path Prioritization）**：为缓解路径爆炸，KARONTE 优先探索函数内那些具有复杂返回值的基本块路径，而延后或剪枝总是返回常量的无害路径
3. **状态清洗（Untaint Policies）**：当内存被安全函数（如带有长度校验的拷贝）覆盖，或约束已被严格限制时，及时移除污点标签，降低误报

关键模块4：不安全交互检测

➤ 设计目标：基于污点状态与约束的安全违规判定

➤ 在数据流图和约束条件收集完毕后，检测模块将评估是否存在安全漏洞：

1. **规则驱动的检测**：关注用户控制的数据在未经过充分验证（Sanitization）的情况下，是否流入敏感函数
2. **内存破坏漏洞（Memory Corruption）**：检测受污染数据是否未经长度约束便作为 strcpy, memcpy 等函数的源缓冲区传入，从而引发缓冲区溢出
3. **拒绝服务（Denial-of-Service, DoS）**：检测攻击者控制的数据是否直接影响内存分配函数（如 malloc）的大小参数或循环条件，引发资源耗尽
4. **扩展性**：可根据需求自定义 Sink 规则（如 UAF 或命令注入），利用已建立的 BDG 与污点信息进行检测

实验评估与关键数据

- **漏洞评估数据集：** 包含 53 个固件样本（来自 NETGEAR, TP-Link, D-Link, Tenda 等及部分底层 Firmware Blobs）
 1. 0-day 漏洞发现
 - KARONTE 成功发现了 46 个真实的 0-day 漏洞
 2. 消除警报爆炸
 - 与不考虑交互的传统单二进制静态分析相比: KARONTE 将每个样本的平均警报数从 722 个锐减至 2 个
 - 警报总数降低了两个数量级（从 20,931 降至 74），解决了静态分析的误报瓶颈

文章评价

➤设计优势

- 奠定了多二进制 IPC 数据流抽象的基础，是后续跨组件固件分析的重要基础设计

➤局限性分析

1. 符号执行开销重，边界二进制过滤会产生漏报
2. 通信范式覆盖不全时，整个跨组件图会断裂

➤商业化潜力

- 依赖建模完备性，符号执行开销重
- 仅适合高价值样本的离线分析，不适合作为批量商用扫描主引擎

Operation Mango: Scalable Discovery of Taint-Style Vulnerabilities in Binary Firmware Services

Wil Gibbs*, Arvind S Raj*, Jayakrishna Menon Vadayath*, Hui Jun Tay*, Justin Miller*, Akshay Ajayan*,
Zion Leonahenahe Basque*, Audrey Dutcher*, Fangzhou Dong*, Xavier Maso[†],
Giovanni Vigna[‡], Christopher Kruegel[‡], Adam Doupé*, Yan Shoshitaishvili*, Ruoyu Wang*

**Arizona State University*

{wfgibbs, arvindsraj, jvadayat, htay2, jmill, aajayan, zbasque, dutcher, fdong12, doupe, yans, fishw}@asu.edu

‡University of California, Santa Barbara

{vigna, chris}@cs.ucsb.edu

†contact@xaviermaso.com

USENIX Security 2024

内容摘要

➤ 关键动机

- KARONTE 边界二进制过滤虽然提升可扩展性，却会漏掉大量内部组件漏洞
- 布尔污点模型容易产生过度污点化误报

➤ 本文工作: 面向二进制固件服务的可扩展 Sink-to-Source 数据流分析框架

1. 除了标记变量是否“被污染”外，还用混合值域和富表达式记录字符串、整数、拼接和转换历史
2. 从危险 Sink 反向追溯潜在 Source，避免传统正向污点分析在大量无关路径中爆炸
3. 输出端通过 TruPoCs 机制(ranked alerts)筛选外部可控、值得人工验证的真实漏洞候选
 - TruPoCs (True PoCable vulnerabilities) 指代可被触发的漏洞

➤ 关键数据

- 大规模数据集中分析 770,374 个二进制，发现 10,834 个 TruPoCs，平均 38.12 秒/二进制
- 在 Karonte 基准上分析二进制数量约为 SaTC 的 27 倍

研究背景：传统正向污点追踪的性能瓶颈与盲区

1. 边界二进制过滤的代价：为了保证分析在可接受的时间内完成，现有工具主动放弃了对固件中绝大多数二进制文件的分析

→ 例如，某个由内部组件引发的跨二进制漏洞，若该组件未被识别为边界服务，则永远无法被传统工具检测到

2. 基础分析维度的缺失：传统的污点追踪（Taint Tracking）只能标记变量“是否被污染”，而无法精确推理被污染数据的具体构成与历史操作

→ 导致面对复杂字符串处理（如 sprintf 拼接）产生大量误报

```
1 // R6400 /usr/sbin/httpd
2 int __fastcall sub_6F800(char *a1, int a2)
3 {
4     sub_18764(a1, "iserver_passcode", v11, 2048);
5     strcpy(v10, v11);
6     v4 = acosNvramConfig_set("iserver_remote_passcode", v10);
7     acosNvramConfig_save(v4);
8     system("killall -SIGUSR1 dlnad");
9 }
10 ...
11
12 // R6400 /usr/sbin/dlnad
13 int main(int argc, const char **argv, const char
14     ↪ **envp)
15 {
16     v10 = acosNvramConfig_get("iserver_remote_passcode");
17     sprintf(v11, "echo \"NoDeviceName\\r\\n%s\" >
18     ↪ /tmp/shares/forked_daapd.remote", v10);
19     system(v11);
20 }
```

边界服务之前工作无法识别，从而无法被分析

仅能标记被污染，但是误报

核心洞察

➤ **Insight 1:** 边界二进制过滤会带来系统性漏报

- KARONTE、SaTC 为了提升可扩展性，只分析少量 Border Binaries，但真实固件中的漏洞可能位于内部服务、辅助程序或后利用链路中；因此 MangoDFA 认为应覆盖固件中的所有相关二进制，而不是只看外部入口程序

➤ **Insight 2:** 污点型漏洞不能只看“是否到达 Sink”

- 传统布尔污点只能判断输入是否传播到 system、strcpy 等危险函数，却无法区分输入是否经过 atoi、格式化、长度限制或常量替换等安全转换；MangoDFA 因此用 Rich Expression 记录值的来源、类型和历史操作

➤ **Insight 3:** 从 Sink 反向追踪比从 Source 正向传播更适合大规模固件分析

- 固件中的输入源数量庞大，正向追踪会探索大量无关路径；MangoDFA 从危险 Sink 出发反向查找可控 Source，只分析与危险调用直接相关的数据流，并在遇到常量、安全值或不可利用转换时快速剪枝

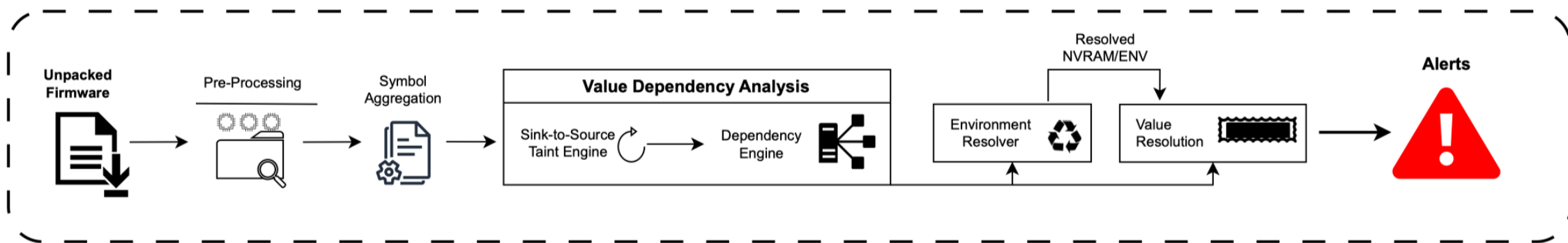
➤ **Insight 4:** 跨二进制传播可以通过环境变量 / NVRAM 的 Setter-Getter 关系轻量拼接

- 很多固件漏洞跨进程传播，但不一定需要重型符号执行；MangoDFA 预先构建环境变量和 NVRAM 的读写字典，在分析时直接连接跨二进制数据流，从而兼顾覆盖率和效率

追求全固件覆盖和可扩展性，用 Sink-to-Source + 富表达式把搜索空间控制在危险数据流附近

MangoDFA 系统架构总览

1. **固件预处理与关键字发现 (Pre-processing & Keyword Discovery):** 解包固件并提取所有 ELF 可执行文件。同时从前端文件（HTML/JS/PHP 等）中收集用于标识用户输入的关键字
2. **进程间通信发现 (IPC Discovery):** 静态收集所有二进制文件对 NVRAM 和环境变量的读写操作，构建跨进程的设值/取值（Setter/Getter）字典
3. **核心分析引擎:** 不局限于正向污点追踪，结合“值分析（Value Analysis）”与“数据依赖分析（Data Dependency Analysis）”的定制化底层引擎
4. **漏洞汇聚点解析 (Resolving Vulnerability Sinks):** 利用本文提出的新型数据流分析技术(MangoDFA)，从敏感函数（Sinks）触发，跨函数、跨二进制地反向解析数据来源，直至确认数据源的可利用性



关键模块1: MangoDFA 的域模型设计

➤ 设计目标: 超越简单污点标记的“富表达式”模型 (Rich Expressions)

➤ MangoDFA 抛弃传统布尔型污点标记, 针对命令注入等漏洞设计混合值域 (Mixed Value Domain)

- 值域定义 (Value Domain): 支持整数、固定长度字符串以及最高抽象层级

→ 每个值都可以表示为多值集合 (MultiValue) 或由多值组合而成的表达式 (Expression)

- 富表达式 (Rich Expressions): 精确记录变量的构成逻辑与历史依赖

→ 例如, 对于操作 `sprintf(cmd, "df -d %d -h %s", atoi(input_1), input_0)`, MangoDFA 能够将其解析为由四部分组成的富表达式

- 优势: 基于富表达式, MangoDFA 能够准确推断出 `input_1` 被转化为整数, 不具备命令注入能力; 而 `input_0` 未受阻拦, 存在真实的注入风险

→ 缓解了污点分析固有缺陷: 过度污点化 (Over-tainting)

规模化优化 I: Sink-to-Source 反向分析

➤ **设计目标：精准剪枝，从结果反推原因的逆向追踪**

➤ 传统的正向分析在遇到污点源时会盲目向下追踪所有可能的分支，导致无效路径极多

- **Sink-to-Source 机制：**MangoDFA 直接从敏感函数（如 `system()`, `strcpy()`）的调用点开始，沿着控制流图逆向追溯潜在的数据源
- **常数与安全值阻断：**在逆向追踪过程中，一旦 MangoDFA 发现某条数据流的源头最终解析为硬编码常数、已知的安全值，或者被不可逆的安全函数处理，则立即终止该数据流的分析

→ 彻底避免了在无害路径上浪费算力，极大地提升了分析速度

规模化优化 II: Assumed Nonimpact策略

➤ 设计目标: 规避路径爆炸的黑盒化调用处理

➤ 在 Inter-procedural 数据流分析中, Step-into 每一个子函数会导致严重的路径爆炸

- **调用规约与原型推断 (Calling Convention & Prototype Inference):** MangoDFA 首先推断出目标子函数的参数数量、类型及全局变量访问情况
- **假设无影响判定逻辑 (Assumed Nonimpact Logic):** 在每个函数调用点, 引擎会进行两项严格评估:
 1. 当前关注的核心变量 (或指向其的指针) 是否被作为参数传入了该子函数?
 2. 该子函数返回的值最终是否会流入当前追踪的漏洞 Sink 点?
- **选择性跳过:** 若上述条件均不满足, MangoDFA 将该函数调用视为黑盒并直接跳过, 假设其对当前追踪的危险数据流“无影响” → 削减跨函数分析的开销

跨边界通信：轻量级环境解析

➤ 设计目标：替代沉重符号执行的跨组件连接机制

- 针对物联网固件中普遍存在的跨二进制文件数据流（如通过 NVRAM 传递），提出了一种轻量级的连接方案：
 - 规避符号探索：Karonte 等传统方法在处理 IPC（进程间通信）时，依赖重量级的约束求解与跨边界符号执行，极易引发状态爆炸
 - 直接数据流链接：在早期的 IPC Discovery 阶段已收集了所有 NVRAM/环境变量的 Setter 和 Getter 字典
 - 在反向数据流分析到达 Getter（如 `nvram_get`）时，Mango 直接查询字典，将数据流链接至对应的 Setter 所在二进制文件继续分析
 - 这种基于静态字典绑定的跨边界解析方案，在保证链路连通性的同时，彻底消除了符号约束求解的运行时开销

漏洞确认与筛选：TruPoCs机制

➤ 设计目标：从海量底层告警到可验证的漏洞报告

➤ 由于 MangoDFA 分析了固件中的所有二进制文件（包含无法从外部触达的死代码）

→ 它会产生数量庞大的原始缺陷告警，为此，系统引入了 TruPoCs 过滤机制以提取真实可触发的漏洞：

1. 数据源分类 (Source Categorization): 基于构建的数据依赖图，Mango 将数据的外部来源分为五类：

- 文件操作（file_ops）、网络操作（network_ops）、环境变量与 NVRAM（env_ops）、命令行参数（argv）以及未知来源（unknown）

2. 可利用性确认： 只有当漏洞路径的源头可以追溯至已知的 env_ops 设值点，或被确认为受用户控制的前端参数（frontend_param）时，MangoDFA 才会将其提升并输出为 TruPoC

3. 有效降低人工验证成本： 相较于传统方法需要人工逆向排查数百万条漏洞上下文路径，TruPoCs 机制将需要人类分析师介入的审查量降低了多个数量级

实验评估与关键结果

- 在 Karonte 提供的基准数据集（49 个固件样本）上，将 MangoDFA 与现有技术进行对比：
 1. 发现更多真实漏洞：MangoDFA 生成了 2,310 个 TruPoCs
 - 经随机采样 100 个命令注入告警进行人工验证，MangoDFA 的真实阳性率（TPR）达到 57%（显著高于 SaTC 的 32.77%）
 - 此外，MangoDFA 在 7 个特定固件中发现了 56 个 SaTC 遗漏的真实漏洞
 2. 成因分析：
 - SaTC 和 Karonte 因为过度依赖“边界二进制（Border Binaries）”过滤策略，直接导致了严重的漏报
 - MangoDFA 覆盖了所有二进制文件，成功捕获了深层的后利用（Post-exploitation）攻击面

实验评估与关键结果

1. 全样本扫描效率对比

- 在上述 49 个样本中，MangoDFA 分析了所有的 6,920 个二进制文件，耗时 946 小时
- 而 SaTC 仅分析了 131 个二进制文件，耗时 860 小时，平均6.56 小时/个
- MangoDFA 的平均单文件分析时间仅 8 分钟/个

2. 海量数据集验证 (Large-scale Dataset)

- 在包含 1,698 个固件样本的大规模真实数据集中，MangoDFA 分析了 770,374 个二进制文件，发现 10,834 个 TruPoCs，平均每个二进制文件的分析时间仅需 38.12 秒

3. 消融实验 (Ablation Study)

- 关闭“假设无影响（Assumed Nonimpact）”会导致分析时间激增 129%
- 关闭“Sink-to-Source反向分析”会使耗时增加 25%

文章评价

➤设计优势

- 在分析时覆盖全固件所有二进制，并用 Sink-to-Source 与富表达式控制规模，解决了边界过滤导致的系统性漏报

➤局限性分析

1. TruPoC 仍是静态分析的漏洞候选，不等于真实 PoC
2. 漏洞类型偏命令注入和缓冲区溢出，对复杂业务逻辑漏洞覆盖有限

➤商业化潜力

- 适合云端批量固件和供应链安全扫描
- 工具输出需配套去重、风险排序和动态验证，否则告警运营成本偏高

OctopusTaint: Advanced Data Flow Analysis for Detecting Taint-Based Vulnerabilities in IoT/IIoT Firmware

Abdullah Qasem*

Security Research Centre
Concordia University
Montreal, Canada

Mourad Debbabi

Security Research Centre
Concordia University
Montreal, Canada

Andrei Soeanu

Security Research Centre
Concordia University
Montreal, Canada

ACM CCS 2024

内容摘要

➤ 关键动机

- 现有固件污点分析常因动态符号执行开销、净化逻辑缺失、间接调用和 NVRAM 传播建模不足而误报漏报

➤ 本文工作: 结合增强 RDA(Reaching Definition Analysis)、净化检查与跨边界传播建模的固件污点分析框架

- 在不依赖昂贵动态符号执行的情况下，提升固件漏洞检测的精度和效率
 1. 以 AIL 中间表示恢复变量语义，基于数据流图从 Sink 反向回溯 Source
 2. 加入净化检查、NVRAM 路径拼接、自定义 Source 发现和间接调用解析

➤ 关键数据

- 在最新固件集中验证 49 个 0-day
- 比 SaTC 快 24%，在复杂间接调用/多参数格式化场景下召回率 83%，显著高于 MangoDFA 的 16%

研究背景：符号执行的性能瓶颈与“过度污点化”

1. 动态符号执行的性能瓶颈

- 传统工具（如 SaTC）从源点 (Source) 向汇聚点 (Sink) 进行前向符号执行，试图穷举所有执行路径
- 在复杂的固件后端中，这会引发计算资源的严重消耗

2. 盲目的污点追踪导致高误报

- 许多被认定为“可达”的污点路径，实际上在中间基本块经历了严格的安全校验（如长度限制、非法字符过滤）
- 由于这些校验不产生新的数据流边缘，传统数据依赖图分析极易忽略它们

3. 间接调用与非标准源的遗漏

- 固件中广泛存在函数指针（间接调用）以及开发者自定义的输入接收函数（封装了原生 API），导致传统工具在构建调用图和识别源点时产生严重漏报

核心洞察：基于逆向追踪与语义检查的数据流分析

➤ **Insight 1:** 弃用动态符号执行，引入 AIL 增强的 RDA

- 采用编译原理中的到达定值分析 (RDA) 替代沉重的符号执行
- 特别是引入了 angr 的 clinic 模块，将 VEX-IR 提升为 AIL (Angr Intermediate Language) 中间语言，直接解决了 VEX-IR 无法精确处理栈变量（仅提供栈地址）的底层缺陷
 - clinic 模块在 angr 反编译流程中负责把 VEX-IR 转换成 AIL，并对函数做早期清理、优化、变量恢复准备。

➤ **Insight 2:** 自汇聚点向源点的逆向回溯 (Sink-to-Source Backtracking)

- 在生成数据依赖图后，通过自底向上（Bottom-up）的深度优先搜索进行逆向回溯，仅关注与 Sink 相关的路径，彻底规避路径爆炸

➤ **Insight 3:** 细粒度的净化语义检查 (Sanitization Inspection)

- 在提取路径后，专门检查路径上的条件分支与字符串长度限制参数，主动剔除已被净化的安全路径

将底层变量恢复、路径净化判断和跨边界传播建模放到同一条分析链中

专门解决真实固件污点分析中最常见的误报/漏报来源

OctopusTaint 系统架构总览

1. 固件解包 (Firmware Unpacking): 利用 binwalk 提取文件系统
2. 预处理与目标筛选 (Pre-Processing): 提取前端文件中的关键字, 匹配并筛选出相关的后端二进制文件 (重点保留引用 libnvram.so 等共享内存库的程序)
3. 核心引擎 (OctopusTaint Engine): 执行核心的 RDA 分析、AIL 转换、DDG 构建, 并协调包括“净化检查”、“逆向回溯”、“NVRAM 处理”、“间接调用解析”等八大子组件
4. 路径连接与过滤 (Connect & Filter): 拼接跨二进制文件的执行路径, 剔除无效的内部调用, 输出最终的高危漏洞告警

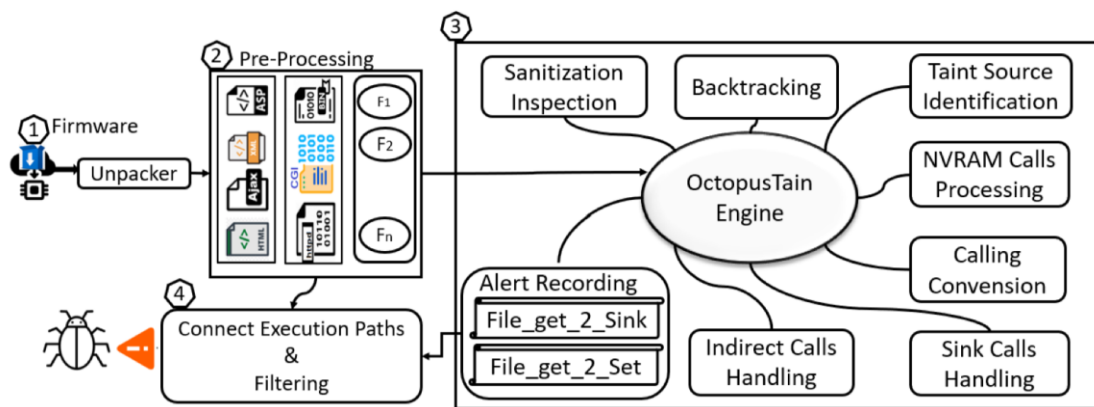


Figure 1: OctopusTaint Architecture and Workflow

核心机制1：污点引擎与逆向回溯

➤ 设计目的：基于 AIL 与 DDG 的高效逆向数据流追踪

1. 中间表示的升维 (AIL Representation): 通过 clinic 模块将基础汇编提升为 AIL

- AIL 为反编译服务，把低级机器语义提升成更接近源代码的表达形式。其能够实施变量恢复与拷贝传播优化，消除冗余数据，并极其精准地模拟 strcpy、sprintf 等危险函数的栈变量行为

2. 逆向回溯算法 (Bottom-up Backtracking)

- 当引擎在代码中遇到危险的系统调用 (Sinks) 时，会提取其关键参数的定义，随后在数据依赖图上启动逆向回溯，直到与前端关键字关联的源点定义 (Source) 建立连接

3. 按需的跨过程分析 (Selective Inter-Procedural Analysis)

- 为了优化性能，仅当被调用函数的至少一个参数已被标记为污点时，引擎才会挂起当前状态，步入 (Step-into) 子函数进行分析，极大减少了不必要的上下文切换

核心机制2：净化检查与误报抑制

➤ 设计目标：通过主动安全约束检验消除“伪漏洞”

1. 安全检查逻辑的提取

- 在回溯生成的候选路径上，引擎会专门评估可能影响 Sink 定义的条件检查语句

2. 危险函数参数校验

- 重点关注字符串操作函数（如 `strncpy`, `snprintf`）的长度参数约束，以及针对非法字符的过滤逻辑

3. 精准剔除 (False Positive Pruning)

- 一旦证实污点数据在到达 Sink 前经历了鲁棒的输入校验或被限制了最大长度，该路径即被标记为“已净化 (Sanitized)”
- 实验表明，该机制成功识别了 EmTaint 误报的 320 个已被净化的安全汇聚点

核心机制3：跨边界追踪与 NVRAM 建模制

➤ 设计目标：重构复杂的进程间通信与间接漏洞路径

➤ 现代固件的 Web 漏洞往往跨越多个程序，OctopusTaint 将此类漏洞定义为“间接/传递性漏洞”

1. 配对追踪机制

- 在逆向分析时，除了记录到达危险函数的路径 (File_get_2_Sink)，还会专门追踪到达 NVRAM 写入函数（如 nvram_set）的路径，记录为 File_get_2_Set

2. 跨文件拼接 (Path Connection)

- 在后处理阶段，系统通过匹配相同的 NVRAM 关键字 Token，递归地将当前程序的 nvram_get 溯源至另一个二进制文件中的 nvram_set，从而还原完整的跨进程数据流

核心机制4：新污点源发现与间接调用解析

➤设计目标：挖掘未知攻击面与修复控制流断层

1. 新污点源识别 (New Taint Source Identification)

- 开发者常封装原生 API（如定义包裹了 `websGetVar` 的自定义函数）
- OctopusTaint 通过识别“接收关键字作为参数，并将用户输入写入栈或全局缓冲区的函数”，自动推断出新的自定义污点源

2. 基于数据依赖图的间接调用解析 (Indirect Call Handling)

- 面对通过寄存器进行函数调用的间接跳转，OctopusTaint 在 DDG 上回溯该调用寄存器的定义
- 若溯源至包含函数地址的只读内存段，即可解析出实际的调用目标，从而重新建立准确的 CFG 进行后续追踪

实验评估与关键数据

➤ 四大评估数据集

- 包括自建最新固件集（160个）、SaTC 基准集
- 包含 1542 个固件的大规模在野数据集，涵盖主流路由器及工业物联网设备

➤ 0-Day 漏洞发现

- 在最新固件集中精准过滤出 109 个高危警告
- 人工验证了 49 个 0-Day 漏洞（含 7 个直接漏洞与多起跨进程传递性漏洞）

➤ 对比评估

- 对比 SaTC (执行效率优势): OctopusTaint 耗时 70 小时完成基准测试，比 SaTC（92 小时）快 24%
- 对比 MangoDFA (细粒度精度优势): 同为基于 RDA 的工具，MangoDFA 在面对复杂间接调用（如 Cisco 固件）和参数极多（超出寄存器数量）的格式化函数时表现不佳，召回率仅 16%，而 OctopusTaint 凭借 AIL 优化和稳健的底层修复，召回率高达 83%

文章评价

➤设计优势

- 对真实误报来源处理细致，尤其是净化检查、AIL 变量恢复、NVRAM 拼接和间接调用解析

➤局限性分析

- 系统依赖 angr/AIL/DDG 等底层恢复质量
→ 遇到反汇编失败、强混淆或架构支持不足时效果会明显波动

➤商业化潜力

- 适合作为高精度静态分析后端，但部署维护成本不低
- 更适合大型设备厂商内部平台

FirmCross: Detecting Taint-Style Vulnerabilities in Modern C-Lua Hybrid Web Services of Linux-based Firmware

Runhao Liu*, Jiarun Dai[†], Haoyu Xiao[†], Yuan Zhang[†], Yeqi Mou*, Lukai Xu*, Bo Yu*, Baosheng Wang*, Min Yang[†]

*National University of Defense Technology, [†]Fudan University

runhaoliu@nudt.edu.cn, {jrdai, hyxiao20, yuanxzhang}@fudan.edu.cn

{myq_2000, xulukai17, yubo0615, bswang}@nudt.edu.cn, m_yang@fudan.edu.cn

NDSS 2026

内容摘要

➤ 关键动机

- 大量现代固件采用 C-Lua 混合 Web 服务
- 传统 C-only 或 Lua-only 工具无法追踪跨语言输入传播和混淆 Lua 字节码

➤ 本文工作：提出面向C-Lua 混合 Web 服务固件的静态程序分析框架

- 利用 Lua 字节码不变量和静态 Diffing 推断混淆规则，恢复可分析 Lua 代码
- 识别 URI 注册、Lua 表访问、参数读取模式和 Lua/C 危险 Sink
- 建模 luaL_register、os.execute 调用外部二进制等 API/IPC 通道，在底层集成 MangoDFA 分析 C 侧Sink

➤ 关键数据

- 在 73 个固件上检测到 696 个漏洞，其中 610 个为 0-day，漏洞检出覆盖率达到 96.67%
- 漏洞覆盖率比 MangoDFA 提升 6.82 倍

研究背景：C-Lua 混合架构与被忽略的攻击面

- **Lua 的兴起与混合架构采用：** Lua 因其高性能和灵活性成为实现 Web 服务的流行语言
 - 对 4,012 个商业设备固件的大规模研究显示，高达 38% 的样本采用了 C-Lua 混合框架，广泛利用 Lua 脚本进行 URI 分发和处理
- **被忽略的攻击面：** 现有的面向 C 二进制的漏洞检测器完全忽略了涉及 Lua 脚本或字节码的漏洞模式
- **工具不兼容的根本原因：** C 二进制文件和 Lua 代码之间存在巨大的语义差异，并伴随着复杂的跨语言交互。如果不进行彻底的重新设计，现有静态方法很难直接应用
- **现实障碍（字节码混淆）：** 在依赖 Lua 字节码（占 34%）的混合固件中，高达 98% 实现了厂商特定的字节码混淆方案，这极大阻碍了任何静态分析的尝试

研究动机：漏洞成因跨语言边界

1. **Lua 处理器接收输入：** wol_wake(input_table, URI_path) 接收已被预先解析为 Lua 表结构的攻击者输入

2. **Lua 中的命令注入：** 输入 user_if 被直接拼接并在 Lua 中传递给 lib.forkExec(), 缺乏严格的过滤

3. **跨边界数据流 (C-Lua 交互)：** 源自 Lua 的 mac 和 ip 输入通过调用传递给名为 portscan 的易受攻击的 C 二进制文件

4. **C 二进制中的缓冲区溢出：** 在 C 二进制文件内部，mac 参数被直接传递给 sprintf() 而没有进行大小检查，导致缓冲区溢出

➤ **盲区：** 如果不建立跨语言通信模型，标准污点分析完全无法检测到此类漏洞

```
01. -- URI handler in Lua
02. function wol_wake(input_table, URI_path)
03.     log("request URI path" .. URI_path)
04.     local user_if = input_table.user_if -- get the user_if field
05.     local mac = input_table.mac -- get the mac field
06.     local cmd1 = string.format([[/usr/bin/ether-wake -b -i %s %s]],
07.                               user_if, mac)
08.     lib.forkExec(cmd1) -- command injection in Lua
09.     local ip = input_table.ip
10.     local cmd2 = "/usr/sbin/portscan "..lib._cmdformat(ip).." "
11.                ..lib._cmdformat(mac).." portscan"
12.     lib.forkExec(cmd2) -- execute vulnerable binary portscan
13. end
14. -- vulnerable function in binary portscan
15. int sub_4027CC(char *mac)
16. {
17.     char file_name[512];
18.     memset(file_name, 0, sizeof(file_name));
19.     // buffer overflow
20.     sprintf(file_name, "/tmp/portscan_result/%s", mac);
21.     ...
22. }
```

研究目标与三大核心挑战

- 研究目标：提出自动化静态检测工具 FirmCross，可适用于现代 C-Lua 混合 Web 服务
- 1. 挑战一：字节码混淆：混淆包含结构混淆（打乱顺序）和数据混淆（转换值）
 - 现有工作（如 LuaHunt）依赖繁重的人工逆向工程，且在面对多种数据混淆共存时会导致探索空间呈指数级增长
- 2. 挑战二：污点源识别：C 二进制使用指针和专用函数，而 Lua 使用表结构来解析输入
 - 现有工具无法定位 Lua 特有的中间源，产生严重的误报和漏报
- 3. 挑战三：数据流模型：C 和 Lua 之间存在频繁且多样化的通信（平均每个固件 167 次）
 - 缺乏准确的 C-Lua 通信模型会导致潜在的跨语言漏洞完全逃脱自动化系统的检测

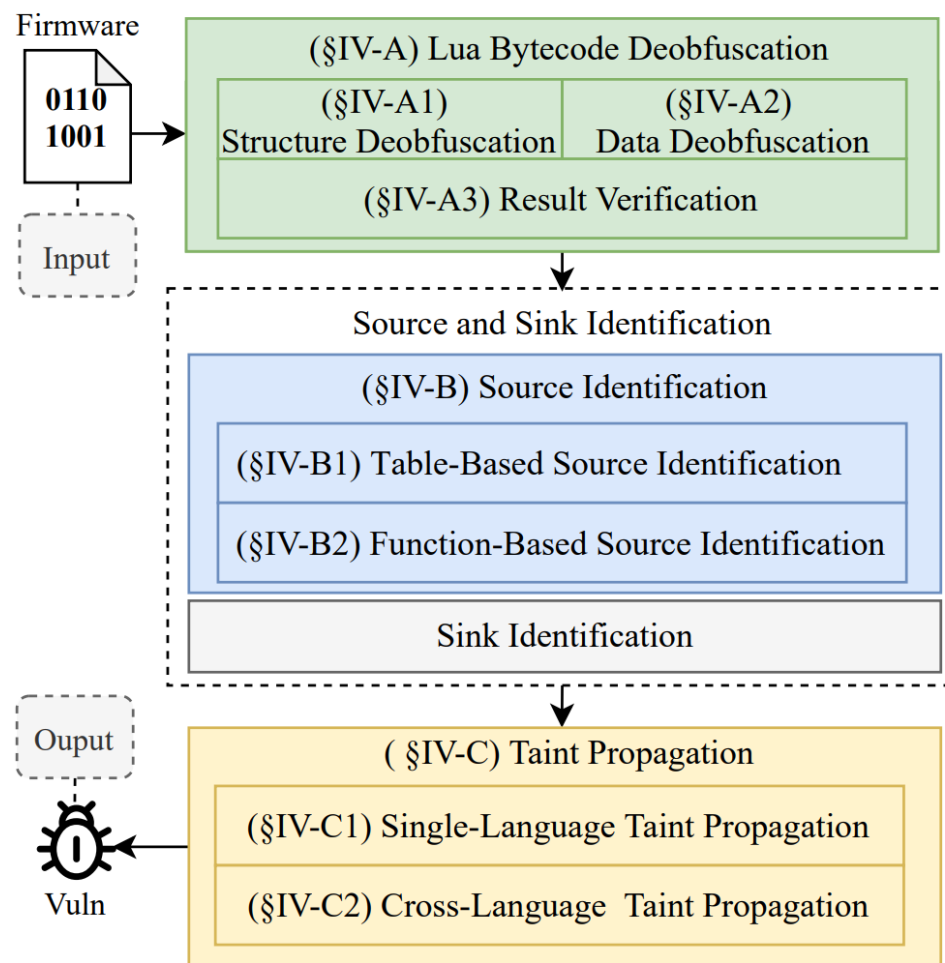
核心洞察

- **Insight 1: 提取不变量以应对混淆:** 混淆后的字节码仍保留长度头、固定终止结构等不变特征
→ 通过提取特征签名推断结构, 并通过静态 Diffing 推断数据转换规则, 从而避免昂贵的变异测试或逆向工程
- **Insight 2: 利用注册机制识别污点源:** Lua URI 处理器遵循高度确定性的注册回调机制
→ 基于参数模式(嵌套表或字符串)和代码中的注册逻辑来识别处理器及攻击者可控的参数
- **Insight 3: 跨语言数据流的确定性模式:** C-Lua 通信遵循严格的标准化范式, 包括基于 API 的模型(如 lua_State 结构调用)和基于 IPC 的模型(如追踪 os.execute() 的命令行参数)

把 C-Lua 边界当作核心问题处理, 因此能覆盖传统 C-only / Lua-only 工具都断掉的漏洞链

FirmCross 的工作流概览

1. **去混淆 (Deobfuscate):** 安全地将混淆的 Lua 字节码转换回可分析的正常字节码
2. **识别 (Identify):** 同时为 C 和 Lua 识别高特异性的污点源和危险汇聚点
3. **传播 (Propagate):** 构建完整的通信模型，并在语言内部和跨语言执行严谨的污点追踪



技术实现概览

1. 结构与数据去混淆:

- **结构:** 通过提取特定偏移量签名并在密文中匹配, 计算结构字段之间的偏移量, 推断厂商的混淆方案
- **数据:** 利用静态 Diffing 揭示数据偏移、异或操作和格式转换等核心转换规律

2. Lua 污点源识别:

- 通过参数模式发现候选注册函数, 并利用内部逻辑进行细致验证
- 通过检测参数中的嵌套访问模式, 将攻击者可控的源与不可控的原始类型区分开来

3. 跨语言污点追踪:

- **基于 API:** 识别通过 `luaL_register()` 注册的 C 函数并追踪 Lua 调用, 同时识别 C 到 Lua 的 API 加载序列
- **基于 IPC:** 监控执行函数 (如 `os.execute()`) 中的命令字符串, 追踪参数是否流入目标二进制文件

实验评估与关键数据

- **实验数据集：**包含来自 11 个不同厂商的 73 个固件镜像，其中约有 14,500 行 Python 实现代码，并集成了 MangoDFA 进行 C 语言底层污点分析
- **优于现有最先进技术 (SOTA)：**FirmCross 检测到了 696 个漏洞，漏洞检出覆盖率为 96.67%
- **漏洞覆盖率大幅提升：**漏洞覆盖率分别比 MangoDFA (仅 C 语言) 和 LuaTaint (仅 Lua) 提高了 6.82 倍和 14.5 倍
- **真实世界的 0-Day 发现：**在 696 个漏洞中，有 610 个是 0-day 漏洞
 - 包含 557 个命令注入和 53 个缓冲区溢出
 - 获得 31 个 CVE-ID，收到了小米、TP-Link、D-Link 和 Tenda 等厂商的官方致谢

实验评估与关键数据

- **去混淆能力 (RQ2):** 在测试 316 个厂商定制的解释器时, FirmCross 达到 100% 的通过率, 平均每个耗时仅 18.03 秒; 而依赖人工逆向的现有工具 LuaHunt 通过率为 0%
- **污点源识别 (RQ3):** FirmCross 在 10 个厂商中识别出 23,306 个有效源 (准确率 84%), 直接促成了 588 个 Lua 漏洞的发现; 大幅领先于 LuaTaint (准确率 18%)
- **跨语言网络 (RQ4):** 成功映射了 1,622 次 IPC 通信和 10,584 次 API 通信, 并发现了 6 个完全源自 Lua 但在 C 二进制中触发的跨语言漏洞, 这些漏洞是传统单语言工具的盲区

文章评价

➤设计优势

- 首个解决 C-Lua 混合固件分析痛点的技术

➤局限性分析

1. 技术栈重且特化于 Lua 生态，对 Python/JS/厂商 DSL 的迁移成本高
2. 不同厂商混淆器和运行时差异会持续增加维护压力

➤商业化潜力

- 对 OpenWrt / Lua 管理面设备厂商很有价值
- 但市场面相对垂直，不适合作为通用固件扫描产品

维度二总结：复杂后端架构与数据流

跨边界与架构泛化问题

Karnote 【IEEE S&P 2020】

从局限于单体到多二进制交互
(BDG) 的IPC建模

FirmCross 【NDSS 2026】

实现C-Lua混合结构的
跨语言污点追踪

大规模分析性能问题

MangoDFA 【USENIX SEC 2024】

从符号执行转向RDA分析，结合
Sink-to-Source逆向回溯分析

OctopusTaint 【ACM CCS 2024】

基于 RDA + AIL 的污点回溯与净
化检查

维度三：隐藏接口发现 (Sink)

- **核心痛点：**IoT设备往往包含大量因历史遗留或调试所需的“隐藏Web接口”，无文档、无入口、无认证，成为重大安全隐患
- **技术演进逻辑（从盲测到智能推断）**
 1. **基石（静态解析与盲测）：**从固件中提取所有文件名和路径结构暴力拼接，并通过响应聚类算法（如返回相同404状态码为一组）来筛选出真实的隐藏接口【WWW, 2022】
 2. **演进（路由分析与LLM推断）：**进一步提出分析固件的“路由机制”。提取公共接口的路由Token（如动作、文件名），利用大模型（LLM）强大的模式学习能力，智能推断出隐藏的路由规则并指导定向Fuzzing【NDSS 2025】
 3. **深化（多语言接口规格恢复）：**面向 C/C++、Lua/Python、动态库混合后端，恢复入口映射、跨语言调用链和层级参数规格【PANGOLIN, USENIX Security 2026】

Game of Hide-and-Seek: Exposing Hidden Interfaces in Embedded Web Applications of IoT Devices

Wei Xie, Jiongyi Chen*, Zhenhua Wang, Chao Feng,
Enze Wang, Yifei Gao, Baosheng Wang, Kai Lu

{xiewei,chenjiongyi,wzh15,chaofeng,wangenze18,gaoyf,bswang,kailu}@nudt.edu.cn
National University of Defense Technology

WWW 2022

内容摘要

➤ 关键动机

- IoT 设备存在大量未公开、无认证的隐藏 Web 接口
- 传统漏洞检测工具缺乏针对访问控制和隐藏接口的自动化发现能力

➤ 本文工作: 面向 IoT 设备隐藏 Web 接口的静态枚举与黑盒验证框架

1. 从文件系统路径、Web server 二进制字符串、文件名和后缀组合候选 URL
 2. 对每个 URL 发送携带认证信息与不携带认证信息的“双子请求”
- 若双子请求响应差异明显，则说明接口受保护；若响应相同，再通过响应聚类过滤伪 404，并用参数差分测试或敏感关键词字典确认配置修改 / 信息泄露类接口

➤ 关键数据

- 在 17 台真实设备上识别 44 个漏洞，其中 43 个为未知漏洞，并获得 8 个 CVE 编号

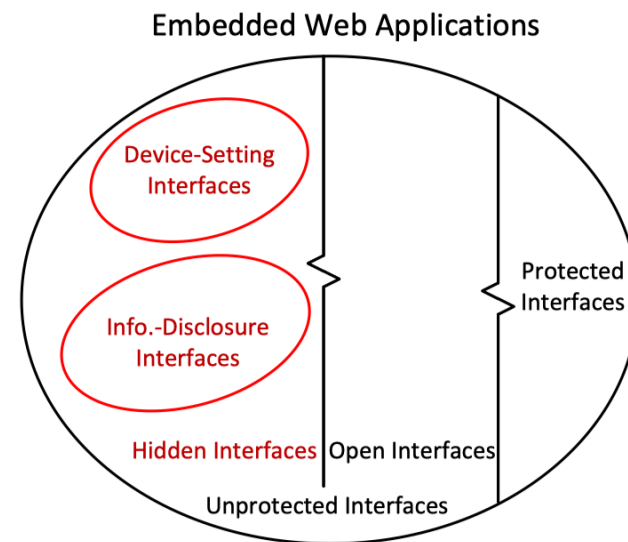
研究背景：访问控制漏洞的自动化检测困境

➤ 传统 IoT 漏洞挖掘工具的局限性

- 现有技术主要聚焦于内存破坏或传统污点型漏洞，缺乏对 Web 接口漏洞的针对性检测

➤ 隐藏接口发现的三大挑战

1. **接口枚举困难：**Web 接口形态多样，不仅表现为独立的文件（如 .php, .cgi），还常作为内部函数硬编码在 Web 服务器二进制文件（如 httpd）中，传统路径爬取无法触达
2. **未保护接口识别困难：**嵌入式 Web 服务器往往不遵守标准的 HTTP 状态码规范（如遇到无效路径时不返回 404，而是返回自定义的 200 错误页面），导致无法直接通过状态码判断接口是否有效且未受保护
3. **隐藏属性判定困难：**触发隐藏接口通常不会导致程序崩溃，缺乏明确的反馈信号来判定某个开放接口是否真正构成了“信息泄露”或“越权配置”的安全威胁



核心洞察

➤ **Insight 1:** 隐藏接口不一定出现在前端页面中，但通常会残留在固件文件系统或 Web Server 二进制中

- 很多 IoT 设备的调试接口、废弃接口和厂商私有接口不会被 HTML/JS 引用，但其 URL 路径、文件名或处理逻辑仍可能以字符串形式保留在固件中，因此可以通过静态提取底层线索生成候选隐藏接口

➤ **Insight 2:** 仅发现隐藏 URL 不等于发现安全问题，关键在于判断它是否绕过认证

- 隐藏接口本身未必危险，真正有安全意义的是匿名用户是否能访问配置修改接口或敏感信息接口
- 因此 IoTScope 将问题从“找 URL”转化为“比较认证请求与匿名请求的访问差异”。

➤ **Insight 3:** HTTP 状态码不能可靠判断嵌入式 Web 接口是否存在

- 许多嵌入式 Web Server 会对不存在的路径返回 200 OK 或自定义错误页，导致传统扫描器误判
- IoTScope 因此使用响应内容相似度聚类，过滤伪 404、默认错误页和无效响应。

➤ **Insight 4:** 隐藏接口漏洞需要结合接口行为进一步确认

- 配置修改类接口需要通过带参数 / 不带参数的响应差异判断是否真实生效
- 信息泄漏类接口则需要匹配从固件 NVRAM、配置文件中提取的敏感关键词，避免把普通页面误判为泄露。

把“隐藏接口发现”转成“认证前后响应是否等价”的黑盒判定问题

IoTScope 系统架构总览

- IoTScope 采用“静态信息提取 + 黑盒动态探测”的混合架构， workflows 包含四个关键阶段

1. 接口枚举 (Interface Enumeration)

- 通过静态分析固件文件系统和二进制可执行文件，提取文件名与路径名，构造潜在的 URL 探测请求

2. 探测请求下发 (Probing Request Delivery)

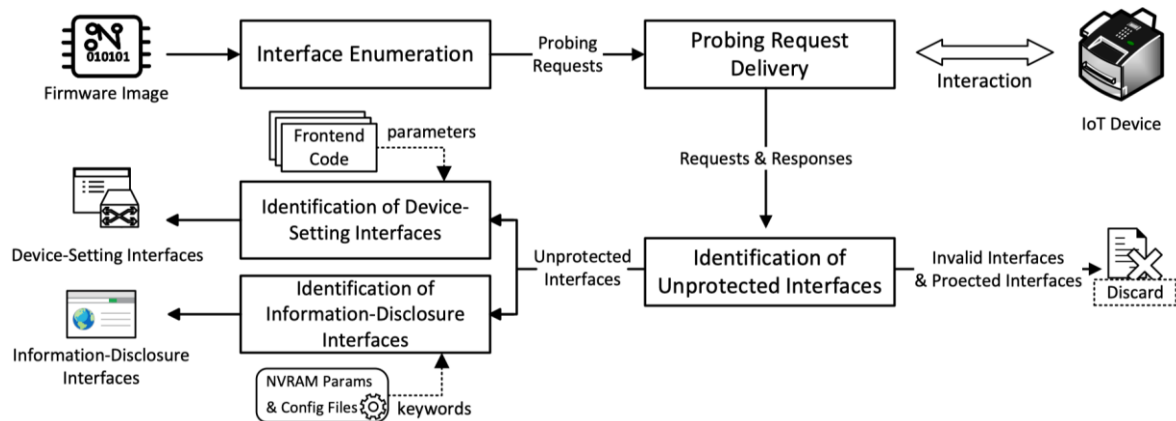
- 采用双子请求 (Twin-requests) 机制向目标物理设备或模拟器发送构造的 HTTP 探测包

3. 未保护接口识别 (Identification of Unprotected Interfaces)

- 利用基于响应字符串相似度的轻量级聚类算法，过滤无效路径和受保护接口

4. 隐藏接口精准判定 (Identification of Hidden Interfaces)

- 结合前端参数和 NVRAM 关键字字典，分别对“设备配置型”和“信息泄露型”隐藏接口进行行为判定

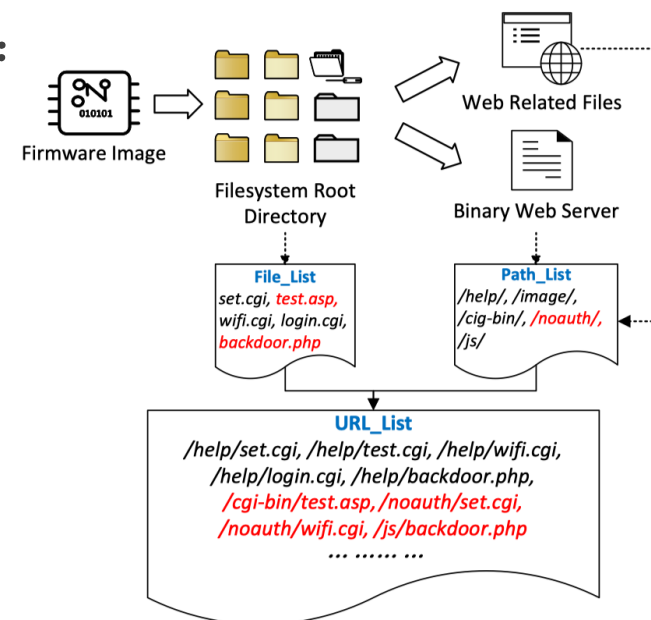


核心模块1：高覆盖率的接口枚举

➤ 设计目标：突破路径爬取限制的静态 URL 构造

➤ 为枚举所有可能的隐藏接口，IoTScope 直接从固件底层提取字符串线索：

1. 文件系统级提取：将整个固件文件系统打包，提取所有符合 Web 接口特征后缀（如 .cgi, .php, .asp, .htm）的字符串，获取独立文件类型的接口
2. 二进制服务器提取：针对硬编码在二进制 Web 服务器（如 mini_httpd, boia）内部的处理函数，提取其二进制文件中的相关目录和路径字符串
3. 结构化拼接策略：将提取到的文件名和路径名通过交叉组合与拼接，构造出海量符合语法规范的 URL 探测列表（平均每个固件生成约 6.2 万个测试 URL）



核心模块2：探测请求下发与双子请求机制

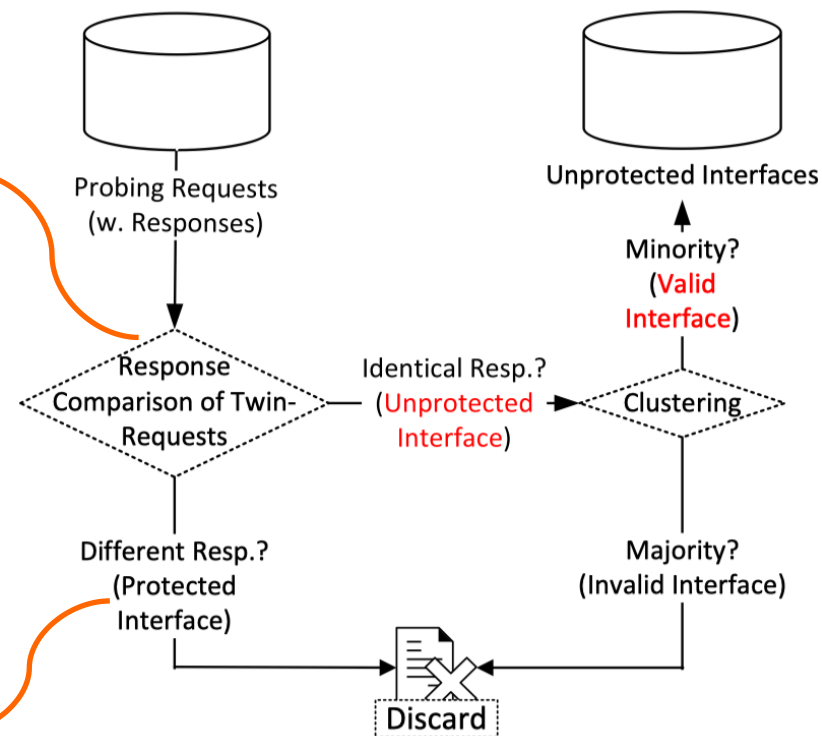
➤ 设计目标：基于黑盒交互的访问控制初步试探

➤ IoTScope 作为一个单线程黑盒探测器，无需对目标固件进行侵入式插桩：

1. 双子请求机制 (Twin-requests): 对于每一个构造出的 URL，IoTScope 均发送两个探测请求：

- 请求 A (携带认证信息): 在 HTTP 头部 (Cookie 或 Authorization 字段) 包含从代理工具抓包获取的合法认证凭证
- 请求 B (无认证信息): 不携带任何身份凭证的匿名请求

2. 初步访问控制推断：如果目标接口对于请求 A 和请求 B 返回了完全不同的响应内容，说明该接口存在访问控制逻辑（即受保护接口，Protected Interface），IoTScope 会将其直接丢弃，仅保留那些对两种请求返回一致响应的候选对象

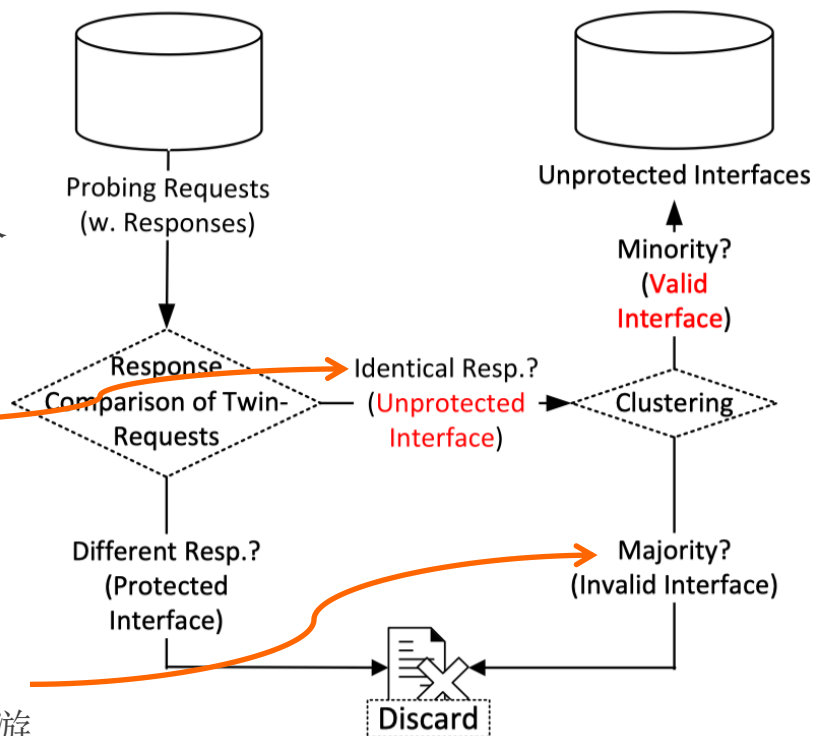


核心模块3：基于聚类分析的未保护接口识别

➤ 设计目标：应对非标准 HTTP 响应的无监督聚类

➤ 针对 IoT Web 服务器不规范的错误处理机制（如用 200 OK 伪装 404 Not Found），IoTScope 引入了轻量级响应聚类算法：

1. 响应相似度度量 (Similarity Measurement): 不依赖 HTTP 状态码，而是计算两个响应报文内容的字符串相似度: $\text{Similarity}(a,b)=2*\text{Mab}/(\text{La}+\text{Lb})$
2. 无监督分组 (Clustering): 将收集到的所有匿名响应报文进行相似度聚类
3. 多数派过滤 (Majority Filtering):
 - 绝大数盲测 URL 都是不存在的无效路径，服务器会返回格式一致的默认错误页面
 - 因此，聚类后规模最大的几个组（Majority）代表了无效接口，将其剔除；而那些游离的少数派（Minority / Outliers）则被确认为真实存在且无访问控制的未保护接口



核心模块4：设备配置型隐藏接口判定（上）

➤设计目标：基于前端参数提取的差分行为测试

➤确认接口开放后，需进一步判定其是否允许未经授权篡改设备配置：

1. **参数提取 (Parameter Extraction)**: 扫描固件中的前端代码（HTML 表单、JavaScript AJAX 请求），利用正则表达式自动提取与后端交互的键值对参数（Key-Value pairs）
2. **差分请求测试 (Differential Testing)**: 向目标未保护接口发送两次探测：
 - 第一次：附加提取到的配置参数
 - 第二次：不附加任何参数
3. **行为判定**: 如果两次请求产生的系统响应（或设备状态）出现实质性差异，则证明该未经授权请求在设备端“生效”，该接口被标记为存在“越权配置”漏洞的高危隐藏接口

核心模块4：信息泄露型隐藏接口判定（下）

- 设计目标：跨组件字典构建与敏感信息匹配
- 部分隐藏接口不会改变系统状态，而是会泄露设备的敏感信息（如管理员密码、网络拓扑）：
 1. 多维关键字字典构建 (Keyword Dictionary)
 - 为了准确识别响应内容中是否包含敏感信息，IoTScope 从固件的 NVRAM 默认参数文件和系统配置文件（如 def_conf.xml）中静态提取键名，构建包含 50 多个高价值标识符的敏感关键字字典
 2. 敏感命中判定 (Information Matching)
 - 对未保护接口的响应文本进行扫描。若该接口返回的文本中命中了字典中的至少两个敏感关键字（如同时出现 password 和 admin 等核心配置项），则将其标记为存在信息泄露风险的隐藏接口

实验评估与关键数据

➤ IoTScope 在包含 11 个主流厂商（如 D-Link, Netgear, Tenda, ASUS 等）、8 种设备类型（路由器、摄像头、GPON 调制解调器等）的 17 款真实物理设备上进行了全面评估：

1. 漏洞发现 (Vulnerability Discovery)

- 成功识别出 44 个真实漏洞（包含 20 个未授权设备配置接口和 24 个敏感信息泄露接口）
- 报告给相关厂商后，所有漏洞均被确认，并已成功获批 8 个 CVE 编号
- 如 CVE-2019-17505 泄露 Wi-Fi SSID 和密码，CVE-2017-5521 泄露管理员账户凭证

2. 执行效率 (Efficiency)

- 处理单台设备的平均耗时仅为 47 分钟，平均每台设备处理 6.2 万个探测请求

文章评价

➤设计优势

- 首个通过黑盒验证确认未授权隐藏接口的技术

➤局限性分析

1. 枚举方式偏暴力，请求量大
2. 主要覆盖 URL 型未授权接口，对认证后隐藏功能和复杂状态接口能力不足

➤商业化潜力

- 商业化使用需严格授权、隔离网络和限速
- 不适合直接做互联网暴露面扫描

EAGLEYE: Exposing Hidden Web Interfaces in IoT Devices via Routing Analysis

Hangtian Liu^{*†§}, Lei Zheng[†], Shuitao Gan^{†§✉}, Chao Zhang^{†‡**}, Zicong Gao^{*},
Hongqi Zhang^{*¶}, Yishun Zeng[†], Zhiyuan Jiang^{||✉}, Jiahai Yang[†]

^{*}Information Engineering University [†]Institute for Network Sciences and Cyberspace (INSC), Tsinghua University

[‡]Zhongguancun Laboratory [§]Laboratory for Advanced Computing and Intelligence Engineering

[¶]Henan Key Laboratory of Information Security ^{||}National University of Defense Technology

^{**}JCSS, Tsinghua University (INSC) - Science City (Guangzhou) Digital Technology Group Co., Ltd.

NDSS 2025

内容摘要

➤ 关键动机

- 隐藏接口不一定表现为未知 URL
- 可能隐藏在 URI、Header、Body 或 Query 中的 Routing Token，IoTScope 覆盖不足

➤ 本文工作：基于路由 Token 推断的 IoT 隐藏 Web 接口发现框架

1. 分析公开请求中的 URI、Query、Header、Body 等字段，识别真正参与路由分发的 Token
2. 利用 LLM 从固件代码上下文中推断隐藏 Token
3. 指导黑盒 Fuzzing 触发隐藏接口

➤ 关键数据

- 在 13 台商用 IoT 设备上发现 79 个隐藏接口，是 IoTScope 的 25 倍，并发现 29 个未知漏洞

研究背景

- Problem: 现有的扫描器 (SaTC/HermeScan) 存在 "Frontend Bias"
 - 假设: 所有后端接口都在前端有迹可循
 - 现实: Hidden Interfaces (隐藏接口) 在前端 HTML/JS 中零引用
- 隐藏接口: 指那些在设备文档中未公开, 但实际上可访问的 Web 接口
 - 调试后门或未成功能, 安全性极差, 是完美的攻击入口
 - 过去对“隐藏接口”的定义不明确, 往往局限于“未授权访问”

研究背景：现有研究局限性

- 静态分析 (Static Analysis) 的困境：
 - 难以定义污点分析的“输入源 (Source)”和“汇聚点 (Sink)”
 - 缺乏明确的代码模式，且存在间接调用 (Indirect Calls) 和数据恢复不完整的问题
 - 多语言混合环境 (C, Lua, PHP 等) 增加了数据流分析的难度
- 动态测试 (Fuzzing) 的难点：
 - Fuzzer 需要目标；由于接口是未知的，无法直接设定目标
 - 随机变异很难触发这些特定的隐藏路径

核心洞察 - 基于路由分析

➤ 路由机制:

- IoT 固件使用特定的输入值（称为 "Routing Tokens"）将请求分发给对应的处理函数

➤ 共享机制:

- 公开接口和隐藏接口通常共享同一套底层路由机制

➤ 转化问题: 将“寻找隐藏接口”的问题转化为“寻找隐藏的路由 Token”

- URI, Header, Body, Query 中都可能包含路由 Token

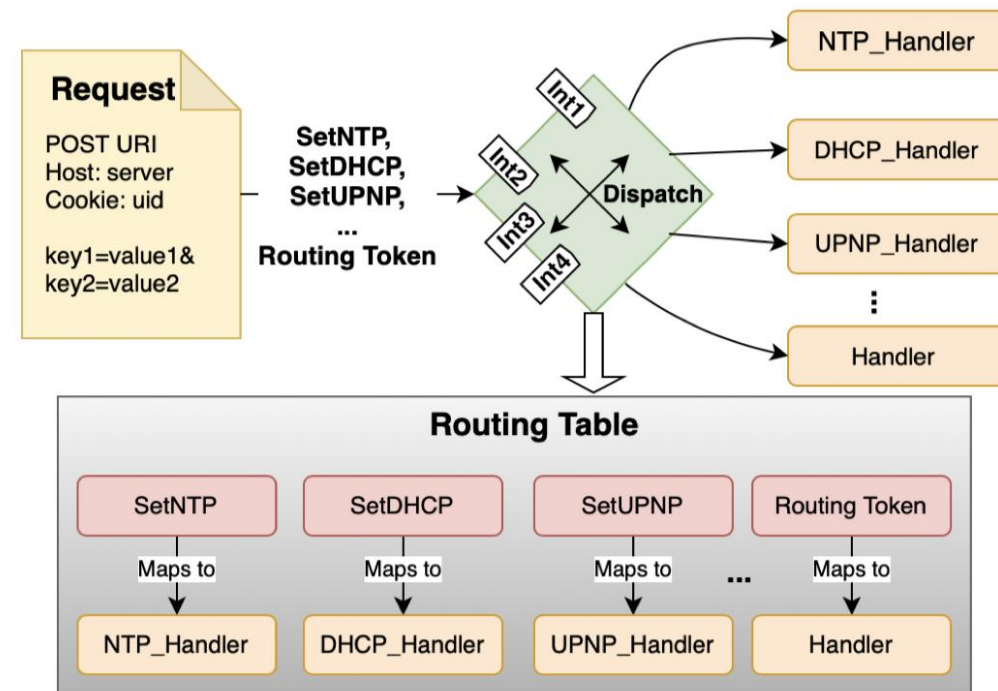


Fig. 1: An example abstract routing mechanism in IoT.

把隐藏接口搜索空间从“所有 URL 暴力枚举”压缩成“有语义约束的 routing token 推断”

因此能覆盖 Header/Body/认证后接口，问题建模比 IoTScope 更深入

EAGLEYE 系统架构概览

- EAGLEYE 采用两阶段方法:

1. 智能路由分析:

- 基于 Token 的比对分析: 识别公开接口中的路由 Token
- LLM 辅助的模式学习: 学习 Token 的代码模式, 从而在固件中发现隐藏的Token

2. 针对隐藏接口的黑盒 Fuzzing:

- 利用提取的 Token 作为字典, 构造针对性的测试用例

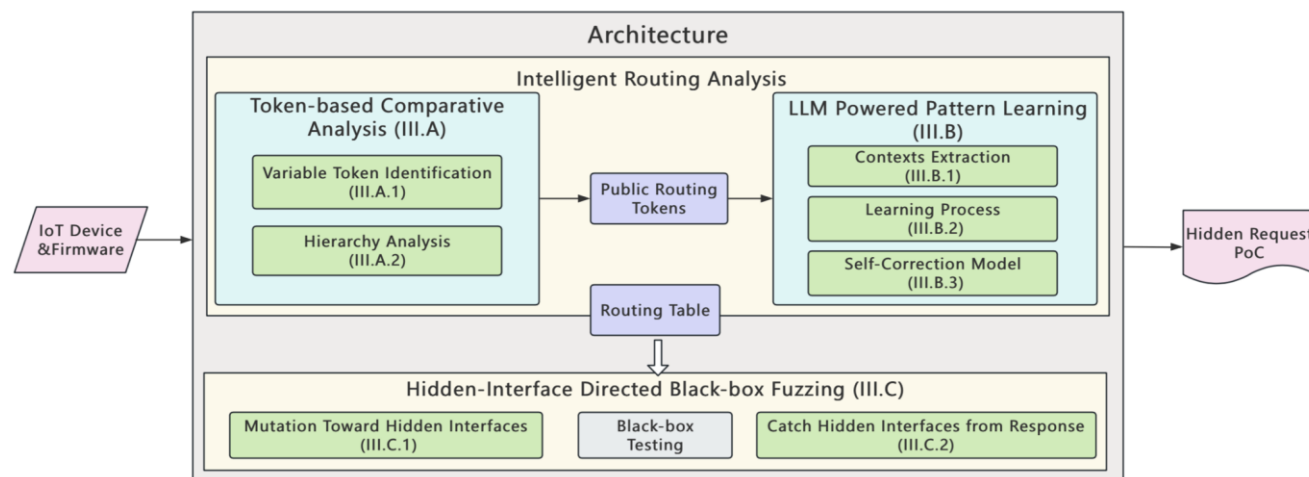


Fig. 4: Workflow Overview.

第一阶段 - 基于 Token 的比对分析

- 目标: 从公开请求中识别出哪些字段是路由 Token
- 步骤:
 1. 变量识别: 收集公开接口请求, 按语法结构 (URI, Query, Header, Body) 分解
 2. 差异比对: 对比不同请求, 找出变化的字段作为潜在的路由 Token, 并过滤掉噪声 (如时间戳、Session ID)
 3. 层级分析: 识别接口的层级依赖关系 (例如: 先有 diagnostic 才能调用 PING), 建立 Token 间的偏序关系

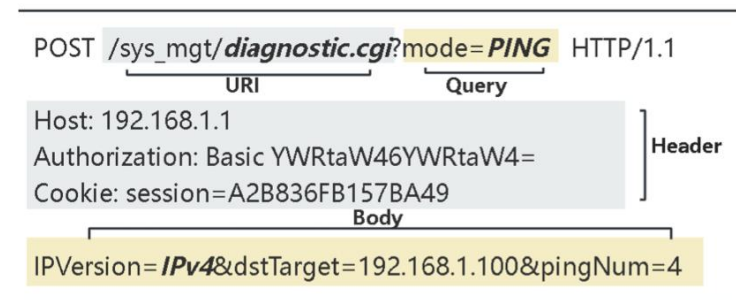


Fig. 5: A request example to illustrate grammatical structure in HTTP.

第二阶段 - LLM 驱动的模式学习

- 洞察:
 - 同一设备中的公开 Token 和隐藏 Token 通常在固件代码中共享相似的代码模式
- 流程:
 1. 上下文提取: 反编译固件, 提取公开 Token 周围的代码上下文 (函数、类、模块)
 2. LLM 分析:
 - 代码分析提示 (Prompt): 让 LLM 分析代码上下文, 提取具有相似特征的其他 Token
 - 正则学习提示: 让 LLM 生成正则表达式以扫描更多 Token

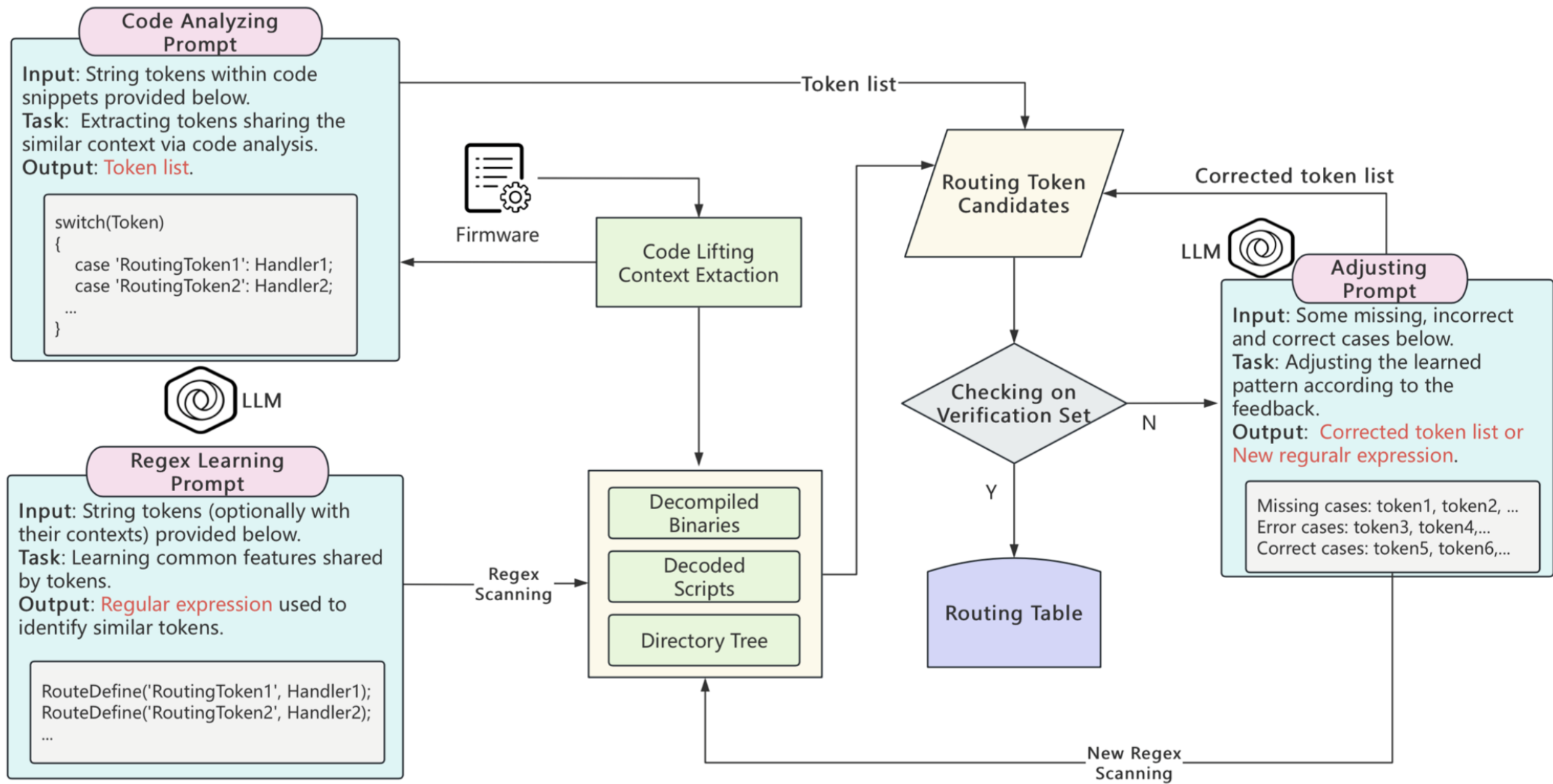


Fig. 6: Two distinct prompts have been crafted to guide the LLM in comprehending the task. An additional adjusting prompt is employed, incorporating feedback from the verification process, to refine and correct the outcomes learned by the LLM.

LLM 的自校正模型 (Self-Correction)

- 挑战: LLM 的初始输出可能包含误报或漏报
- 解决方案: 迭代式的自校正循环
 - 验证集: 保留一部分已知的公开 Token 作为验证集
 - 反馈调整: 检查 LLM 输出是否覆盖了验证集, 将错误/遗漏案例反馈给 LLM 进行 Prompt 调整
 - 结果: 通常在 1-2 轮校正后达到最高准确率

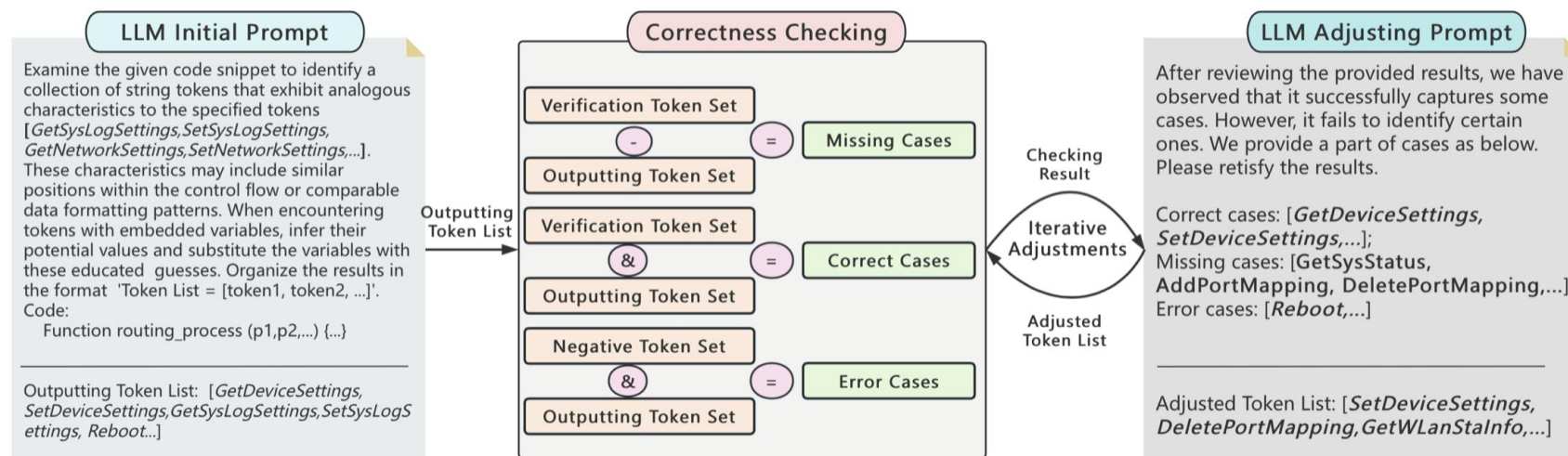


Fig. 12: An illustrative example to demonstrate the specific self-correction process for LLM.

第三阶段 - 定向黑盒 Fuzzing

- 构造请求: 以公开请求为模板
 - 使用提取的“隐藏路由表”作为变异字典，替换原始 Token
 - 根据层级分析结果，处理多级 Token 依赖
- 参数推断:
 - 从设备响应（错误信息、表单）中提取所需参数，不断完善请求体
- 判定标准:
 - 如果请求获得了有效响应（非错误），则视为发现了一个潜在的隐藏接口

实验评估 - 设置

- 测试对象:
 - 13 款来自主流厂商 (Netgear, Cisco, D-Link, TP-Link 等) 的 IoT 设备
- 设备类型:
 - 涵盖 WiFi 路由器, VPN 路由器, NAS, IP 摄像头等
 - 包括二进制, LUA, PHP, Python 等多种后端实现
- 对比基线:
 - IoTScope

Vendor	Model	Version	Device Type	Web Type
Netgear	SRX5308	4.3.5-5	SSL VPN	Bin+LUA
	R7000	1.0.11.136	WiFi Router	Bin+HTM
Cisco	RV-042	4.2.3.14	VPN Router	Bin+HTM
Motorola	CX2L	1.0.1	WiFi Router	Bin
TrendNet	TEW-811DRU	1.0.10.0	Wifi Router	Bin+ASP
Linksys	WRT54GL	4.30.18	Wifi Router	Bin+ASP
Ubiquiti	EdgeRouterX	2.0.9	Ether Router	Bin+Python
DLink	DNS-320	1.11B01	NAS	Bin+PHP
	DIR-823G	V1.0.2B05	Wifi Router	Bin
Mercury	MNVR816	2.0.1.0.5	Video Recorder	Bin+LUA
ZTE	C520	2.1.6T3	IP Camera	Bin+LUA
Zyxel	WSQ50	V2.20	Wifi Router	Bin+LUA
TPLink	Archer A7	V5_1.2.1	Wifi Router	Bin+LUA

实验结果 - 发现能力

➤ 总体发现: 在 13 款设备中发现了 79 个隐藏 Web 接口

- 27 个接口可绕过认证（未授权访问）
- 52 个接口需认证后访问

➤ 准确性: 经人工验证，零误报 (No False Positives)

➤ 与 IoTScope 对比:

- EAGLEYE 发现的隐藏接口数量是 IoTScope 的 25 倍 (79 vs 3)
- 原因: IoTScope 仅关注 URL 中的未授权接口，而 EAGLEYE 覆盖了全类型的 Token（Header/Body）及认证后接口

Vendor	Model	#HINT	#B-Authen	#A-Authen
Netgear	SRX5308	3	0	3
	R7000	3	2	1
Cisco	RV-042	9	1	8
Motorola	CX2L	34	6	28
TrendNet	TEW-811DRU	1	0	1
Linksys	WRT54GL	6	3	3
Ubiquiti	EdgeRouterX	0	0	0
DLink	DNS-320	11	7	4
	DIR-823G	6	6	0
Mercury	MNVR816	1	1	0
ZTE	C520	3	0	3
Zyxel	WSQ50	1	1	0
TPLink	Archer A7	1	0	1
Total	-	79	27	52

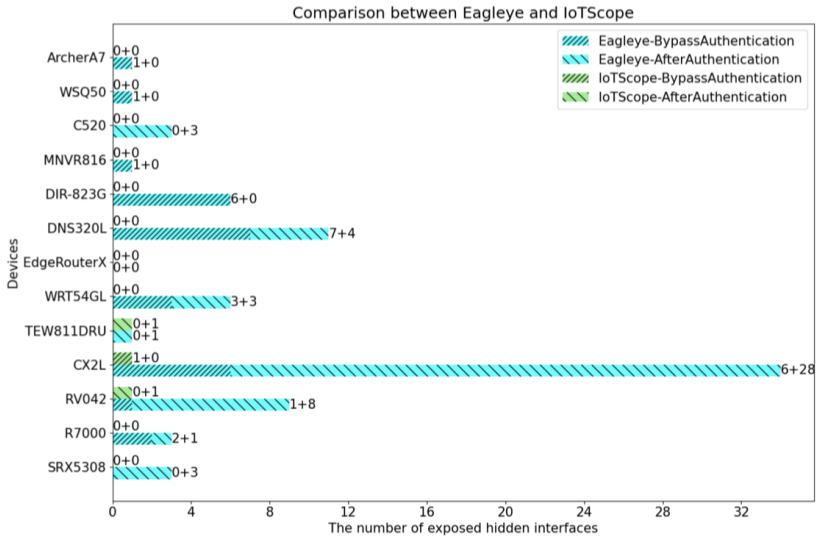


Fig. 7: Comparison with IoTScope. EAGLEYE outperforms IoTScope, exposing 25X more hidden interfaces.

发现的漏洞与危害

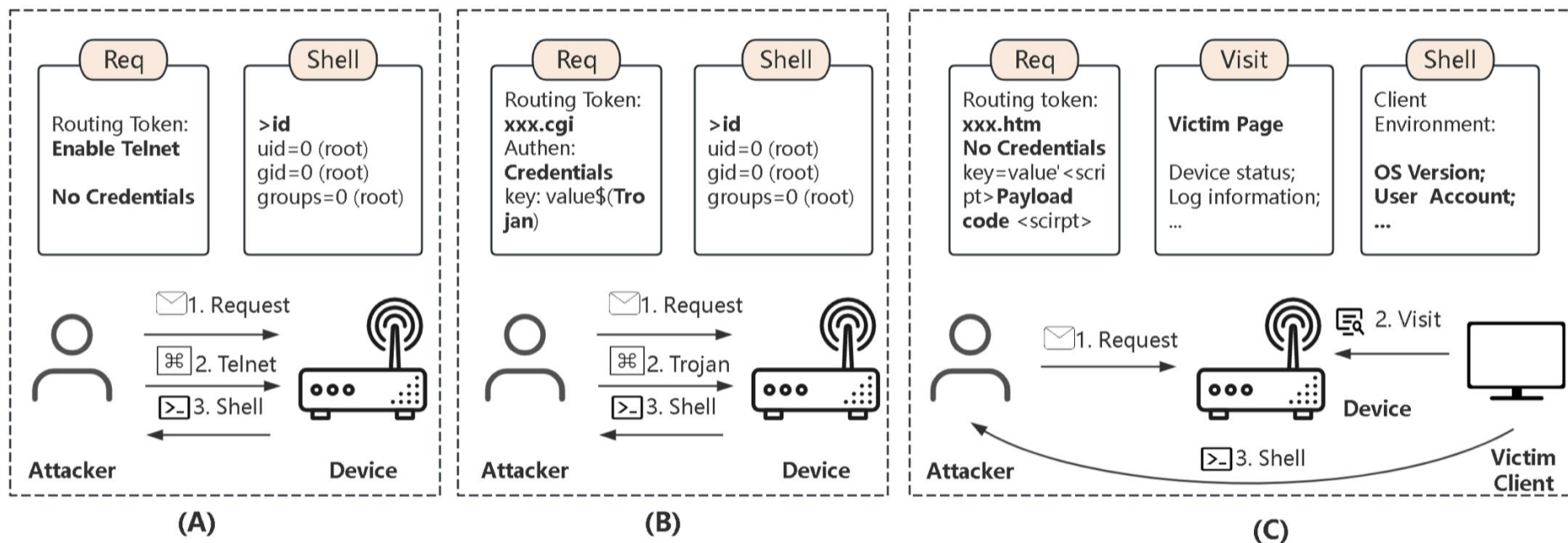
- 漏洞统计:
79 个隐藏接口导致发现了 29 个未知漏洞
- 漏洞类型:
 - 后门 (Backdoors), 跨站脚本 (XSS), 命令注入 (Command Injection), 信息泄露
- CVE 编号:
 - 已分配 7 个 CVE
 - 例如: Netgear R7000 (CVE-2024-1430), Cisco RV-042 (CVE-2024-20362), Linksys WRT54GL (CVE-2024-1404)

TABLE IV: Discovered vulnerabilities on hidden interfaces. #VUL=the number of vulnerabilities, #CVE=the number of assigned CVEs, #ID=detailed CVE ID EAGLEYE obtained.

Vendor	Model	#VUL	#CVE	#ID
Netgear	R7000	3	2	CVE-2024-1430, CVE-2024-1431
Cisco	RV-042	2	1	CVE-2024-20362
Motorola	CX2L	6	1	CVE-2024-25360
Linksys	WRT54GL	3	3	CVE-2024-1404, CVE-2024-1405, CVE-2024-1406
DLink	DNS-320L	5	0	-
	DIR-823G	6	0	-
Mercury	MNVR816	1	0	-
ZTE	C520	1	0	-
Zyxel	WSQ50	1	0	-
TPLink	Archer A7	1	0	-
Total	-	29	7	-

典型攻击案例

- 绕过认证的后门 (Backdoor): 攻击者无需凭证即可开启 Telnet 服务, 获取 Root Shell
- 权限提升 (Privilege Escalation): 认证用户通过隐藏函数注入任意系统命令
- 存储型 XSS: 恶意脚本注入到隐藏参数中, 攻击查看页面的任何用户



文章评价

➤设计优势

- 把隐藏接口从 URL 枚举提升到 routing token 推断，覆盖 Header/Body/认证后接口
- 问题建模明显优于 IoTScope

➤局限性分析

1. 需要公开请求样本和较好的反编译上下文
2. LLM 推断 token 仍可能受提示和设备差异影响，反馈闭环成本不低

➤商业化潜力

- 可用于厂商内部排查隐藏接口
- 作为通用商用工具会受设备交互样本、认证状态和 LLM 成本限制

PANGOLIN: Fuzzing Multilingual IoT Firmware with LLM-Driven Code Analysis

Zhipeng Jia¹, Xiaokang Yin^{1,}, Shuitao Gan⁴, Chao Zhang^{2,5,*}, Hangtian Liu³,
Jiangan Ji¹, Enzhou Song¹, Ruijie Cai¹, Jinglei Tan³, Shengli Liu¹*

¹Information Engineering University

²Institute for Network Sciences and Cyberspace, Tsinghua University

³State Key Laboratory of Mathematical Engineering and Advanced Computing

⁴Laboratory for Advanced Computing and Intelligence Engineering

⁵JCSS, Tsinghua University (INSC) - Science City (Guangzhou) Digital Technology Group Co., Ltd.

USENIX Security 2026

内容摘要

➤ 关键动机

- 多语言 IoT 后端中，仅发现隐藏入口不足以触发深层逻辑，还必须恢复参数位置、类型、层级结构和约束

➤ 本文工作：由 LLM 驱动代码语义分析的多语言 IoT 固件 Fuzzing 框架

1. 使用 LLM 分析 API 分发机制、跨语言调用链和参数解析逻辑，恢复入口映射与层级参数规格
2. 结合真实设备响应进行校正，用语义感知 Fuzzing 触发深层隐藏接口漏洞

➤ 关键数据

- 在 12 台设备上发现 68 个未知漏洞，是 LABRADOR 的 2.96 倍
- 其中 45 个位于隐藏接口，而 EAGLEYE 只能发现 4 个

研究背景：多语言后端中的隐藏接口新形态

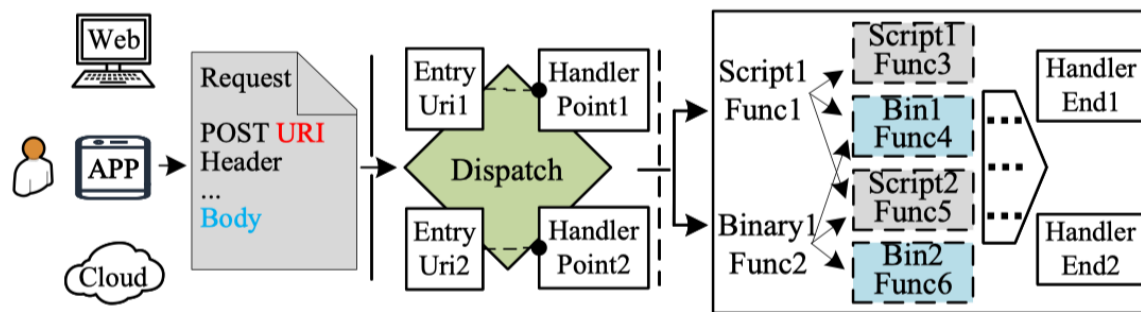
➤ EAGLEYE 已能从公开接口中学习路由 Token，推断隐藏接口

➤ 但现代 IoT 固件的 Web 后端正在变得更复杂：

1. C/C++ 二进制、Lua/Python 脚本、动态库插件共同处理请求
2. 参数可能以 JSON、字典、列表、结构体等层级形式跨语言传递
3. 隐藏接口不仅“URL 不可见”，其完整参数规格也不可见

➤ 结果：仅发现路由 Token 仍不够

→ 还必须恢复接口如何分发、参数如何解析、哪些组合能触发深层路径



核心洞察：从“发现 URL”到“恢复接口规格”

➤ **Insight 1:** 隐藏接口共享后端分发逻辑

- 公开接口与隐藏接口往往复用同一套 dispatch 函数、注册表或函数指针表

➤ **Insight 2:** 接口有效性依赖参数规格

- URL / Token 只是入口，真正决定能否触达深层逻辑的是参数名、位置、类型、层级和约束

➤ **Insight 3:** LLM 需要运行反馈校正

- LLM 能理解跨语言代码语义，但可能产生错误规格
- 设备响应中的“缺参数 / 格式错误 / 字段非法”信息可反向修正规格

把 Fuzzing 从“随机改请求”提升为“基于后端语义改请求”，并用设备返回的缺参、格式错误、非法字段等反馈反向修正参数规格，能触发更深层隐藏接口逻辑

PANGOLIN 系统架构总览

1. 代码表示预处理

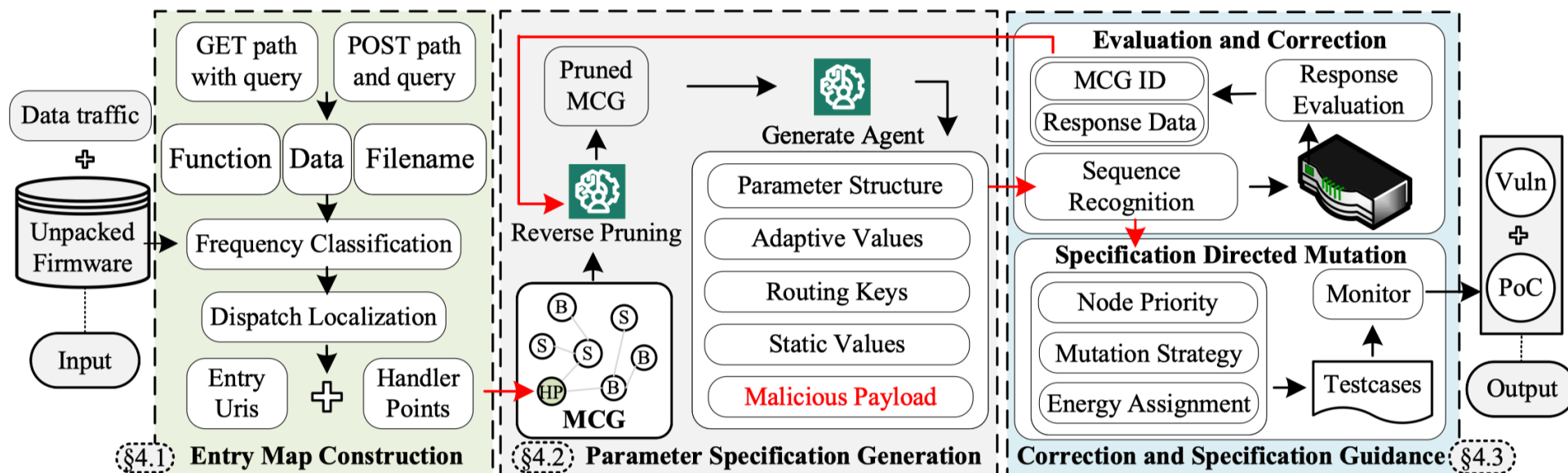
- 脚本按函数切片，补充全局变量与 import 别名
- 二进制反汇编为伪代码，并回填字符串、函数指针表等上下文

2. 入口映射恢复

- 定位 dispatch 函数
- 构建树结构的接口分发路径
- 由 LLM 恢复入口 URI 与后端处理函数的映射

3. 参数规格生成与校正

- 构建多语言调用图 MCG
- LLM 生成层级参数规格
- 结合设备响应反馈进行校正并指导 Fuzzing



核心模块1：入口映射恢复

➤ 设计目标：从多语言后端中恢复“入口 URI → 处理函数”的映射关系

➤ PANGOLIN 总结了三类常见分发机制：

1. 注册函数：启动阶段建立 URI-handler 映射
2. 字符串比较：通过 `strcmp` / 条件分支进行多级路由
3. 函数表分发：通过数据段函数指针表完成间接调用

➤ 分析流程：

1. 根据公开 URI 片段定位可能的 `dispatch` 函数
2. 调用关系组织成树结构
3. 让 LLM 沿路径逐层解释分发语义，恢复隐藏接口入口

核心模块2：多语言调用图与参数规格生成

设计目标：恢复跨脚本、跨二进制、跨动态库的完整参数解析逻辑

► 多语言调用图 MCG:

- 将脚本函数、二进制函数、动态库函数及跨语言调用统一建模
- 对完整调用图进行剪枝，仅保留与目标接口相关的路径

► 参数规格内容:

- 参数位置：URI / Header / Body
- 数据结构：JSON 字典、列表、嵌套对象
- 语义约束：固定取值、数值范围、字符串格式、路径格式
- 组合关系：哪些参数必须共同出现

→ 比传统字典式种子更适合触发深层隐藏接口逻辑

Prompt Template

Task: Based on parameter input methods, identify two types of custom functions or dirs: (1) parameter extraction and execution; (2) sanitizers, follow these principles:

- 1) If found all, and the context indicates that the function terminates, output NULL.
- 2) If incomplete discovery, and the code suggests that other callees may exist, provide up to three candidates.
- 3) If context suggests that no candidate callees, output NULL.

Output Format: Return the results in JSON format as `{'result': {'functions': [], 'dirs': []}}`.

Prompt Template

Task: Generate parameter specifications based on pruned MCG content corresponding to the entry URI, , follow these principles:

- 1) Confirm the request method (GET, POST, ...).
- 2) Trace the parameter parsing process to reconstruct the complete parameter structure.
- 3) Identify parameters with unique values, Label as NoMuta.
- 4) Generate adaptive values based on type checking and format validation. Label as NoMuta.
- 5) Identify parameter values corresponding to different sinks in a branch as routing values, and combine them with other parameter values to generate multiple result sets. Label as NoMuta.
- 6) Strong Sanitize few shot: Filter ' in ", filter all special characters (;,\$ & ""'), parameter length limit, command whitelist, parameter type and format regex matching limit.
- 7) If a vulnerability exists, identify its type and mark it (CI, BOF, DoS, XSS, AFR, AFU) and generate payloads based on the default command. Label as Muta.

Output Format: Return the results in JSON format as {"result":{"normal_requests":[{"method":"","parameters":""}, {}], "label":[{"method":"","paramlabel":""}, {}]}}, 12

核心模块3：响应反馈驱动的语义 Fuzzing

➤ 设计目标：用真实设备反馈校正 LLM 生成的接口规格

➤ 响应驱动校正：

- 若响应提示缺少参数、格式错误、字段非法，则反向修正参数规格
- 若请求推进到更深处理逻辑，则提升该规格优先级

➤ 语义感知变异：

- 命令注入风险参数：插入 shell 元字符
- 路径参数：尝试目录穿越 payload
- 枚举参数：优先测试合法值附近的异常组合

→ Fuzzing 不再均匀随机变异，而是由后端代码语义和运行响应共同驱动

实验评估与关键数据

- 测试对象： 12 个来自 8 个主流厂商的多语言 IoT 设备
- 接口发现能力：
- 共发现 2,856 个后端接口，其中 1,793 个未暴露在前端中
- 漏洞发现结果：
- 发现 68 个此前未知漏洞，45 个漏洞位于隐藏接口中
 - 漏洞类型包括命令注入、XSS、信息泄露、拒绝服务、任意文件操作
 - 31 个漏洞已分配漏洞编号
- 对比结果：
- 对于隐藏接口，EAGLEYE只能检测出4个漏洞
 - PANGOLIN-H可以检测出45个漏洞

Table 2: Distribution of detected vulnerabilities (RQ1).

Brand	Model	Type	Vuln	CVE/CNVD
HIKSEMI	MAGE20PRO	CI	5	2
		DoS	1	0
		leak	1	0
TP-LINK	TL-IPC42A-4	CI	2	0
		Arbitrary File Reads	1	0
Cisco	IOS-XE(C)	Arbitrary File upload	1	0
		IOS-XE(Isr)	2	1
Linksys	E5600	CI	15	11
		XSS	4	1
Netgear	EX8000	CI	3	3
		EX6250	3	3
xiaomi	R3A	CI	3	1
		AR300M16	6	6
GL-iNet	MT300N-V2	CI	6	0
		XE300	6	0
Ruijie	EG105GW	CI	9	3
Total			68	31

Table 4: Comparison between PANGOLIN-H and EAGL-EYE (RQ2).

Brand	Model	EAGLEYE				PANGOLIN-H			
		HINT	B-A	A-A	vuln	HINT	B-A	A-A	vuln
HIKSEMI	MAGE20PRO	408	14	394	2	408	14	394	5
TP-LINK	TL-IPC42A-4	0	0	0	0	17	0	17	2
		119	0	119	0	119	0	119	1
Cisco	IOS-XE(C)	56	0	56	0	56	0	56	3
		0	0	0	0	25	0	25	10
Linksys	E5600	11	0	11	1	11	0	11	1
		10	0	10	1	10	0	10	1
Netgear	EX6250	271	0	271	0	271	0	271	3
		0	0	0	0	281	0	281	4
xiaomi	R3A	0	0	0	0	272	0	272	4
		0	0	0	0	289	0	289	4
GL-iNet	MT300N-V2	0	0	0	0	34	3	31	7
		0	0	0	0				
Ruijie	EG105GW	0	0	0	0				
Total		875	14	861	4	1,793	17	1,776	45

HINT=the number of hidden interfaces, B-A=the number of hidden interfaces bypassing authentication, A-A=the number of hidden interfaces after authentication. 131

文章评价

➤设计优势

- 不仅找隐藏入口，还恢复参数规格和跨语言调用链
- 能触发更深层隐藏接口逻辑

➤局限性分析

1. 依赖 LLM、真实设备响应和多语言代码恢复
2. 对无错误反馈、强状态机或响应极少的设备会失效

➤商业化潜力

- 受限于 LLM 成本、运营难度等原因，仅适合高端设备安全评估服务

维度三总结：隐藏接口发现

IOTScope 【WWW 2022】

依靠静态特征提取与盲测，通过海量探测请求与对比算法筛选隐藏接口



采用海量请求的暴力推断模式，性能更低

EAGLEYE 【NDSS 2025】

通过分析公开接口，利用LLM分析上下文路由模式，智能推断出隐藏接口



利用已知公开接口和LLM的语义理解能力推断，更智能且轻量级

PANGOLIN 【USENIX SEC 2026】

面向多语言 IoT 后端，恢复入口映射、跨语言调用链和层级参数规格



将隐藏接口发现从“找到入口”推进到“恢复可触发的接口规格”

维度四：漏洞验证、利用生成与评测基准

- **核心痛点：**静态污点分析完毕后会大量告警，黑盒Fuzzing又缺乏内部覆盖率指导
→ 如何高效验证漏洞并生成PoC（概念验证代码）？
- **技术路线（从静态告警到动态反馈）**
 1. **数据可达与漏洞确认：**针对传统静态分析仅关注数据可达性而忽略安全校验（导致海量误报）的缺陷，创新性提出“约束语义不一致性”分析视角，通过交叉比对前后端防御逻辑的缺失与冲突来确认真实漏洞【NÜWA, USENIX Security 2025】
 2. **动态反馈闭环：**在黑盒模糊测试中，利用Web响应报文中的字符串来推断后端的执行轨迹与到达Sink的距离，实现导向性Fuzzing【LABRADOR, IEEE S&P 2024】
 3. **动静结合与Agent自动利用：**结合Fuzzing探测真实可达的输入点，然后由LLM大模型Agent执行深层上下文语义分析并自动组装漏洞PoC【FirmAgent, NDSS 2026】
 4. **告警真实性验证：**将静态污点报告/安全补丁转化为入口点、参数和路径约束驱动的真实设备黑盒验证任务【Bond, USENIX Security 2026】【P2PGen, ACM CCS 2026】
 5. **Web Fuzzing技术基准建设：**构建可验证分层 benchmark，补齐 Web 漏洞研究基础设施【PHPBench, ACM CCS 2026】

From Constraints to Cracks: Constraint Semantic Inconsistencies as Vulnerability Beacons for Embedded Systems

Jiaxu Zhao^{1,2,3,4}, Yuekang Li⁵, Yanyan Zou^{1,2,3,4✉}, Yang Xiao^{1,2,3,4}, Naijia Jiang^{1,2,3,4}
Yeting Li^{1,2,3,4}, Nanyu Zhong^{1,2,3,4}, Bingwei Peng^{1,2,3,4}, Kunpeng Jian^{1,2,3,4}, Wei Huo^{1,2,3,4✉}

¹Institute of Information Engineering, Chinese Academy of Sciences, China

²School of Cyber Security, University of Chinese Academy of Sciences, China

³Key Laboratory of Network Assessment Technology, Chinese Academy of Sciences, China

⁴Beijing Key Laboratory of Network Security and Protection Technology, China

⁵University of New South Wales, Australia

USENIX Security 2025

内容摘要

➤ 关键动机

- 传统 Source-Sink 可达性分析忽略前端校验、后端校验与 Sink 安全需求之间语义差异，导致高误报和漏报

➤ 本文工作：基于前后端约束语义不一致性的嵌入式 Web 服务漏洞检测方法

1. 前端侧：从 HTML maxlength/select、JavaScript 条件语句等位置提取显式约束，区分正向限制与负向限制
2. 后端侧：通过函数摘要、双向切片和过程间 CFG，提取分支条件、默认值、截断逻辑等后端显式约束，并根据 strcpy/sprintf/system/open 等 Sink 上下文推断期望约束
3. 比较这些约束是否互相覆盖，进而检测漏洞

➤ 关键数据

- 在 13 个厂商的基准上确认 51 个真实告警，覆盖 28 个已知漏洞，并额外挖掘 12 个关联漏洞
- 相比 SaTC、LARA、Mango、OctopusTaint 分别多发现 18、6、17、19 个漏洞

研究背景：传统数据流追踪的“盲区”

1. 数据可达 $\neq$ 漏洞可利用

- 传统的静态污点分析（如 SaTC、KARONTE）仅关注用户输入是否能到达敏感函数（Sink）
- 然而，现代固件在数据传播路径上通常部署了各种安全检查（如前端长度限制、后端非法字符过滤）

2. 单一维度分析的失效

- 仅分析后端：若忽略前端的输入约束，后端程序看似缺乏检查的路径实际上可能是安全的，导致误报
- 仅依赖数据流：无法感知目标敏感函数（如 `strcpy`）真正需要满足的“安全底线（如缓冲区容量）”
- **研究目标：**系统亟需一种能够全面提取并理解前后端“安全约束语义”，并通过逻辑比对直接暴露“防御缺失”的高级静态分析框架

核心洞察：漏洞的本质是“约束语义的不一致”

➤ NÜWA 的核心理论建立在对系统输入校验逻辑的结构化抽象之上，将系统中的约束定义为三类

1. **前端显式约束 (Front-end Explicit Constraints):** 前端 Web 页面 (HTML/JS) 中直接限制用户输入的规则 (如 HTML 的 `maxlength` 属性、JS 中的输入合法性判断)
2. **后端显式约束 (Back-end Explicit Constraints):** 后端二进制程序中，在数据到达 Sink 之前，开发者显式编写的安全检查代码 (如对输入长度的校验或非法字符的替换)
3. **后端期望约束 (Back-end Desired Constraints):** 敏感函数 (Sink) 为了不触发内存破坏或命令注入，而必须满足的内在安全要求 (例如 `strcpy` 的目标缓冲区大小)

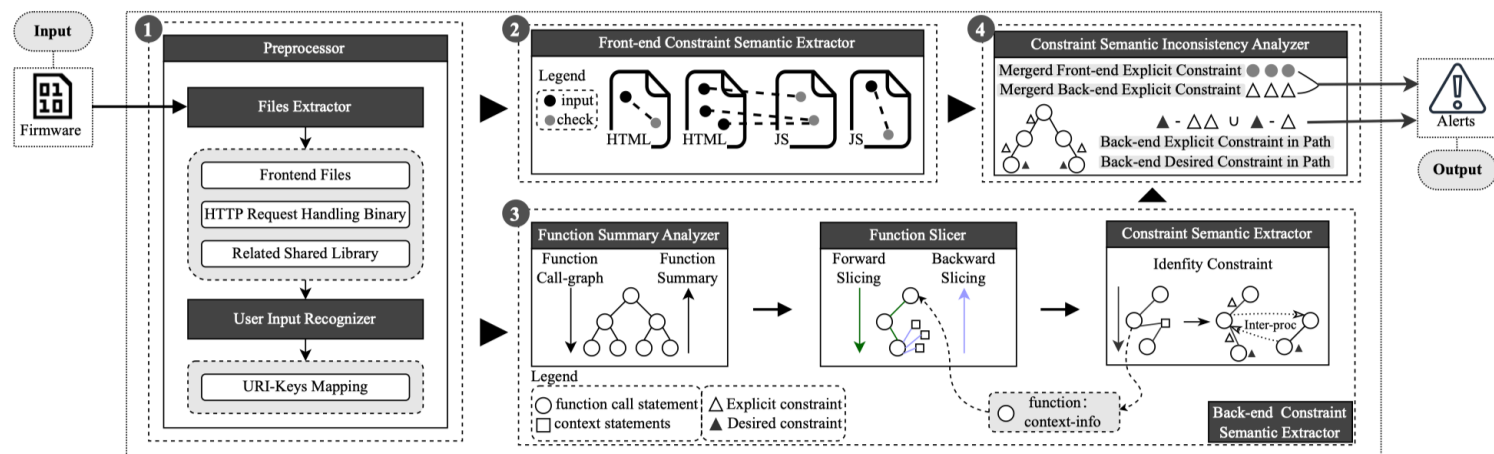
➤ 核心洞察：如果前端约束与后端显式约束存在冲突 (不统一)，或者后端的显式约束未能满足 Sink 的期望约束，这种“语义不一致性”即是漏洞存在的信号

将安全约束统一成六类标准化表示，再做集合差分

既能发现“后端缺少 Sink 所需检查”，也能发现“前端有检查但后端没有等价检查”的绕过型漏洞

NÜWA 系统架构总览

- NÜWA 将固件作为输入，输出约束语义不一致性告警。其 workflow 包含四个关键阶段：
 1. **固件预处理 (Preprocessing)**: 提取固件文件系统，分离前端文件与后端请求处理二进制文件，并利用 LARA 的技术提取用户输入的 URI-Key 映射关系
 2. **前端约束语义提取 (Front-end Constraint Semantic Extraction)**: 定位前端中的用户输入，从 HTML 属性和 JS 条件语句中提取与输入相关的显式约束语义
 3. **后端约束语义提取 (Back-end Constraint Semantic Extraction)**: 利用函数摘要分析和程序切片 (Slicing) 技术精简控制流，在过程间控制流图 (ICFG) 中提取后端的显式约束与期望约束
 4. **不一致性静态分析 (Constraint Semantic Inconsistency Analysis)**: 对比上述三类约束语义，识别两类不一致性并生成高价值漏洞告警



核心模块1： 前端约束语义提取

- 设计目的： 结构化解析前端表单与脚本中的安全保护

- 前端的显式约束主要通过 HTML 属性和 JavaScript 条件语句实现：

1. HTML 约束解析：

- 属性约束： 将 `<input>` 元素中的原生校验属性（如 `maxlength`）直接转换为标准化约束语义（如 `<len, >`）
- 固定值约束： 解析 `<select>` 下拉菜单，将其转化为预定义选项的严格限制（如 `<fix, ['wan', 'lan']>`）

2. JavaScript 条件语句解析：

- 针对包含多重逻辑（`&&`, `||`）的条件语句，NÜWA 逐层拆解
- 处理内置函数调用（如 `isalnum`）、算术运算与比较运算
- 依据上下文控制流（如该判断是否阻止了后续的 HTTP 请求发送），精确判定提取出的语义是“正面约束（Positive）”还是“负面约束（Negative）”

核心模块2： 后端分析基础 - 函数摘要与切片（上）

- **设计目标：规避路径爆炸与聚焦输入上下文**

- 由于后端程序逻辑庞杂，直接进行跨过程的数据流分析将面临严重的状态爆炸

- NÜWA 采用两项技术进行降维：

1. **基于摘要的过程间分析 (Summary-based Inter-procedural Analysis):**

- 摒弃了极易导致超时的克隆级（Cloning-based）跨过程分析，转而构建自底向上的函数摘要
 - 将函数行为抽象为参数间的依赖关系（例如 `strcat(dest, src)` 被摘要为 `<0:, 1:>`，表示 `dest` 受其自身及 `src` 共同影响），大幅提升分析速度

2. **双向代码切片 (Forward and Backward Slicing):**

- 前向切片：从输入处理逻辑出发，剔除与输入数据无关的代码
 - 后向切片：从相关变量向上追溯，保留影响约束判断的上下文（如缓冲区分配大小、相关赋值操作）

核心模块2： 后端约束语义提取（下）

- **设计目标：在 ICFG 中锚定显式校验与期望安全边界**

- 在切片精简后的过程间控制流图（ICFG）上，NÜWA 分别提取两类后端约束：

1. 显式约束提取 (Explicit Constraints):

- 输入处理点：分析获取用户输入的定制函数，如提取默认值设置（转化为 `<not_null, True>`）或强制长度截断
- 条件分支：分析代码执行路径中的条件跳转（如遇到特定字符则执行 `goto`、`break` 或 `abort`），将其转化为负面约束语义

2. 期望约束提取 (Desired Constraints):

- 面向内存的安全期望：针对内存相关 Sink（如 `strcpy`, `sprintf`），根据切片上下文提取目标缓冲区的容量大小，将其定义为输入数据必须满足的“长度期望约束”
- 面向命令注入的安全期望：提取针对 `system` 等执行函数的字符过滤期望

核心模块3：约束语义不一致性分析

- 设计目标：通过异构语义集合的逻辑计算进行安全保护不一致分析
- NÜWA 设计了六种标准化的内部表示来统一多源异构的语义，并执行严格的集合运算

1. 不一致性 1：后端显式约束 vs. 后端期望约束

- $I(P) = \text{Diff}(\text{Merge}(\text{EC}(P)), \text{DC}(P))$
- 含义：在目标路径 P 上，将所有后端显式检查（EC）进行合并后，如果仍无法覆盖敏感函数所要求的安全底线（DC），即产生非空差异集合 I(P)，表明存在直接可利用的内存破坏或注入漏洞

2. 不一致性 2：前端显式约束 vs. 后端显式约束

- $I = \text{Diff}(\text{Merge}(\text{EC}), \text{Merge}(\text{FC}))$
- 含义：比较合并后的前端约束（FC）与后端约束（EC），如果存在前端进行了限制而后端缺失校验的情况，意味着攻击者只需绕过前端（如直接发送构造的 HTTP 报文）即可攻击，构成越权或漏洞利用

实验评估与关键结果

- NÜWA 收集了来自 13 个厂商的 31 个具有明确触发条件的已知漏洞构建基准数据集，并与 SOTA 工具（SaTC, LARA, Mango, OctopusTaint）进行了严格的同源对比

1. 漏洞发现能力

- NÜWA 成功确认了 51 个真实告警，不仅覆盖了数据集中的 28 个已知漏洞，还额外挖掘出了 12 个关联漏洞
- 相比之下，性能最好的基线工具 LARA 仅检测出 29 个真实告警
- NÜWA 相比 SaTC、LARA、Mango 和 OctopusTaint 分别多发现了 18、6、17 和 19 个漏洞

2. 漏洞分析精度

- NÜWA 最终的漏洞定位准确率高达 76%，且其前端显式约束、后端显式约束及期望约束的提取准确率分别达到了 95%、86% 和 85%

文章评价

➤设计优势

- 从约束一致性角度解释漏洞
- 比单纯 Source-Sink 可达性更接近真实安全缺陷，也更容易得出修复建议

➤局限性分析

1. 约束提取不完整时会直接影响分析有效性
2. 复杂权限、业务状态、跨请求逻辑和非建模 Sink 仍难覆盖

➤商业化潜力

- 适合用于固件发布前的安全检查，但由于静态特性仍需要 PoC 验证闭环

LABRADOR: Response Guided Directed Fuzzing for Black-box IoT Devices

Hangtian Liu^{1,2,4}, Shuitao Gan^{2,4}✉, Chao Zhang^{2,3}✉, Zicong Gao^{1,2,4},
Hongqi Zhang^{1,5}, Xiangzhi Wang⁶, Guangming Gao⁴

¹*Information Engineering University,*

²*Tsinghua University, ³Zhongguancun Laboratory,*

⁴*Laboratory for Advanced Computing and Intelligence Engineering,*

⁵*Henan Key Laboratory of Information Security,*

⁶*University of Electronic Science and Technology of China*

IEEE S&P 2024

内容摘要

➤ 关键动机

- 真实 IoT 设备通常无法插桩
- 黑盒 Fuzzing 缺少覆盖率和目标距离反馈，难以高效触发深层 Sink

➤ 本文工作：基于响应反馈的黑盒 IoT 设备定向 Fuzzing 框架

1. 利用响应字符串反推后端显式代码块执行轨迹
2. 将请求/响应双向距离用于定向变异和能量调度

➤ 关键数据

- 在 14 款真实或模拟企业级 IoT 设备上发现 79 个 0-day，其中 61 个获得 CVE 编号
- 是静态工具 SaTC 的 8.57 倍

研究背景：黑盒测试中的反馈缺失瓶颈

1. 灰盒测试的困境

- 现代灰盒模糊测试依赖插桩或硬件追踪来收集代码覆盖率，但这在无法获取底层执行权限的真实物理物联网设备上是不可行的

2. 黑盒测试的盲目性

- 现有的物联网黑盒模糊测试工具（如 SNIPUZZ）仅能利用响应差异推断消息片段，无法感知后端深层代码的执行状态与目标距离

➤ 核心观察

- 物联网设备的 Web 接口主要采用人机交互模式，请求与响应中不可避免地包含大量语义字符串
- 由于前端、后端二进制程序和网络响应之间共享大量特征字符串，可以通过分析设备返回的响应文本，逆推后端哪些代码块被成功触发

核心洞察

➤ **Insight 1:** 真实 IoT 设备难以插桩，但 HTTP 响应可以间接泄露后端执行状态

- 灰盒 Fuzzing 依赖覆盖率反馈，但真实物理设备通常无法修改固件或插入探针
- LABRADOR 观察到，后端程序执行到特定代码块时，常会把固定字符串写入 HTTP 响应，因此响应文本可以作为近似执行反馈

➤ **Insight 2:** 响应字符串可以充当黑盒环境中的“伪覆盖率”

- LABRADOR 静态提取后端二进制中的响应字符串及其引用位置，运行时将设备返回内容与这些字符串匹配，从而推断哪些显式代码块被执行，绕开真实设备无法获得覆盖率的问题

➤ **Insight 3:** 定向 Fuzzing 需要同时考虑输入与输出两侧的距离

- 仅看请求是否接近目标路径不够，因为黑盒设备缺少真实执行轨迹
- LABRADOR 同时从请求侧估计 seed 对目标 Sink 的潜在推进能力，从响应侧估计实际执行是否更接近目标，形成 IO Oriented Distance

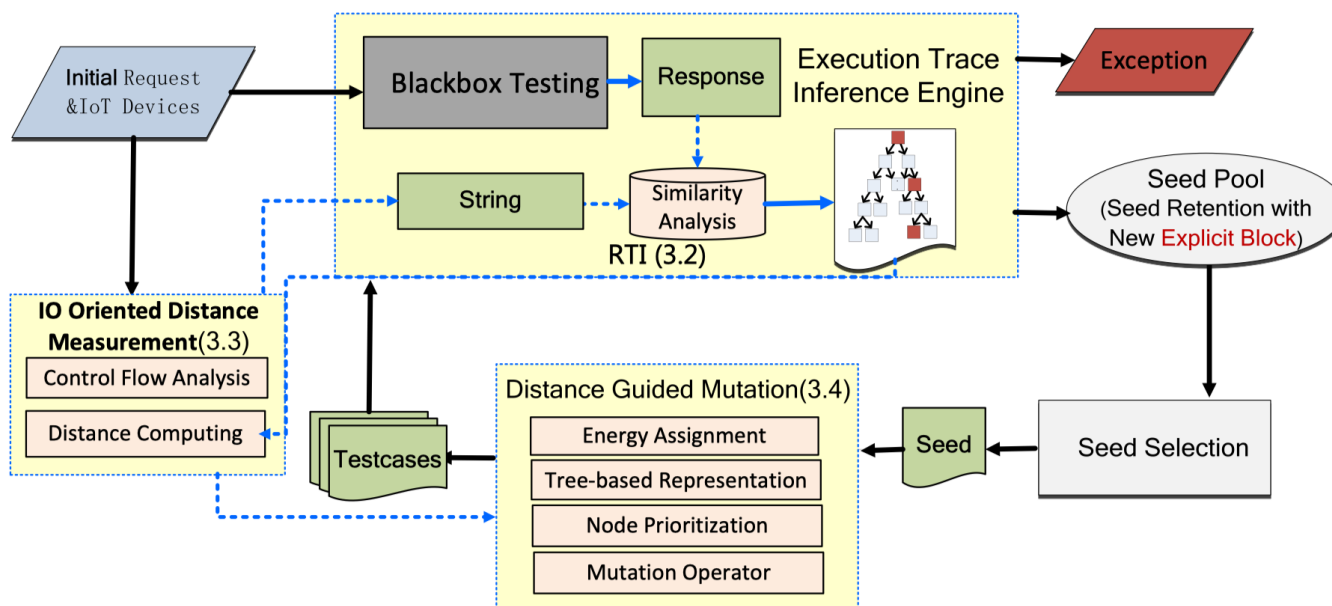
➤ **Insight 4:** IoT Web 请求需要结构化变异，不能像普通字节流一样随机打乱

- HTTP 请求具有 URI、Header、Cookie、Body、参数树等结构，盲目字节变异容易破坏协议格式
- LABRADOR 将请求解析成树状结构，并根据距离、节点类型、历史收益和树深度分配变异能量

用“响应字符串”在黑盒环境中近似恢复灰盒反馈，绕开无法插桩的问题，同时保持对真实设备的直接测试能力

LABRADOR 系统架构总览

- LABRADOR 的整体架构将“黑盒环境”转化为“灰盒模型”，其 workflow 包含三个核心组件：
 1. **基于响应的执行轨迹推断 (RTI, Response-based Execution Trace Inference)**: 通过静态分析提取后端二进制中的“显式字符串”，并将其与动态响应报文进行相似度比对，以推断执行轨迹
 2. **面向 IO 的距离度量 (IODM, IO Oriented Distance Measurement)**: 基于简化的控制流图 (SCFG)，分别从请求 (Input) 和响应 (Output) 两个视角计算当前测试用例到达危险函数 (Sink) 的距离
 3. **距离引导的变异策略 (Distance Guided Mutation)**: 将请求转换为树状语法结构，利用推断出的覆盖率和距离反馈，动态调度变异能量并分配节点的变异优先级



核心模块1：基于响应的执行轨迹推断

➤ 设计目标：从响应文本逆推深层代码覆盖率

➤ 在无法插桩的情况下，LABRADOR 创新性地将字符串作为追踪锚点：

1. **显式字符串与显式块 (Explicit Strings & Blocks):** 静态提取后端二进制中能够输出到响应的字符串（即显式字符串），并将引用这些字符串的代码块标记为“显式块”
2. **双模字符串相似度分析 (Similarity Measurement)**
 - 针对长度相近的字符串，使用莱文斯坦距离 (Levenshtein distance) 捕捉微小变异
 - 针对长度差异较大的拼接字符串（如后端将前缀与变量拼接），使用最长公共子串 (LCS) 进行包含关系匹配
3. **轨迹重构：**若响应中的文本与二进制中的潜在显式字符串相似度超过阈值，则认定该字符串对应的“显式块”被成功执行，从而在黑盒状态下还原出近似的代码覆盖率

核心模块2：控制流分析与目标定位（上）

➤设计目标：精简过程间控制流图以支撑距离计算

➤为了在缺乏源码的情况下测量距离，LABRADOR 对后端控制流进行了静态建模：

1. **属性标记：**在完整的跨进程控制流图 (ICFG) 中，将基本块标记为四类：与请求相关的显式块 (EP)、与响应相关的显式块 (OP)、漏洞目标块 (SP/Sink) 以及普通块
2. **目标函数定位 (Sink Identification)：**建立包含常规危险函数（如 `system`, `strcpy`）符号的数据库，并开放专家知识接口以定义特定漏洞模式，将其统一标记为 SP 节点
3. **图精简与可达性分析：**执行从 SP 节点到其他节点的可达性分析，剔除所有无法到达 Sink 的无关节点，生成规模仅为原图约 10% 的简化控制流图 (SCFG)，极大降低了后续动态距离计算的开销

核心模块2：面向 IO 的距离度量 IODM（下）

➤ 设计目标：基于字符串增益的请求与响应双向评估

➤ 给定一个测试用例（Seed），LABRADOR 从输入和输出双重视角量化其与漏洞点的距离：

1. **字符串潜在增益 (String Gain)**: 由于请求由多个字符串 Token 组成，不同 Token 变异后触发漏洞的概率不同

→ 显式字符串的增益定义为“从引用该字符串的显式块出发，能到达的所有 Sink 节点权重的累加”

2. **静态请求距离 (Request Distance)**: 在执行前，计算请求中所有字符串 Token 增益倒数的调和平均值，以此评估该 Seed 的先天变异潜力

3. **动态响应距离 (Response Distance)**: 在执行后，提取响应中实际出现的显式字符串计算增益，生成动态距离

→ 该距离反映了实际执行轨迹与 Sink 的逼近程度，用于实时修正变异策略

核心模块3：树状表示与能量调度（上）

➤设计目标：感知网络语义的结构化变异控制

➤为避免破坏 HTTP 协议的严格格式，LABRADOR 将请求转化为结构化模型：

1. **树状表示法 (Tree-based Representation)**: 将 HTTP 请求解析为包含头部 (Header)、参数 (Parameter) 和属性 (Attribute) 的抽象语法树
 - 参数节点是触发深层逻辑的关键，头部和属性节点则采用低频变异以保证请求能够被 Web 服务器合法解析
2. **动态能量分配 (Energy Assignment)**: 基于 IODM 计算出的距离反馈，结合种子在近期变异中产生更优测试用例的历史表现 (Historical Performance) 计算其价值
 - 距离 Sink 越近且历史表现越好的种子，将被分配更多的变异能量（即生成更多的派生测试用例）

核心模块3：节点优先级与定向变异（下）

➤ 设计目标：精准分析高价值语义节点的变异引擎

➤ 在分配能量后，LABRADOR 需决定对树模型中的哪些节点实施何种变异：

1. 节点选择概率 (Mutation Probability)：节点的选中概率由四个维度联合决定：

- 节点增益 (Gain)：到达 Sink 的理论概率越高，优先级越高
- 树深度 (Depth)：越深层的节点通常代表后端更复杂的解析路径，优先级越高
- 所在区域 (Domain)：不同区域赋予不同基础权重（如 Parameter 区域权重极高）
- 动态距离修正：结合最新响应反馈，向上调优那些使得执行轨迹更接近 Sink 的节点概率

2. 双层变异算子 (Mutation Operators)

- 类型算子：感知目标 Sink 类型（如命令注入需插入 ; 或 |）与数据类型（如数字极值替换）
- 结构算子：对树的子节点进行复制、添加、删除或替换，实现大规模路径探索

实验评估与关键数据

➤ LABRADOR 在 14 款真实的物理或模拟企业级物联网设备上进行了评估

1. 漏洞发现数量

- LABRADOR 共发现了 79 个未知的 0-day 漏洞（涵盖命令注入、内存破坏与 XSS）
- 其中 61 个已获 CVE 编号

2. 对比动态分析工具

- 与缺乏距离反馈的 SNIPUZZ 和 BOOFUZZ 相比，LABRADOR 发现的漏洞数量超过其 44倍

3. 对比静态分析工具

- 与最先进的固件静态污点分析工具 SaTC 相比，LABRADOR 发现的漏洞数量是其 8.57倍
- 主要原因是静态分析缺乏隐式控制流信息，且容易因字符串匹配规则死板而产生大量漏报

文章评价

➤设计优势

- 用响应字符串近似恢复黑盒执行轨迹，绕开真实设备无法插桩的问题

➤局限性分析

1. 高度依赖响应中有可匹配字符串
2. 静默接口、二进制协议、统一错误页或响应模板化严重时反馈质量会下降

➤商业化潜力

- 适合用于设备黑盒 Fuzzing，但需要稳定重启/恢复机制和安全隔离，运营成本不低

FirmAgent: Leveraging Fuzzing to Assist LLM Agents with IoT Firmware Vulnerability Discovery

Jiangan Ji^{*†}, Chao Zhang^{†‡✉}, Shuitao Gan[§], Lin Jian^{*}, Hangtian Liu^{*},
Tieming Liu^{*✉}, Lei Zheng[†], Zhipeng Jia^{*†}

^{*}Information Engineering University, [†]Institute for Network Sciences and Cyberspace, Tsinghua University,

[‡]JCSS, Tsinghua University (INSC) - Science City (Guangzhou) Digital Technology Group Co., Ltd.

[§]Laboratory for Advanced Computing and Intelligence Engineering

NDSS 2026

内容摘要

➤ 关键动机

- 静态分析误报高且缺少 PoC，Fuzzing 又难覆盖深层 Sink
- 单独依赖 LLM 也容易分析不可达路径

➤ 本文工作：结合 Fuzzing 能力与 LLM Agent 语义推理的固件漏洞发现与 PoC 生成框架

- 提出一个从动态信息收集到 LLM 语义推理再到 PoC 生成的闭环框架
 1. 先用 Fuzzing 捕获真实可达输入源和路径信息
 2. 再由 LLM Agent 修复反编译代码、追踪污点并生成 PoC

➤ 关键数据

- 在 14 款固件上确认 182 个真实漏洞，精度 91%，远高于 HermeScan 的 33%
- 漏洞数量为 HermeScan 的 2.6 倍

研究背景：静态与动态分析及 PoC 生成的困境

1. 静态分析的误报与验证负担

- 现有静态工具（如 SaTC）能够发现潜在的漏洞位置，但缺乏自动验证其真实性的能力
- 这要求安全分析师投入大量人工成本进行漏洞分流与确认

2. 动态模糊测试的可达性盲区

- 由于物联网固件中大量条件分支与特定输入值紧密绑定，模糊测试往往无法覆盖大部分代码库
- 经验证，模糊测试平均只能到达约 25% 的汇聚点（Sink），这意味着仅依赖 Fuzzing 会遗漏大量深层漏洞
- 研究目标：自动合成有效 PoC 的闭环验证方案
 - 结合 Fuzzing 收集的真实可达输入源（消除误报源头）
 - 利用大模型进行复杂上下文数据流追踪（消除漏报）

核心洞察

➤ **Insight 1:** Fuzzing 的价值不只是发现漏洞，还可以为 LLM 提供真实可达性证据

- Fuzzing 虽然难以独立覆盖所有深层 Sink，但它能捕获真实请求、执行路径、可达输入源和运行时约束，这些动态证据可以约束 LLM 的分析范围，避免 LLM 在不可达路径上“空推理”

➤ **Insight 2:** LLM 适合补足二进制静态分析中缺失的语义理解

- 传统污点分析容易受反编译错误、函数参数缺失、间接调用和 Sanitizer 判断影响
- FirmAgent 利用 LLM 修复反编译伪代码、理解函数语义、识别净化逻辑，并辅助完成从 Csource 到 Sink 的精细污点传播

➤ **Insight 3:** 漏洞发现的最终目标应是可验证 PoC，而不是静态告警

- 不停留在“发现潜在 Source-Sink 路径”，而是把 Fuzzing 得到的合法请求骨架与 LLM 提取的语义约束结合起来，自动组装可触发漏洞的 PoC，从而形成“发现-验证-利用样例生成”的闭环

不单纯依靠“LLM 审计代码”，而是用 Fuzzing 把 LLM 的推理空间锚定在真实执行路径上

最后再用合法请求骨架和语义约束自动组装 PoC，形成漏洞验证闭环

FirmAgent 系统架构总览

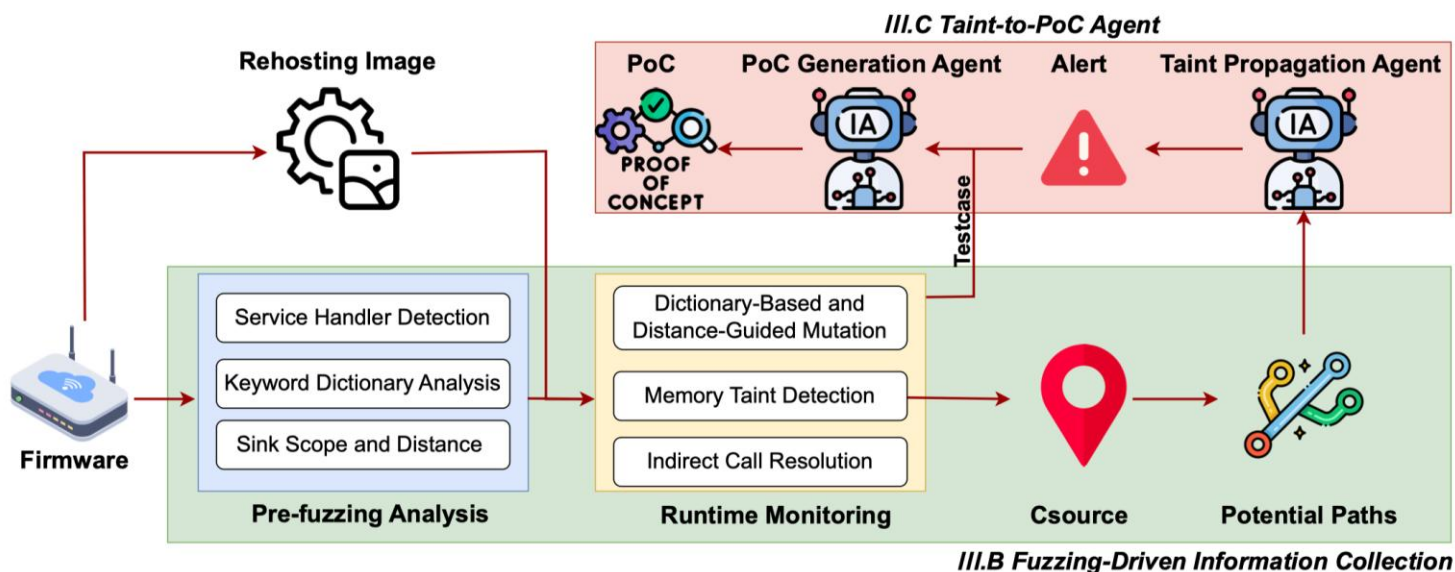
- FirmAgent 的架构设计旨在将黑盒/灰盒环境转化为富语义的 LLM 推理模型，包含两大核心阶段

1. 模糊测试驱动的信息收集 (Fuzzing-Driven Information Collection):

- 结合轻量级静态预分析与运行时插桩，提取服务处理函数并监控内存污点状态
- 动态识别真实可达的外部输入点 (Csource)，并补全控制流图（包括间接调用）

2. 污点追踪至 PoC 生成代理 (Taint-to-PoC Agent):

- 基于收集的 Csource 和潜在路径，部署 Taint Propagation Agent 执行细粒度的语义级数据流分析
- 当发现漏洞时，调用 PoC Generation Agent 结合 Fuzzing 测试用例与静态约束，自动组装 PoC



核心模块1：模糊测试驱动的预分析（上）

➤ 设计目标：提取语义特征以指导高覆盖率模糊测试

➤ 为克服模糊测试在探索深层服务处理函数时的困难，FirmAgent 首先执行轻量级静态预分析

1. 服务处理函数检测 (Service Handler Detection)

- 提取 Web 服务注册的 URI 作为变异字典，帮助 Fuzzer 构造精确的输入

2. 关键字字典分析 (Keyword Dictionary Analysis)

- 识别可接收外部数据的参数关键字，并将其作为特定变异的输入依据

3. 汇聚点范围与距离计算 (Sink Scope and Distance)

- 通过模式匹配识别常见的危险 API（如 system, strcpy）作为目标 Sink
- 使用 Dijkstra 算法计算每个基本块到 Sink 的最短距离，生成距离映射

→ 这一指标将直接用于引导 Fuzzer 的变异能量调度

核心模块1：运行时监控与 Csource 识别（下）

➤ 设计目标：轻量级插桩与真实输入源点的动态捕获

➤ 在用户态模拟环境中，FirmAgent 执行高度定向的模糊测试与信息收集

1. 距离与字典引导的变异 (Directed Mutation)

- 利用预分析阶段提取的关键字字典系统性地替换输入请求中的参数，并为每个关键字附加自定义污点标签，以精确追踪数据流

2. 内存污点检测 (Memory Taint Detection)

- 为降低监控开销，FirmAgent 仅在预先计算的 Sink 范围内实施 QEMU 选择性插桩
- 当观察到某处内存状态从无污点转为受污染时，系统记录该指令地址，将其确认为真实输入源（Csource）

3. 间接调用解析 (Indirect Call Resolution): 通过 QEMU 插件动态捕获间接调用目标，修复静态控制流断层

核心模块2：污点传播代理 - 反编译代码修复

➤ **设计目标：修复反编译缺陷以构建准确的推理底座**

➤ 在利用 LLM 进行污点分析前，必须解决底层静态工具的反编译精度问题：

1. IDA 反编译的局限性

- 由于架构和上下文的限制，IDA 经常丢失关键的函数参数、忽略返回值赋值或构建错误的控制流结构例如，IDA 可能错误解析 `system()` 的签名，导致后续相关的污点传播关系断裂

2. LLM 驱动的代码精炼 (Decompiled Code Refinement)

- FirmAgent 利用大语言模型的语义上下文理解能力，对提取的潜在路径代码进行重建
- 提示 LLM 恢复有意义的变量名、推断隐式的字符串格式化行为及参数构造过程，从而为后续的污点分析提供语义正确的 C 代码表示

核心模块2：污点传播代理 - 语义追踪与告警验证

➤ 设计目标：跨过程的细粒度数据流推理与误报抑制

➤ 修复代码后，污点传播代理（Taint Propagation Agent）开始在函数级别执行精准的语义推理

1. 迭代式的跨过程分析：

- 向 LLM 提供精炼代码、Csource 和 Sink，要求其判定污点传播关系
- 对于跨函数传播，LLM 需识别调用处的受污染参数，并将这些参数作为目标子函数的初始污点源，持续迭代直至到达 Sink

2. 基于 **Few-shot** 的告警验证机制：为抑制 LLM 对固件防御逻辑理解不足产生的误报，FirmAgent 引入了二次验证模块

- 识别净化逻辑 (Sanitizer Processes)：剔除已被安全条件语句（如非法字符过滤）拦截的路径
- 识别系统文件源：确保读取自非用户可控环境（如 /etc/passwd）的数据不被误判为污点

核心模块3：PoC 生成代理

➤ 设计目标：融合动态可达性与静态语义约束的自动利用合成

➤ 一旦产生漏洞告警，PoC 生成代理负责自动构造可触发漏洞的实体网络报文

1. 输入约束提取

- 从 LLM 污点分析的过程中，隐式地提取到达 Sink 必需的语义前置条件（如特定的条件分支绕过、合法格式要求以及特定的 Payload）

2. 测试用例桥接与补全

- 模糊测试阶段已提供一个语法合法且能到达 Csource 的底层报文骨架（包含正确的 URL 与 HTTP Header）
- PoC 生成代理将这些 Fuzzing 测试用例与静态语义约束结合，推断相关参数所需的确切值
- 通过填充特定数据（如 NtpDstOffset: 'A'*300），组装成能够导致缓冲区溢出或命令注入的完整可用 PoC

实验评估与关键数据

- 在涵盖 7 个主流厂商的 14 款真实固件上，FirmAgent 展现了压倒性的优势

1. 漏洞挖掘成果

- FirmAgent 成功确认了 182 个真实漏洞（包括 45 个命令注入和 137 个缓冲区溢出）
- 其中，140 个为前所未有的 0-day 漏洞，并已有 17 个获得 CVE 编号

2. 精度与全面超越基线

- FirmAgent 的漏洞检测精度高达 91%，远远超过静态工具 HermeScan (33%)
- 其发现的漏洞数量分别是 HermeScan 的 2.6 倍
- 所有基线工具报告的漏洞均被 FirmAgent 完全覆盖

3. PoC自动化有效性

- 在生成的 182 个可用 PoC 中，有 167 个（91.8%）在没有任何人工干预的情况下直接成功触发了漏洞

文章评价

➤设计优势

- 把 Fuzzing 的真实可达性和 LLM 的语义推理结合，可有效辅助 PoC 生成

➤局限性分析

- 依赖仿真、QEMU 插桩、反编译和 LLM，多重因素使得PoC生成失败概率高

➤商业化潜力

- 更适合作为厂商内部红队/安全实验室平台进行漏洞研究与辅助PoC生成

Bond: Constraint-Directed Fuzzing for Automated Validation of Taint Analysis Results in Linux-based IoT Firmware

Jiaqian Peng^{†‡}, Puzhuo Liu^{*§¶☒}, Kai Cheng^{†☒}, Zhaoteng Yan[‡], Jie Liu[†]
Chengnian Sun^{||}, Hongsong Zhu^{†‡}*

[†] Institute of Information Engineering, Chinese Academy of Sciences

[‡] School of Cyber Security, University of Chinese Academy of Sciences

[§] Ant Group

[¶] Tsinghua University

^{||} University of Waterloo

USENIX Security 2026

内容摘要

➤ 关键动机

- 静态污点工具会产生大量潜在告警
- 但缺少外部入口、参数和路径约束，人工验证成本极高

➤ 本文工作：面向静态污点告警自动验证的约束驱动黑盒 Fuzzing 框架

1. 把不同工具输出的 Source、Sink 和调用路径统一预处理
2. 恢复外部入口点、请求参数和路径约束
3. 用约束引导黑盒 Fuzzing 在真实设备上确认漏洞

➤ 关键数据

- 在 19 台来自 8 个厂商的 IoT 设备上评估，输入为 SaTC、Mango、OctopusTaint、EmTaint 产生的 2,776 条污点报告，成功确认 1,349 条报告可在真实设备上触发，并发现 155 个此前未知漏洞

研究动机：静态污点告警的验证瓶颈

- 静态污点分析工具能够发现大量 source-sink 路径
 - SaTC、Mango、OctopusTaint、EmTaint 等工具已经能产生规模化告警
- 但这些路径往往只是“潜在漏洞”
 - 入口点可能不是真实外部可访问接口
 - 请求参数可能不完整
 - 路径约束可能无法满足
- 真实设备黑盒测试又缺乏覆盖率反馈，吞吐低、成本高
 - 需要把静态告警自动转化为可执行的真实设备验证任务

Table 1: Firmware Taint Analysis Tool Output Information.

Tool	Action	Taint Source		Sink	Path		Constraint
		CTS	ITS		Function	Block	
Arbiter [21]	✗	✗	✗	✓	✓	✗	✗
Karonte [2]	✗	✓	✗	✓	✓	✓	✗
SaTC [3]	✗	✗	✓	✓	✓	✗	✗
Mango [4]	✗	✗	✓	✓	✓	✗	✗
OctopusTaint [5]	✗	✗	✓	✓	✓	✓	✗
EmTaint [6]	✗	✓	✗	✓	✓	✗	✗

CTS: Classical taint sources that are interface library functions directly receiving user input (e.g., recv, read).
ITS: Inferred intermediate taint source [10], a custom function that processes user input received via library functions and returns a part of the input to be used by other functions (e.g., webGetVar).

核心洞察：从“潜在路径”到“约束化验证任务”

➤ 静态污点报告仍然有价值：

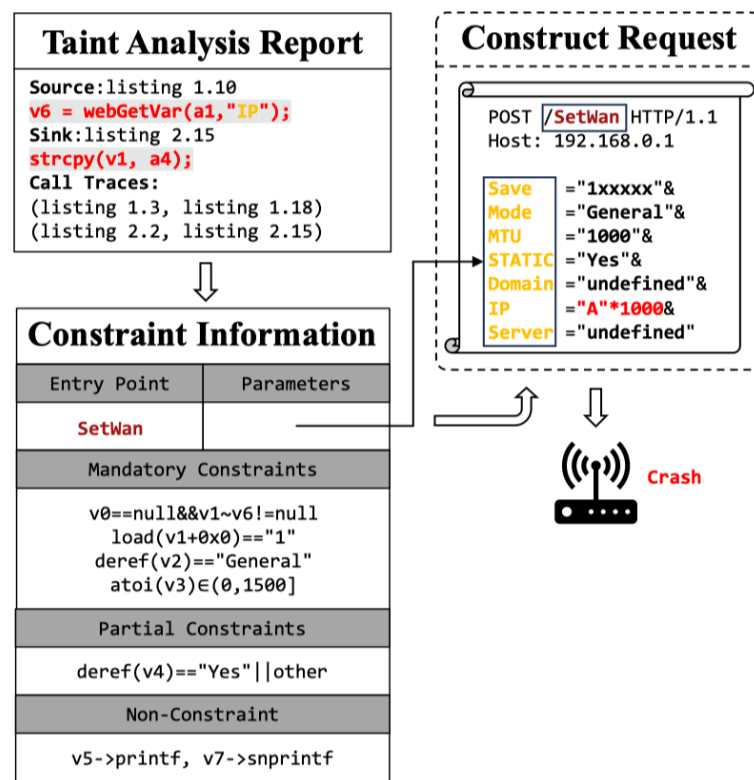
- 它给出了 source、sink 和可疑调用链，但缺少如何从外部请求触发这条路径的信息

➤ Bond 的关键转化：

1. 从告警路径反向恢复外部入口点
2. 在入口点到 sink 之间划分可达区域
3. 提取必须满足的路径约束
4. 生成协议合法的请求并进行定向黑盒 fuzzing

→ 用静态约束减少黑盒测试中的无效请求

把“静态告警验证”从人工分析问题变成“带入口、参数、路径条件的定向测试问题”，因此能显著降低告警分流成本



Bond 系统架构总览

1. 污点报告对齐与预处理

- 统一不同工具的 source、sink、call trace 输出
- 解包固件、恢复 CFG/CG、提取函数指针表等辅助信息

2. 入口点识别与可达区域划分

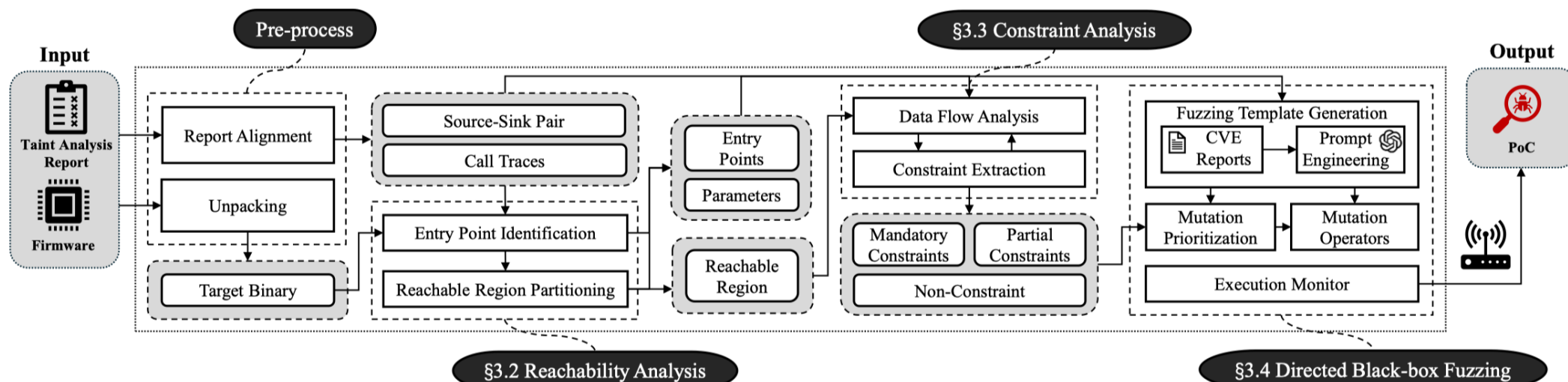
- 从 sink 或 source-sink 路径反向遍历
- 恢复 URI、关键字、注册函数、函数表到处理函数的映射

3. 约束分析

- 在可达区域内提取分支条件和库函数约束
- 将参数划分为必达约束、局部约束、非约束参数

4. 约束引导黑盒 Fuzzing

- LLM 辅助生成协议模板
- 按约束优先级进行层级变异



核心模块1：入口点识别与可达区域划分

- 设计目标：找出静态告警对应的外部可访问接口
- 入口分发结构：
 - 嵌套 strcmp 判断 URI 或 action 字段
 - 函数指针表分发
 - 初始化阶段注册关键字到处理函数的映射
- 识别策略：
 - 从告警中的 sink 或 source-sink 路径反向遍历 CFG/CG
 - 提取分发关键字或注册参数
 - 与前端文件中提取的关键词交叉验证
- 可达区域划分：
 - 只保留入口点到 sink 之间与当前告警相关的局部验证空间
 - 降低后续参数枚举和约束分析成本

Taint Analysis Report	
Source:listing 1.10 v6 = webGetVar(a1,"IP"); Sink:listing 2.15 strcpy(v1, a4); Call Traces: (listing 1.3, listing 1.18) (listing 2.2, listing 2.15)	

Constraint Information	
Entry Point	Parameters
SetWan	
Mandatory Constraints	
v0==null&&v1~v6!=null load(v1+0x0)== "1" deref(v2)== "General" atoi(v3)∈(0,1500]	
Partial Constraints	
deref(v4)== "Yes" other	
Non-Constraint	
v5->printf, v7->snprintf	

核心模块2：约束分析与层级变异

➤ 设计目标：让黑盒请求优先满足到达漏洞路径的关键条件

➤ 约束分类：

1. Mandatory constraints: 不满足就偏离漏洞路径，必须优先满足
2. Partial constraints: 影响局部路径选择，用于探索替代路径
3. Non-constraints: 不参与条件判断，降低变异优先级

➤ 语义分类：

- 字符串匹配、数值范围、字节级检查、空值要求、网络格式、启发式语义约束

➤ Fuzzing 策略：

- LLM 从公开 PoC、厂商公告、相似型号信息中生成协议模板
- 先满足 mandatory 约束，再逐步探索 partial 与非约束参数

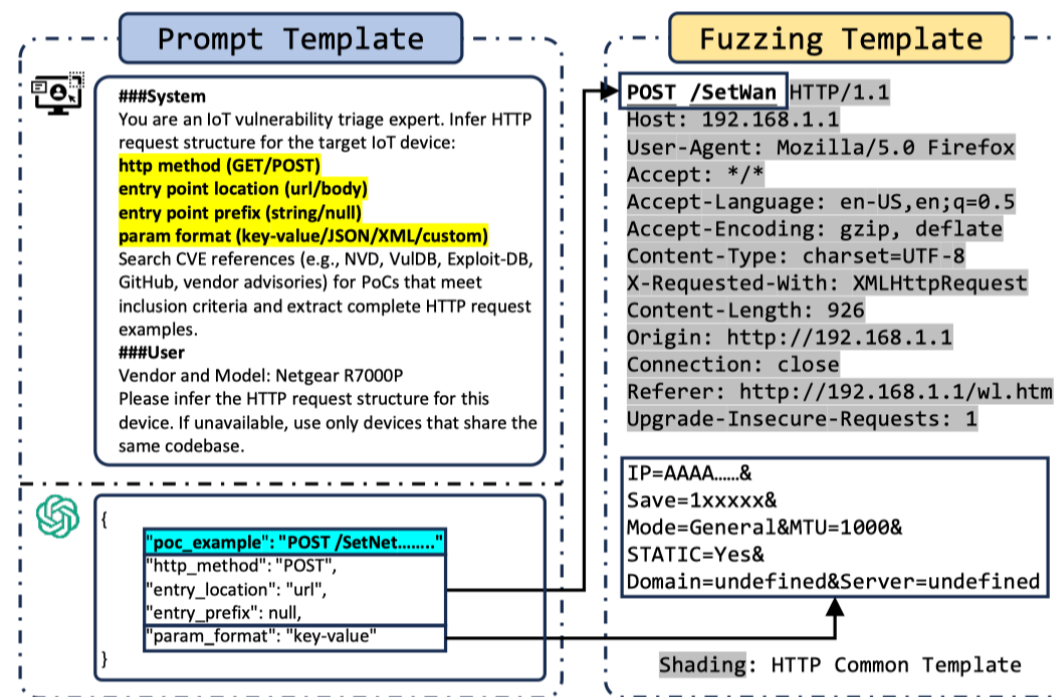


Figure 3: LLM-assisted fuzz template generation.

实验评估与关键数据

- 测试对象：19 台来自 8 个厂商的 IoT 设备
- 输入告警：
 - 来自 SaTC、Mango、OctopusTaint、EmTaint 的 2,776 条污点报告
- 验证结果：
 - 成功确认 1,349 条告警可在真实设备上触发
 - 发现 155 个此前未知漏洞，其中 108 个已获得 CVE/PSV 编号
- 已知漏洞召回：
 - 在 60 个已知漏洞数据集上，召回率 91.67%
 - 平均触发时间 5 分 53 秒
 - 相比现有 IoT fuzzing 工具，验证能力最高提升 5.5 倍

Vendor	Product	Type	Bug IDs
TP-Link	TL-WR841N	BoF	CVE-2025-53711 to CVE-2025-53715 4 unassigned
Linksys	E1700	BoF	CVE-2025-9525 to CVE-2025-9527
Linksys	E1700	CI	CVE-2025-9528
Linksys	RE6500	BoF	CVE-2025-9245 to CVE-2025-9253 CVE-2025-9355 to CVE-2025-9363 CVE-2025-9392 to CVE-2025-9393 CVE-2025-9481 to CVE-2025-9483 CVE-2025-8833, CVE-2025-14133 2 unassigned
Linksys	RE6500	CI	CVE-2025-5438 to CVE-2025-5447 CVE-2025-9244, CVE-2025-9575
D-Link	DIR816	BoF	CVE-2025-5623 to CVE-2025-5624 CVE-2025-5630, 1 unassigned
D-Link	DIR816	CI	1 unassigned
D-Link	DIR619L	BoF	CVE-2025-6114 to CVE-2025-6115 CVE-2025-6367 to CVE-2025-6374 CVE-2025-6614 to CVE-2025-6617 CVE-2025-55599, CVE-2025-55602 CVE-2025-55611, 8 unassigned
D-Link	DCS932L	CI	CVE-2025-5571
TOTOLink	A3002R	BoF	CVE-2025-6486 to CVE-2025-6487 7 unassigned
TOTOLink	A3002R	CI	CVE-2025-6485, 1 unassigned
TOTOLink	CA300-PoE	CI	CVE-2025-6618 to CVE-2025-6621 1 unassigned
Tenda	AX3	BoF	CVE-2025-55603, CVE-2025-55605 CVE-2025-55606
Tenda	O3V2	BoF	CVE-2025-7416 to CVE-2025-7423 CVE-2025-55613, 1 unassigned
Tenda	O3V2	CI	CVE-2025-7414 to CVE-2025-7415
Belkin	F9K1122	BoF	CVE-2025-7084 to CVE-2025-7094
Belkin	F9K1122	CI	CVE-2025-7081 to CVE-2025-7083
Netgear	R8500	BoF	PSV-2022-0181 to PSV-2022-0182 1 unassigned
Netgear	R7000P	BoF	CVE-2024-51004, CVE-2024-51013
Netgear	XR300	BoF	CVE-2024-51016
Netgear	WNDR4500	BoF	16 unassigned
Netgear	D6400	CI	CVE-2025-7407
Motorola	CX2L	CI	4 unassigned
Total	\	\	108 assigned, 47 unassigned

CVE: the vulnerability IDs assigned by CNAs (TP-Link Systems Inc., Netgear, VulDB).

PSV: the vulnerability IDs assigned by Netgear.

Unassigned: the vendor has confirmed the vulnerability but needs to fix it before releasing details.

文章评价

➤设计优势

- 把 Source-Sink 报告转成约束化黑盒验证任务，能显著降低人工分流成本

➤局限性分析

1. 依赖上游静态工具的漏洞报告质量
2. 复杂认证、强状态、多步骤业务流和非 HTTP 协议仍会限制验证成功率

➤商业化潜力

- 适合用于厂商内部的漏洞 “告警确认/回归验证”

Reproducing Web Application Vulnerabilities with Patch-Guided Routing Inference and Sink Exploration

*Youkun Shi, Yuan Zhang, Feng Xue, Jiarun Dai, Lei Zhang, Fengyu Liu, Bocheng Xiang,
Xiapu Luo*

ACM CCS 2026

内容摘要

➤ 关键动机

- PHP Web 漏洞复现中，安全补丁通常既不是外部入口也不是真正 Sink
- 传统 patch-based fuzzing 难以生成有效 PoC

➤ 本文工作：基于安全补丁引导的 Web 漏洞路由恢复、Sink 探索与 PoC 生成框架

1. 先解析安全补丁，定位相关文件、函数、分支和变量
2. 结合上述信息，让 LLM Agent 分析框架配置、控制器结构、注解路由和调用上下文，生成候选 URL，并在目标代码入口插入 reachability beacon，用运行反馈修正路由假设，迭代高可信度的候选URL
3. 动态修改分支条件进行 Forced Path Exploration，并在解释器层监控漏洞指标函数，以识别多层封装后的真实 Sink
4. 把可达 URL、参数、路径条件和 Sink 条件组装成 PoC 并真实执行验证

➤ 关键数据

- 在 140 个已知漏洞上生成 124 个 PoC
- 在 74 个 ground-truth 漏洞中复现 71 个，而现有技术 Predator 仅复现 14 个，提升 407.14%

研究背景：Patch 到 PoC 的自动复现问题

➤ 漏洞复现的目标：给定一个安全补丁，自动构造能够触发补丁前漏洞的 PoC

- 用于确认漏洞存在性、评估影响范围、验证补丁是否真正有效

➤ 传统 patch-based directed fuzzing 的思路：

- 将补丁位置作为目标，引导 fuzzer 寻找能到达漏洞路径的输入

➤ PHP Web 应用的特殊困难：

1. 外部入口不是文件路径，而是框架路由、控制器方法、注解、配置和重写规则的组合
2. 补丁常常位于上游校验逻辑，而真实危险 sink 隐藏在 DAO / DBAL / 模板引擎 / 框架封装之后

→ 仅以补丁位置为 fuzzing 目标，往往只能“到达补丁”，不能真正“复现漏洞”

核心洞察：补丁作为中间锚点

➤ **Insight 1:** 补丁不是入口，也不是 sink，而是连接二者的语义锚点 (line 10)

- 向上：需要从补丁位置恢复外部可访问 URL
- 向下：需要从补丁位置继续探索到真正危险操作

➤ **Insight 2:** 路由推断需要运行时反馈 (line 1, 4)

- 只看 HTTP 状态码无法判断是否真正到达补丁相关入口
- 需要在目标入口插入 reachability beacon，验证候选 URL 的可达性

➤ **Insight 3:** Sink 发现需要越过框架封装 (line 22)

- 危险操作可能被多层函数包装
- 需要在解释器层识别最终触发 SQLi / XSS / CMDi 的指标函数

→ P2PGen: 以补丁为锚点，向上恢复入口，向下探索 sink

CVE-2023-1578
SQL Injection in GitHub repository pimcore/pimcore prior to 10.5.19.

a) CVE Description

```
_pimcore_search:  
  resource: "@PimcoreAdminBundle/Controller/Searchadmin/"  
  type: annotation  
  prefix: /admin/search  
  options:  
    expose: true
```

b) Routing File: Resources/config/pimcore/routing.yml

```
1 /*@Route("/search")*/  
2 class SearchController extends AdminController {  
3   ...  
4   /*@Route("/find", name="search_find",  
5     methods={"GET", "POST"})*/  
6   public function findAction(Request $request, ... ) {  
7     echo "reach_e238151d42ca1ccf8659e3deae61b014";  
8     // The Inserted Beacon  
9     ...  
10    $allParams = $request->request->all();  
11    ...  
12    $types = explode(',', preg_replace("/[^a-z,]/i", "",  
13      $allParams['type'] ?? ""));  
14    $conditionParts = [];  
15    $searcherList = new Data\Listing();  
16    ...  
17    if (is_array($types) and !empty($types[0])) {  
18      $conditionTypeParts = [];  
19      foreach ($types as $type)  
20        $conditionTypeParts[] = $db->quote($type);  
21      $conditionParts[] = '( maintype IN (' . implode(',',  
22        $conditionTypeParts) . '))';  
23    }  
24    ...  
25    if (count($conditionParts) > 0)  
26      $condition = implode(' AND ', $conditionParts);  
27    ...  
28    $hits = $searcherList->load($condition); // Vulnerability Sink  
29    ...  
30  }
```

c) Security Patch: Controller/Searchadmin/SearchController.php

把补丁当成入口与 Sink 之间的中间路标，同时解决“怎么进来”和“触发哪里”两个 PoC 生成难题

P2PGen 系统架构总览

1. Patch 分析与目标定位

- 解析安全补丁，定位补丁相关文件、函数、分支和变量
- 将补丁位置作为后续入口恢复与 sink 探索的中间锚点

2. LLM 驱动的路由推断

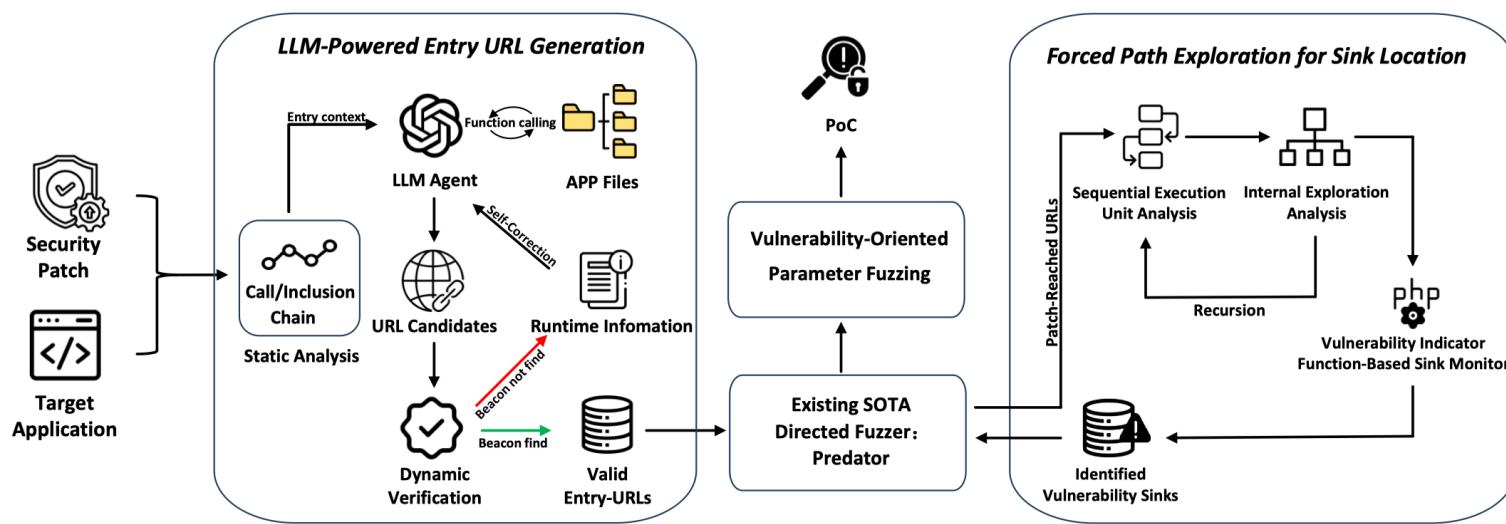
- 分析控制器结构、框架配置、注解路由和调用上下文
- 生成候选 URL，并用 reachability beacon 反馈修正

3. 强制路径探索与 sink 监控

- 从补丁位置向下游探索不同执行路径
- 在解释器层监控漏洞指标函数，确认真实危险操作

4. PoC 生成与验证

- 将URL、参数、路径条件和 sink 触发条件组合为 PoC
- 通过真实执行确认 PoC 是否能复现漏洞



核心模块1：LLM 驱动的路由推断

➤ 设计目标：从补丁相关代码恢复外部可访问 URL

→ 解决“补丁文件不等于外部入口”的问题

1. 输入信息：

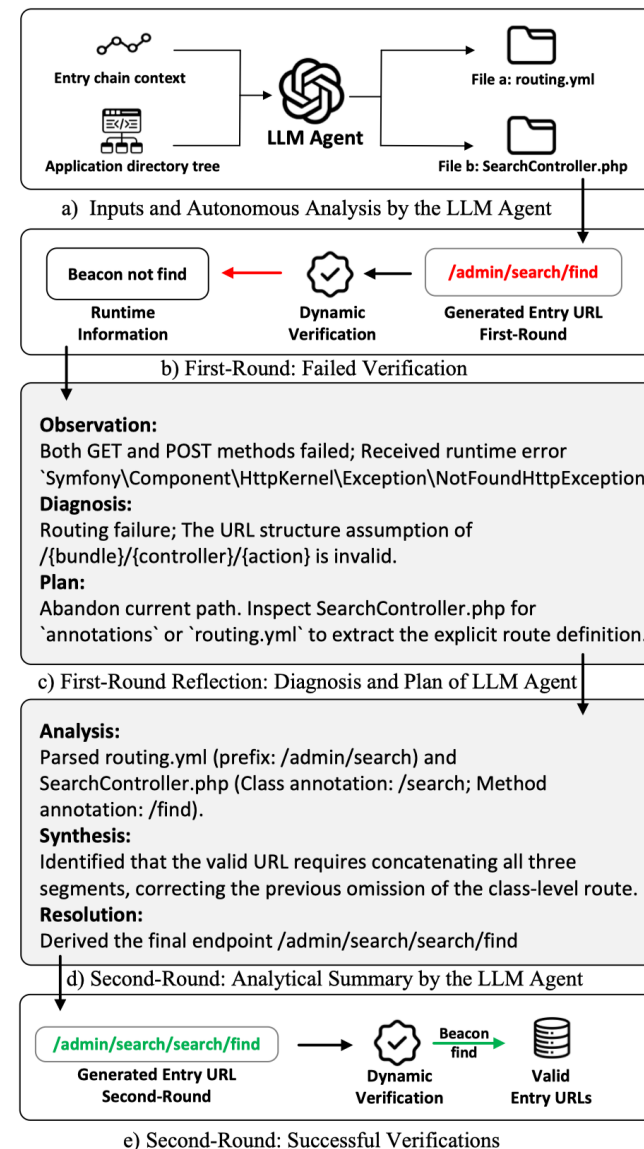
- 补丁文件和应用程序源码

2. 基于 LLM Agent 的可达URL生成：

- 理解框架代码与配置语义，生成候选 URL 与必要路径参数
- 根据运行反馈不断修正路由假设

3. 可达性信标：

- 在目标代码入口处插入可达性标记
- 若候选请求触发 beacon，则说明 URL 真正到达补丁相关入口
- 比仅依赖 HTTP 200 / 404 状态码更可靠



核心模块2：强制路径探索与 Sink 监控

➤ 设计目标：从补丁位置继续向下游找到真正危险操作

1. 强制路径探索：

- 从补丁位置向下游执行时，动态修改分支条件
- 使不同路径能够被确定性遍历
- 将程序逻辑拆分为顺序执行单元，逐段探索，缓解路径爆炸

2. 解释器级 sink 监控：

- 监控 SQLi、XSS、CMDi 等漏洞函数
- 即使危险操作被框架函数多层封装，仍能识别最终 sink



```
1 Controller/Searchadmin/SearchController.php:load()
2   → models/Search/Backend/Data/Listing/Dao.php:load()
3   → lib/Doctrine/DBAL/Connection.php:fetchAll()
4   → lib/Db/PimcoreExtensionsTrait.php:executeQuery()
5   → lib/Doctrine/DBAL/Connection.php:executeQuery()
6   → lib/Doctrine/DBAL/Driver/PDOConnection.php:query()
```

a) PHP Layer: Call Stack of the Vulnerability Sink of CVE-2023-1578

```
1 static PHP_METHOD(PDO, query){
2     // Parse SQL string
3     zend_parse_parameters(1, "s", &statement, &statement_len);
4     // Prepare SQL (driver-specific)
5     dbh->methods->preparer(dbh, statement, statement_len, stmt, NULL);
6     ...
7     trace_id = get_http_header("HTTP_X_TRACE_ID");
8     zend_execute_data *f = EG(current_execute_data);
9     char trace[2048], *p = trace;
10    for (; f; f = f->prev_execute_data) {
11        p += sprintf(p, "%s:%d",
12                    ZSTR_VAL(f->func->op_array.filename), f->opline->lineno);
13    }
14    snprintf(path, sizeof(path), "/tmp/%s.cov", trace_id);
15    FILE *fp = fopen(path, "a");
16    if (fp) { fprintf(fp, "Call Stack:%s", trace); } // Record Call Stack
17    ...
18    // Execute SQL Statement
19    stmt->methods->executer(stmt);
20    ...
21 }
```

b) C Layer: Vulnerability Indicator Function Monitor for PHP SQLi Sinks

实验评估与关键数据

- 数据集：15 个流行 PHP 应用， 140 个已知漏洞
- PoC 生成能力：
 - 成功为 124 个漏洞生成 PoC
 - 整体生成率达到 88.57%
- Ground-truth 复现能力：
 - 在 74 个具有公开 PoC 的漏洞中成功复现 71 个
 - 复现成功率达到 95.95%
- 与 directed web fuzzer Predator 对比：
 - Predator 复现 14 个， P2PGen 复现 71 个
 - 提升幅度为 407.14%

Applications	Stars	Version	LoC	CVEs	Avail. PoCs
OpenEMR	4.6k	5.0.1	1,139,760	23	12
LibreNMS	4.5k	22.10.0	195,139	12	11
Microweber	3.4k	1.2.9	214,252	12	10
Pimcore	3.7k	6.9.6	218,392	11	8
phpMyAdmin	7.7k	4.6.0	208,528	10	1
Cacti	1.8k	1.2.22	157,376	10	10
GLPI	5.4k	9.1	165,265	10	1
EcommerceCB	1.4k	7c3c32d	70,909	10	2
Dolibarr	6.7k	8.0.2	719,716	10	1
Piwigo	3.6k	2.9.2	153,004	8	5
osTicket	3.6k	2.9.2	124,674	6	4
MantisBT	1.7k	2.1.0	65,753	5	4
phpIPAM	2.6k	1.3.2	71,709	5	5
PrestaShop	8.9k	1.7.8.1	382,559	4	0
MISP	6.0k	2.4.30	27,231	4	0
Total	-	-	3,914,267	140	74 (52.86%)

Variants	Reproduced Vulns	Recall (%)	Imprv.(%)
Predator	14 / 74	18.92%	-
Predator _{Entry}	43 / 74	58.11%	+207.14%
Predator _{Entry+FPE}	66 / 74	89.19%	+53.49%
P2PGen	71 / 74	95.95%	+7.58%

文章评价

➤设计优势

- 把补丁作为入口和 Sink 之间的语义锚点，路由恢复与下游 Sink 探索结合
- 适合自动复现 Web CVE

➤局限性分析

1. 主要面向 PHP Web 应用，要求源码、补丁和可运行环境
2. 对网络设备二进制固件的直接适用性有限

➤商业化潜力

- 适合用于厂商内部的补丁验证和漏洞复现平台的构建

PHPBench: Automated Generation of Verifiable and Hierarchical Benchmarks for PHP Web Fuzzing

Youkun Shi, Yuan Zhang, Lei Zhang, Jiarun Dai, Tianhao Bai, Fengyu Liu, Bocheng Xiang, Xiapu Luo, Min Yang

ACM CCS 2026

内容摘要

➤ 关键动机

- PHP Web Fuzzing 缺少统一、真实、分层且可验证的 benchmark
- 现有教学靶场和自选 CVE 容易夸大工具能力

➤ 本文工作：面向 PHP Web Fuzzing 的可验证、可分层 Benchmark 自动生成框架

1. 将漏洞原因建模为 Source、Sink 和 Flawed Protections，并通过系统性削弱控制流检查、数据流净化和过滤逻辑生成分层变体
2. 在真实 PHP 应用中动态探索用户工作流，用 malicious probe 回放请求，寻找用户输入仍然活跃且位置足够深入的插入点
3. 插入后自动适配或合成 PoC，只有能被 PoC 确认触发、且不破坏原应用功能的漏洞才进入最终 benchmark

➤ 关键数据

- 用 28 个真实漏洞原因生成 375 个唯一变体，并插入真实应用，构建基准数据集
- 在评估 7 个主流现有 Web Fuzzer 后，发现其最佳召回率仅 28.45%，仍有显著提升空间

研究背景：Web Fuzzing 缺少可信基准

➤ Web fuzzing 工具长期缺乏统一可靠的 benchmark

➤ 现有评测方式存在明显偏差：

1. 教学靶场

- 漏洞模式简单，无法代表真实应用中的路由、状态和过滤逻辑

2. 自选历史 CVE

- 容易受应用版本、漏洞类型和选择标准影响

3. 聚合基线工具结果

- 会天然遗漏所有工具都找不到的困难漏洞
- 也可能把无效报告误当作 ground truth

→ 需要自动化、真实性、层次性、可验证性兼备的评测基准

核心洞察：Benchmark 也需要“可利用性证明”

➤ **Insight 1:** 真实应用比教学靶场复杂得多

- 路由、鉴权、状态、过滤、业务 workflows 都会影响漏洞可达性

➤ **Insight 2:** 漏洞样本需要难度层级

- 如果样本都太简单，工具之间难以拉开差距
- 如果样本都太难，又无法观察渐进式能力提升

➤ **Insight 3:** 插入漏洞不等于可利用漏洞

- 插入代码可能不可达，或破坏原应用逻辑
- 因此每个样本都必须配套自动 PoC 作为可利用性证明

把“生成漏洞”和“证明可利用”绑定，只有能被自动 PoC 触发且不破坏原应用功能的样本才被纳入基准测试集，避免不可达或失真的样本污染评测

PHPBench 系统架构总览

1. 漏洞原因建模

- 从真实漏洞中抽象 source、sink 和 flawed protections
- 将漏洞原因表示为可系统变形的结构

2. 分层漏洞变体生成

- 系统性削弱控制流检查、数据流净化和过滤逻辑
- 生成不同难度层级的漏洞变体

3. 真实应用插入点选择

- 动态探索真实用户 workflows
- 寻找用户输入活跃、位置深入、外部可触达的插入点

4. 自动 PoC 合成与验证

- 自动适配或合成 PoC
- 只有能被 PoC 触发的漏洞进入最终 benchmark

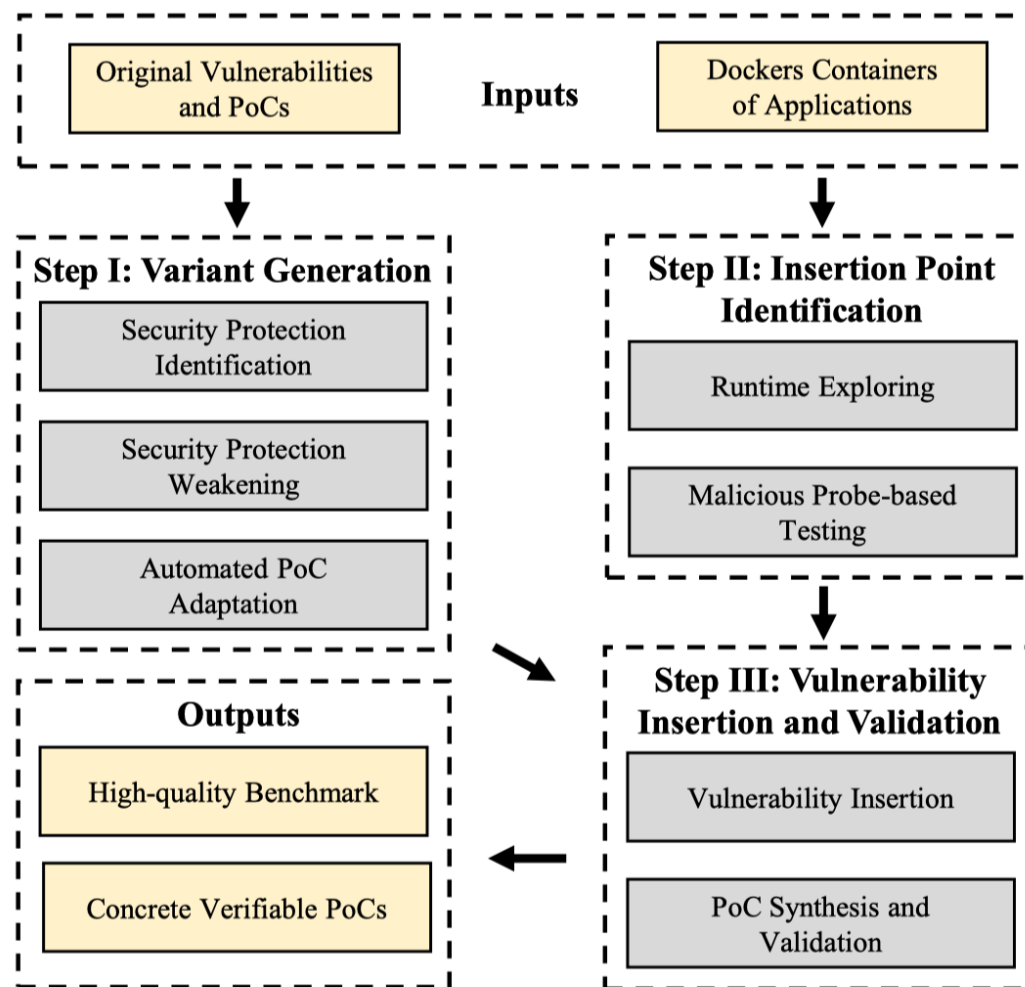


Figure 1: The workflow of PHPBench.

核心模块1： 分层漏洞变体生成

➤ 设计目标： 让 Benchmark 能区分工具处理不同难度漏洞的能力

➤ 漏洞原因表示：

- Source: 用户可控输入
- Sink: SQLi / XSS / CMDi 等危险操作
- Flawed protections: 有缺陷的过滤、净化或控制流检查



$$V = \langle S, K, \mathbb{P} \rangle$$

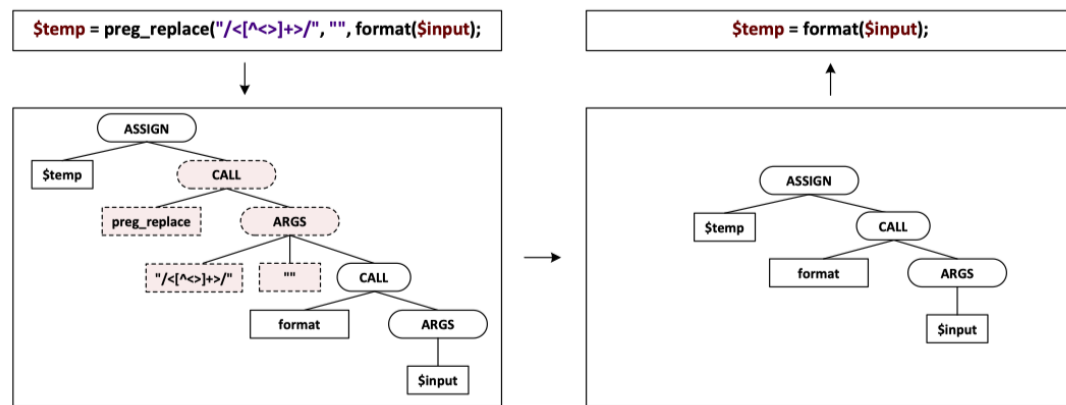
- S is the program point representing the *Source* where untrusted data controlled by users.
- K is the program point representing the *Sink* where a security-sensitive function is performed.
- $\mathbb{P} = \{p_1, p_2, \dots, p_n\}$ is the set of flawed security protections applied to the path from S to K . This set can be empty ($\mathbb{P} = \emptyset$), indicating a complete lack of protection.

Let V_A and V_B be two vulnerabilities with root causes $V_A = \langle S_A, K_A, \mathbb{P}_A \rangle$ and $V_B = \langle S_B, K_B, \mathbb{P}_B \rangle$ respectively. We state that V_A is a **hierarchical variant** of V_B , denoted as $V_A \leq V_B$, if they share the same source and sink, and the security protections for V_A are a subset of those for V_B . Formally:

$$V_A \leq V_B \iff (S_A = S_B) \wedge (K_A = K_B) \wedge (\mathbb{P}_A \subseteq \mathbb{P}_B)$$

➤ 变体生成策略：

- 从真实漏洞原因出发，系统性削弱保护逻辑
- 生成从“几乎无保护”到“多层但有缺陷保护”的分层样本



➤ 设计优势：

- 避免教学靶场过于简单，也避免所有样本都难到无法触发
- 让工具能力差异可以被细粒度度量

核心模块2：插入点选择与 PoC 验证

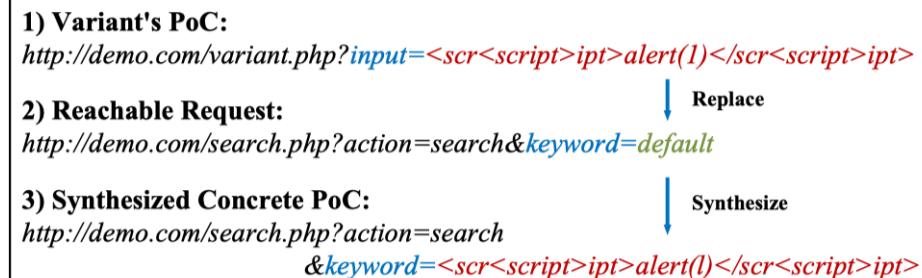
➤ 设计目标：保证生成漏洞既真实、又可达、又不破坏原应用逻辑

1. 面向真实应用的插入点选择：

- 先通过动态探索发现真实用户 workflows
- 再用 malicious probe 回放请求，寻找用户输入仍然活跃的位置
- 插入点需要位于真实功能路径中，且足够深入（选取活跃的最深位置）

2. 自动 PoC 合成：

- 对每个生成并插入后的漏洞自动适配或合成 PoC
- 只有可被 PoC 确认触发的样本才进入最终 benchmark



实验评估与启发

➤ 基准构造结果：

- 使用 28 个高质量真实漏洞原因生成 427 个分层变体
- 其中 375 个为唯一变体
- 插入 10 个流行 PHP 应用，覆盖 197 个不同功能

➤ 可验证性：

- 最终保留的每个插入漏洞都能被自动 PoC 确认触发
- 人工检查确认原应用功能完整性保持 100%

➤ 工具评估结果：

- 评估 7 个现有 fuzzing/scanning 工具，最好工具召回率也只有 28.45%
 - 传统 ad-hoc 评测会显著夸大工具能力
- 在 DVWA 上可达 100% 召回，但在 PHPBench 上只有 4.53%

Table 2: Breakdown of results from each step of the application-targeted insertion stage across our dataset.

Applications	LoC	Insertion Point Identification			Quality of Insertion Points		Vulnerability Insertion & Verification	
		Collected Requests	Qualified Requests	Insertion Points	Functional Diversity	Execution Trace Depth (Avg.)	Vulnerability Insertion	<Functionality, PoC> Verification (Pass Rate)
WordPress	587,187	8,412	435	145	80	3560.53 (71.87%)	80 / 145	<46, 80> (100.00%)
OpenEMR	1,788,910	101,036	226	96	41	2318.01 (55.81%)	89 / 96	<37, 89> (100.00%)
Joomla	564,737	12,521	642	40	21	1856.92 (63.66%)	27 / 40	<12, 27> (100.00%)
PrestaShop	2,044,183	25,551	332	56	39	4909.09 (74.97%)	19 / 56	<10, 19> (100.00%)
Drupal	56,847	21,730	963	27	20	50.11 (92.10%)	26 / 27	<19, 26> (100.00%)
Piwigo	244,270	5,104	226	33	22	414.62 (56.13%)	20 / 33	<12, 20> (100.00%)
MyBB	161,351	7,314	386	46	28	572.36 (75.76%)	42 / 46	<26, 42> (100.00%)
Hosp. Mgmt.	14,352	4,649	73	44	20	11.39 (95.71%)	36 / 44	<14, 36> (100.00%)
Doctor Appt.	4,530	5,274	33	15	9	6.73 (72.60%)	14 / 15	<8, 14> (100.00%)
Login Mgmt.	11,074	4,319	30	22	13	12.5 (96.53%)	22 / 22	<13, 22> (100.00%)
Total	5,477,441	195,910	3,346	524	293	1972.33 (63.33%)	375 / 524	<197, 375> (100.00%)

Table 3: Performance comparison of existing work on the constructed benchmark. Values denote mean ($\pm$ standard).

Tool	Reached Insertion Points		Detected Vulnerabilities	
	Num. ($\pm$ std)	Recall (%)	Num. ($\pm$ std)	Recall (%)
webFuzz+	130.00 (± 2.00)	34.67%	26.67 (± 3.21)	7.11%
Witcher+	214.33 (± 10.97)	57.15%	41.33 (± 3.51)	11.02%
PHUZZ	90.67 (± 2.52)	24.18%	17.00 (± 1.00)	4.53%
Arachni	151.67 (± 10.69)	40.45%	43.00 (± 6.93)	11.47%
w3af	166.33 (± 1.15)	44.35%	33.33 (± 0.58)	8.89%
ZAP	119.67 (± 4.62)	31.91%	13.00 (± 0.00)	3.47%
Wapiti	199.67 (± 6.66)	53.25%	72.67 (± 0.58)	19.38%
Wapiti+	199.67 (± 6.66)	53.25%	106.67 (± 2.08)	28.45%
Avg.	159.00	42.40%	44.21	11.79%

文章评价

➤设计优势

- 把基准数据集的真实性、层次性和可验证性相结合
- 每个漏洞样本都有 PoC 证明，评测可信度明显高于教学靶场

➤局限性分析

1. 自动插入漏洞仍不完全等同自然漏洞
2. 范围集中在 PHP Web，不能直接代表 IoT 固件二进制或多语言后端漏洞

➤商业化潜力

- 适合作为厂商漏洞检测产品评测和回归测试基准

维度四总结：漏洞确认与利用生成

静态分析误报缓解问题

NÜWA, 【USENIX SEC 2025】

跳过传统底层污点追踪，创新提出对比前/后端语义不一致性(语义约束差异)作为漏洞存在的确认信号

动态技术基准建设

PHPBench, 【ACM CCS 2026】

构建Web Fuzzing领域自动化、真实性、层次性、可验证性兼备的评测基准，用于评估现有技术

动态分析漏洞确认问题

LABRADOR, 【IEEE S&P 2024】

引入基于网络响应解析的距离导向黑盒Fuzzing，提升代码覆盖率与执行轨迹还原

FirmAgent, 【NDSS 2026】

利用Fuzzing收集真实输入，再由LLM进行+语义推理，自动化生成漏洞PoC

Bond, 【USENIX SEC 2026】

将静态污点告警转化为约束化验证任务，在真实设备上确认告警能否被触发

P2PGen, 【ACM CCS 2026】

基于给定的漏洞补丁，自动化生成漏洞的PoCs用于验证补丁有效性和完整性

缓解措施：从漏洞发现技术到设备防护策略

- 维度1：建立接口与参数资产清单
- 维度2：跨组件数据流建立安全边界
- 维度3：隐藏接口治理与认证
- 维度4：用“约束一致性”主动式发现漏洞



防御维度1：建立接口与参数资产清单

1. 统一接口注册

- 所有 Web handler、CGI、Lua handler、REST API 都必须进入中心注册表
- 每个接口明确声明 URI、Method、认证要求、权限等级、参数列表
- 未注册接口不得进入发布固件

2. URI-Key 规格化管理

- 为每个参数定义类型、长度、范围、枚举值、编码格式
- 参数规格同时用于前端页面、后端校验、测试用例生成

3. 输入源点显式标注

- 对 websGetVar、getenv、cJSON_GetObjectItem、Lua table 访问等输入函数做统一封装
- 标注哪些 Source 来自外部用户，哪些来自系统内部
- 对用户可控 Source 默认施加强校验

4. 发布前检查

- 比对前端提取的参数、后端实际解析的参数、接口注册表
- 发现“后端存在但前端无入口”的参数时，强制进入安全审查

Source不应让攻击者从二进制中逆向发现，而应该成为厂商可管理、可审计的资产

防御维度2：跨组件数据流建立安全边界

1. 对 IPC 数据键做分级

- NVRAM key、环境变量、临时文件路径、Socket 消息字段都应纳入数据资产
- 标注哪些 key 可能被 Web 请求间接控制
- 高风险 key 禁止直接进入命令执行等危险函数

2. Setter / Getter 双向约束

- Setter 写入前做格式化和净化
- Getter 读取后仍要重新验证，不能默认信任存储层数据
- 对跨进程传播的数据保留“用户可控”语义

3. 跨语言调用边界防护

- Lua 调 C、C 调 Lua、脚本调用二进制时，必须明确参数 schema
- 禁止 Lua 中拼接命令后直接传给 C 二进制或系统命令
- 对 os.execute、forkExec、system 等接口建立安全包装层

4. 组件最小权限

- Web 服务进程、配置服务、系统命令执行服务分离权限
- 低权限 Web 组件不应直接修改系统级配置
- 高危操作通过受控 API 完成，而不是通过 shell 字符串拼接

在每一次跨边界数据传输过程中均设立安全约束，防止恶意数据流动

防御维度3：隐藏接口治理与认证

1. 消除未注册路由 Token

- 所有 action、cmd、op、method、service name 等路由 Token 必须登记
- 构建阶段扫描二进制和脚本中的 Token
- 发现注册表之外的 Token，直接阻断发布

2. 调试接口发布门禁

- test、debug、diagnostic、factory、dev、backup 等接口不得进入正式版本
- 必须保留的诊断接口应绑定认证、权限、审计日志和访问来源限制

3. 隐藏接口统一认证

- 路由分发前先做认证和授权，而不是进入 handler 后再判断
- 敏感操作必须验证角色权限和 CSRF 防护

4. 降低自动化枚举价值

- 未授权、参数错误、不存在接口返回一致化响应
- 不在响应中泄露内部路径、函数名、固件版本、调试信息
- 对大量路径探测、Token 枚举、异常参数组合做速率限制和告警

攻击者利用路由Token推断隐藏接口；厂商可反向使用Token扫描消除未登记入口

防御维度4：用“约束一致性”主动式发现漏洞

1. 三类约束统一建模

- 前端显式约束：maxlength、正则校验、下拉枚举、JS 检查等
- 后端显式约束：长度检查、字符过滤、路径规范化、权限判断等
- Sink 期望约束：缓冲区容量、命令参数字符集、文件路径边界等

2. 识别约束不一致

- 前端限制长度，但后端未限制 → 高风险
- 后端检查长度，但未检查命令分隔符 → 命令注入风险
- Sink 需要固定大小缓冲区，但 Source 可传入超长数据 → 溢出风险
- 前端隐藏参数，但后端仍可解析 → 隐藏输入风险

3. 自动生成验证用例

- 从 URI-Key 清单生成合法请求骨架
- 从 Source-Sink 路径提取关键参数
- 从约束缺口生成漏洞验证用例

4. 修复后回归

- 每个已确认漏洞转化为回归测试
- 后续固件版本必须重复验证同一路径
- 防止补丁只修前端、不修后端

将“漏洞验证能力”前移到发布前，而不是等漏洞被披露后再采取缓解措施

其他缓解措施：默认配置安全

1. 默认关闭 WAN 侧管理入口
2. 管理面仅允许 LAN、VPN 或可信管理通道访问
3. 禁止默认弱口令，每台设备使用唯一初始凭据
4. 对接口枚举、爆破、异常参数组合做限速和告警
5. 所有敏感接口必须有认证、授权、日志和速率限制

增加攻击者发现入口成本，且即使发现入口，也无法绕过认证、授权和输入约束

缓解措施总结：建立内部安全流水线

1. 固件解包与资产提取

1. 提取 HTML / JS / Lua / PHP / CGI / ELF / shared library
2. 生成 URI、Key、Token、NVRAM key、Sink 函数清单

2. Source 识别

1. 前后端关键词匹配、URI-Key 结构化恢复
2. LLM 辅助判断输入处理函数语义

3. Propagation 追踪

1. 分析 NVRAM、环境变量、文件、Socket 等 IPC
2. 建立跨二进制依赖图
3. 建立 C-Lua / 脚本-二进制跨语言数据流

4. Hidden Interface 检测

- 从文件系统和二进制中提取路径和路由 Token
- 对未登记接口做黑盒探测，恢复隐藏接口的参数规格

5. Sink 与约束分析

- 检查 system、popen、execve 等危险操作
- 比对前端约束、后端约束和 Sink 期望约束
- 输出高置信漏洞候选

6. 动态验证与回归

- 根据 URI-Key 和 Token 字典生成 Fuzzing 请求
- 在真实设备或仿真环境中验证可达性

